

When developing a CBR retrieval system using myCBR, the first step is determining the case attributes. Cases are represented by a coupled vector of attributes $\text{Case} = [\text{Problem attributes}, \text{Solution Attributes}]$. The problem attributes are used to measure the similarity of a target case and the cases in a case base. The solution attributes are stored as part of the solution information for the cases.

Once the attributes have been determined, the second step is the local similarity assignation to each problem attribute. MyCBR provides several options to compute similarity for each problem attribute. Within the available options, three similarity functions have been used for the current research:

1. Integer/Float similarity: this similarity function is used for numeric attributes. The similarity is obtained by a difference between a reference value and the input values of the function. A mathematical function is needed to define how the similarity decreases as the input values get further from the reference value. This mathematical function can be linear, exponential or determined by discrete points in the Cartesian plane. Figure 5.3 shows the procedure to edit simple integer similarity functions in MyCBR Workbench [Alt+12].
2. Symbol: this similarity measure is advisable for variables with a fixed set of options. These options are organized in a similarity matrix and some numerical values are given to establish the similarity among the options. Figure 5.4 presents a generic example of a symbol similarity matrix and how it is managed in MyCBR Workbench. This similarity function has been modified for some attributes in the current research to interact with the ontology model. The similarity matrix values are automatically obtained from the classes and relations of an ontology using feature-based similarity approach proposed by [Sán+12]. This feature-based similarity between two terms a and b is equal to 1 minus the normalized dissimilarity between a and b , as it is shown in equation 5.1.

$$\text{sim}_{\text{norm}}(a,b) = 1 - \log_2 \left(1 + \frac{|\phi(a) \setminus \phi(b)| + |\phi(b) \setminus \phi(a)|}{|\phi(a) \setminus \phi(b)| + |\phi(b) \setminus \phi(a)| + |\phi(b) \cap \phi(a)|} \right) \quad (5.1)$$

where $|\phi(a) \setminus \phi(b)|$ are the features of a that are not present in b , $|\phi(b) \setminus \phi(a)|$ are the features of b that are not present in a , and $|\phi(b) \cap \phi(a)|$ are the features that both a and b have in common.

3. String: attributes similarity is obtained based on open text strings. Unlike the `Symbol` type attribute

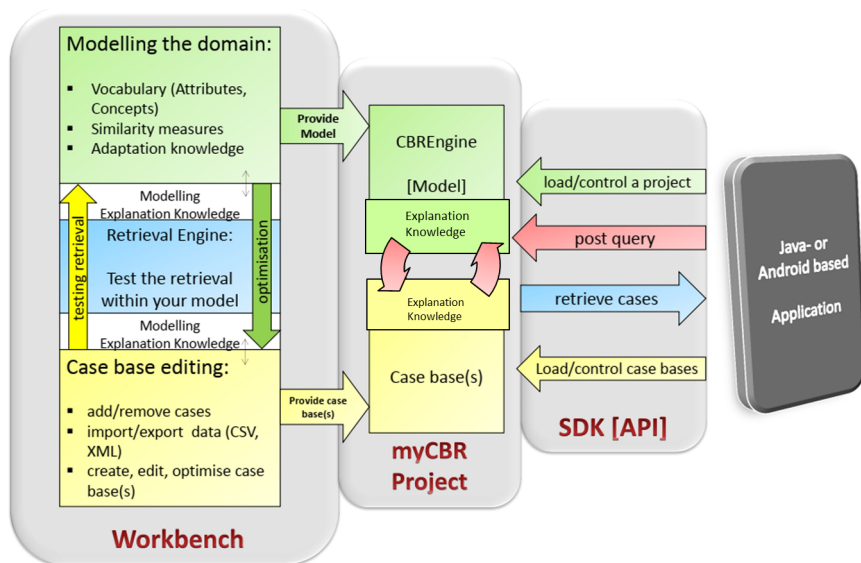


Figure 5.2: MyCBR platform description, [Alt+12]

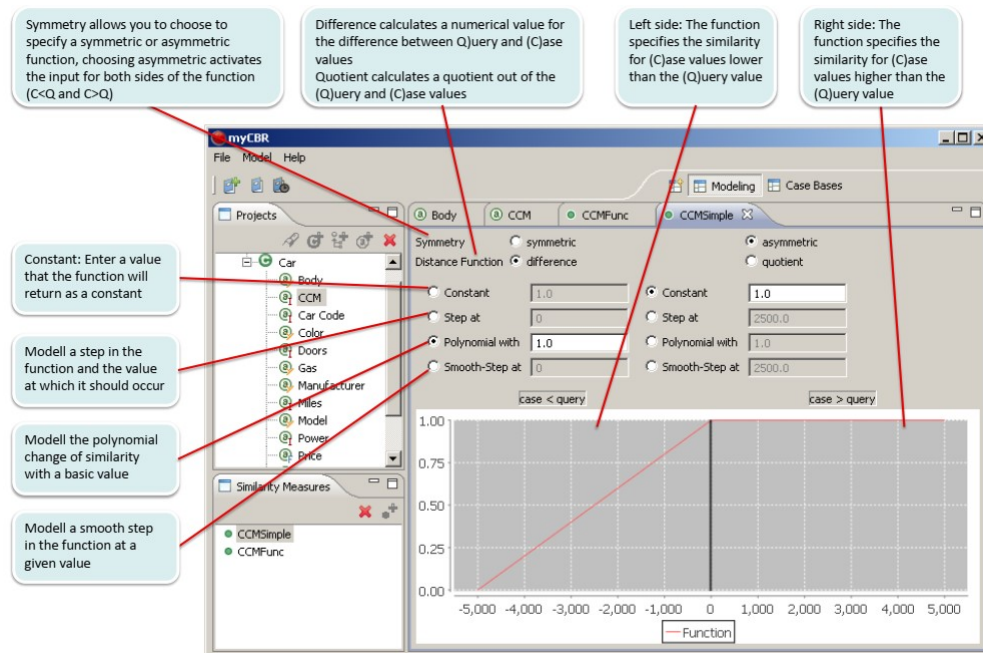


Figure 5.3: Simple integer similarity example, [Alt+12]

that has a certain number of categories, String type has only the restriction of having sentences or words as input. MyCBR offers three options to compute string-based similarity which are: Equality, Ngram and Levenshtein. For the current case study, the Levenshtein function was selected to compute the similarity string-based attributes. Levenshtein function offers a flexible means to compute similarity based on each character of the string [Lev66]. This is especially useful when there is a vast set of options that are unknown when building similarities. It also tolerates little orthographic errors from the user when typing the maintainable item of the target case. MyCBR offers string-based similarity only in its SDK; this similarity function is not yet available in MyCBR workbench.

The CBR system developed in this research was developed using the MyCBR SDK. The SDK was privileged over the Workbench for three main reasons:

1. String-based similarity functions are only available in the SDK.
2. The SDK provides access to modify the similarity functions. This allows the implementation of ontology-based similarity which is not originally included in MyCBR.
3. The SDK provides a flexible means to integrate the developed CBR retrieval solutions with other components of a Decisions Support System (DSS).

The complete development of the retrieval engine using the SDK and its integration with other parts of the DSS has been documented in a JavaDoc HTML and in the coding guide in Appendix C

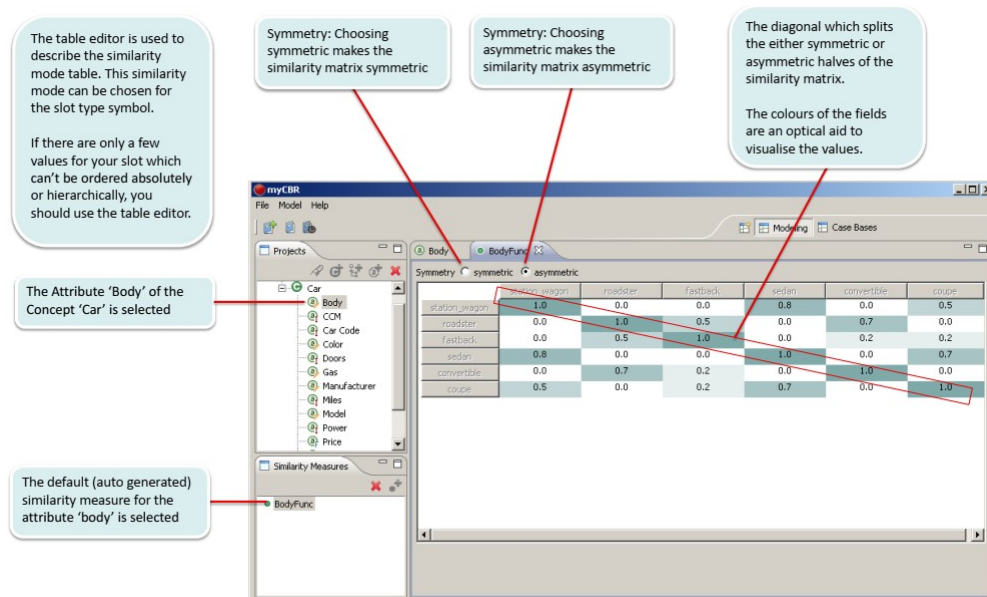


Figure 5.4: Symbol similarity matrix example, [Alt+12]

5.3 Development of the retrieval engine for predictive maintenance components selection

While the previous section provided the theoretical background of Case-Based Reasoning (CBR), this section explains the development of the retrieval engine for the predictive maintenance (PdM) component selection which is part of the DSS proposed in this thesis framework.

5.3.1 Case representation

"A case is a piece of knowledge in a particular context representing an experience that teaches an essential lesson to reach the goal of the reasoner" [Kol93]. Cases are often represented in a coupled form [problem, solution], in which similarities are applied on the "problem" part so that the "solution" part is retrieved. Case representation is composed of three main parts [BKP05]:

1. Defining the attributes of the case.
2. Defining the structure for the case content.
3. Organizing the case base.

5.3.1.1 Defining the attributes of the case

For the use case of predictive maintenance design, the cases are composed of three main attributes and some other complementary attributes. A case consists of a model that fulfils a specific predictive maintenance function on a determined system of interest. These three attributes are represented in OPMAD as:

- `Predictive maintenance model`: Every recorded case will have one or several models used to fulfil a specific Predictive maintenance function.
- `Predictive maintenance module function`: this attribute includes the function or functions that are fulfilled by predictive maintenance system for a specific maintainable item.
- `Maintainable item`: this class gathers the different machines, equipment, components or technological systems on which the model was applied and validated on each consulted article. The instances included in this class can match the subclasses of `CCO: Artifact`. The `CCO: Artifact` class and its subclasses structure were not directly reused to define the predictive maintenance case as `CCO: Artifact` has a more general purpose that goes beyond maintainable items.

Some complementary attributes have also been included to support the similarity assessment and the solution suggestion:

- `Maintainable item type`: the use case for the new predictive maintenance system may not be the same as the recorded implementations in the case base; however, they might share some similarities that could be considered to adjust a potential solution. Case studies may find some similarities if they are within the same “family” of applications as for example rotary machines or electronics devices.
- `Condition data`: This attribute gathers the instances that describe the inputs for a predictive maintenance model. Each model can be linked to several instances of the class condition data. For example, a neural network to fulfil a fault detection function of jet engines can have as condition data variables temperature, pressure, spinning speed, health index, reliability among others.
- `Condition data type`: this attribute aims helping in the similarity assessment. It is a complementary measure to `Condition data` class. It divides the condition data into different categories (see case base creation section).
- `Predictive maintenance model type`: this attribute aims at providing additional information to the proposed solution. It clusters the models into different categories (see case base creation section).
- `Predictive maintenance model configuration`: this attribute provides important information about the implementation of predictive maintenance models. It is part of the solution attributes.
- `Module synchronization`: this class aims at providing information of the synchronization between the `Predictive maintenance module` and the `maintainable item`. This is a problem attribute based on the structural requirements of a new predictive maintenance system. Online applications deal with real-time data which is constantly assessed to perform diagnostics or prognostics. Off-line applications gather information from an operative cycle to be assessed later. Online models normally are embedded in small control and monitoring boards with limited processing and storage memory while the offline application may be carried out by high-performance computers. Online scopes look for fast results in real time to trigger immediate actions in operation. Off-line scopes may target the maximum accuracy as there is more freedom in terms of time to perform the computations.
- `Predictive maintenance article title`: It is part of the solution attributes and its purpose is to provide traceability to a predictive maintenance case information source.
- `Predictive maintenance article identifier`: this attribute is intended to be part of the solution attributes of the case as it provides the traceability link to find the case information source. This source can be needed to find more detailed information about the case.
- `predictive maintenance article publication year`: this attribute is intended to be part of the similarity measures for the retrieval engine. Newer publications are privileged over older ones.

Some other attributes are part of the case but they are not intended to be part of the similarity assessment attributes or the solution suggestion. These attributes have been added for consistency in the ontology:

- Predictive maintenance system: this attribute aims at naming the system that has one of several predictive maintenance modules to fulfil one or several predictive maintenance functions on a Maintainable item.
- Predictive maintenance module: this attribute has been created to link the Predictive maintenance model with the Predictive maintenance model complying with BFO and CCO standards. The edges between these classes help compute some local similarities for the CBR system (see local similarities section).
- Predictive maintenance article: this attribute aims to be the link between the predictive maintenance case and information entities extracted from research articles such as predictive maintenance article title, predictive maintenance article identifier, and predictive maintenance article publication year.

5.3.1.2 Defining the structure for the case content

An extensive review on case representation techniques can be found in [ESE15]. The selection of the case representation is directly related to the similarity assessment that will be performed. Traditional methods for case representation include:

- Feature vector representation: this is the simplest representation of a case in which the cases are composed by a set of attributes that are used to describe the problem and the solution. The case retrieval is performed by the weighted sum of the similarities between the target case attributes and the cases in the case base. This case representation has limited capabilities to support semantic similarity as there is no representation of the knowledge domain.
- Frame-based representation: Frames provide a natural way for structured and concise knowledge representation. Frames are composed of different slots to organize knowledge. Each case is represented by a frame and each frame slot represents a case attribute. Frame-based representations have been (partially) formalized by description logic [BKP05]. The notion of “cases as terms” [Pla95] argues that viewing structured cases as terms in feature logics (a particular brand of description logics) helps in better understanding several aspects of case-based reasoning.
- Object-oriented representation: this case representation is suitable for complex case data structures. Cases are represented as a set of objects that have their own set of attributes.
- Textual representation: this case representation is suitable when cases are written in natural language texts. Specialized methods can be used to address these cases in an automated or semi-automated way.
- Hierarchical representation: In this approach, a case is represented at multiple levels of detail, possibly using multiple vocabularies. When solving a new problem, similar cases are retrieved from the case base at different levels of abstraction, their solutions are then combined and refined until achieving a final solution [BW96].
- Predicate-based representation: a predicate is a relation among objects, and it consists of a condition part and an action part, IF (condition) and THEN (action). Predicates that have no conditional part are facts. Cases can be represented as a collection of predicates [PS04]. The advantage of predicate representation is that it uses both rules and facts to represent a case, and it enables a case-based designer to build hybrid systems that are integrated rule/case-based [BKP05].

The above-mentioned case representation techniques have limitations in knowledge representation. They have few (if any) structures to describe the relations and constraints among the case features [BKP05]. Case representations using ontologies aims at solving this problem as ontologies provide not only the set of terms that constitute a case but also the relations among these attributes. Ontologies can be useful for designing knowledge-intensive CBR applications because they have powerful capabilities in knowledge acquisition, representation, and semantic understanding [GPH13]. Taking advantage of the created ontology for the current study, the case representation and structure is described using the proposed ontology in Chapter 4. Figure 4.4 showed the different attributes (classes in ontology) of the case represented in OPMAD. These ontology classes can be divided into two different groups: the problem attributes and the solution attributes:

- **Problem Attributes** = [PdM Function, Maintainable Item, Maintainable Item Type, Condition Data Type, Module synchronization, PdM Article Publication year]
- **Solution Attributes** = [PdM Model, PdM Model Configuration, PdM Model Type, Module Performance Indicator, PdM Article Identifier, PdM Article Title]

5.3.1.3 Organizing the case base

The case base corresponds to an instantiated version of OPMAD. The OPMAD structure provides the guidelines for the case base organization. The ontology has been populated using predictive maintenance cases retrieved from an extensive literature review about implemented predictive maintenance systems. The review presented in Chapter 2 was the starting point. An explanation of how the articles were consulted to instantiate OPMAD classes is presented hereafter:

1. **Predictive maintenance case:** the instances of this class are recorded in the form of “Case N ”, where N is an integer value starting from 1 that denotes the order in which the cases have been recorded. A case of a paper is identified when three interrelated attributes are found: the `Predictive maintenance function` that is fulfilled by a predictive maintenance system, the `Model` that fulfils the function, and the `Maintainable item` for which the predictive maintenance system has been developed. In a single article, it is likely to find several cases because of different factors such as:
 - There is a comparison of different models to fulfil the same PdM function. If several models are developed to fulfil the same PdM function, there is an instance in the database for each implemented model.
 - There are different PdM functions addressed by different models. The study can address more than one PdM function in a single PdM system. If every function is addressed by a different model, there is an instance for each addressed function.
 - A model that fulfils a PdM function has been implemented on different maintainable items. Some studies apply their approach to many case studies. An instance is recorded for each case study.
2. **Predictive maintenance system:** the instances of this class are named in the form of “System N ”, where N is a consecutive number according to the order in which the articles that explain these systems were consulted.
3. **Predictive maintenance module:** the instances of this class do not provide much information for the case retrieval or for the solution suggestion. These instances of `Predictive maintenance module` are named in the format of “Module N ” where N is the case number it belongs to.

4. Predictive maintenance module function: the predictive maintenance functions can be clustered in eight different types (sub-classes). The instances receive the name of their subclass plus an integer number that records the order in which they were recorded. For example, instances of the sub-class `Fault detection` will be named as "fault detection1", "fault detection2", ..., "fault detection N ", where N is the total number of instances of the sub-class `Fault detection`. The Predictive maintenance function sub-classes are:
 - 4.1. Features extraction: this PdM function identifies features from data that can be used for diagnostics or prognostics.
 - 4.2. Fault detection: this PdM function aims at detecting a faulty condition on a technical system given some symptoms.
 - 4.3. Fault identification/isolation: this PdM function allows to identify the source of the faulty condition when at least two faults present similar symptoms.
 - 4.4. Degradation modelling: a fault is not static; it evolves over time until it becomes a failure. This progression of the fault is usually called degradation. This PdM function aims at providing a representation of such degradation. Reliability and health indexes are some examples of values that can be used to model the degradation on maintainable items.
 - 4.5. Health assessment/degradation analysis: this PdM function aims at determining the current state of maintainable items based on a pre-existing degradation model.
 - 4.6. Next state forecasting: this PdM function aims at forecasting the health state of a maintainable item. It is applicable when the life cycle of the maintainable item can be divided into discrete steps. This prognostics function aims at determining one-step-ahead or multiple-step-ahead if a failure is likely to happen.
 - 4.7. Remaining useful life (RUL) calculation: this PdM function aims at calculating the remaining work cycles the technical system or one of its components can work before failure.
5. Predictive maintenance model: the instances of this class are not limited by any type of restrictions. The instances of this class have been left as "open-text" as it is normal to find articles proposing new models that have been named by the article authors. Some of the instances in the `Model` class can be used in the future as subclasses in a more detailed version of OPMAD that has a more detailed level of instantiation.
6. Maintainable item: the instances of this class have been left as "open-text" as it is normal to find articles proposing new models that have been named by the article authors. Some of the instances in the `Maintainable item` class can be used in the future as subclasses in a more detailed version of OPMAD that has a more detailed level of instantiation.
7. Maintainable item type: the instances of this class aim at helping in the similarity measure between a target case and the cases in the case base. It is a complementary measure to `Maintainable item` class. The instances for this class have for the moment been limited to the following values:
 - (a) Rotary machines
 - (b) Reciprocating machines
 - (c) Electrical components
 - (d) Structures
 - (e) Energy cells and batteries
 - (f) Lubricants
8. Condition data: the instances of this class include all variables that are input for a Predictive maintenance model. The instances of this class can be found in different cases. Special attention must be paid to the instances of this class if one article includes more than one.

9. Predictive maintenance case: each case can have different data used as input for the predictive maintenance model. For example, there could be a model which aims to model degradation and later performs health assessment. For the first function, the model might use sensors data such as temperature and pressure. The output of the model would be a health index which is at the same time part of the inputs for the health assessment.
10. Condition data type: the instances of this class are limited to the following four options:
 - (a) Signals: direct measurements from the technical system sensors whose reads are signals. Signal processing models are usually needed to extract features that can be used for diagnostics and prognostics. These signals can come from different types of sensors, such as: temperature, vibrations, pressure, spinning speed among others.
 - (b) Time series (Discrete measurements or values): a single value is obtained to describe the state of input from the technical system at a specific work cycle. It can be obtained from sensors, models outputs or other data sources.
 - (c) Structured text-based: this type of variable is similar to time series. The data is recorded in discrete events, but the variables are text-based with a limited number of possible options for the instances. It allows an easier analysis.
 - (d) Text-based maintenance/operation logs: declared knowledge obtained from those who operate or maintain the technical system. Includes symptoms that are not measured by any other means but the human perception. Specialized techniques are needed to address natural language processing.
11. Predictive maintenance model type: it clusters the single-model approaches into three main instances: knowledge-based models, data-driven models and physics-based models. For multi-model approaches, there could be any combination of the proposed instances.
12. Predictive maintenance model configuration: the instances in this class provide information about the implementation of a Predictive maintenance model. In a first attempt, the instances are limited to two possible options: single model approach and multi-model approach. A more detailed analysis is possible by adding the specific configuration of the multi-model approaches as they can be implemented in parallel, in series, or embedded one in another [Mon+20].
13. Module synchronization: the instances of this class are limited to three possible options: "online", "off-line", and "not mentioned".
14. Module performance indicator: this class aims at storing the instances of performance indicators used to validate a model that fulfils a specific predictive maintenance function. For example, fault detection is usually assessed with the percentage of accurate detection, remaining useful life estimation is validated with a standard deviation of the results. More than one performance indicator can be included for each instance. No limited number of options are set for the instances in this class.
15. Predictive maintenance article: the instances of this class are named in the form of "Article N ", where N is the consecutive number that keeps track of the order the articles were consulted. The instances of this class do not provide much information for the solution suggestion as the traceability to the case information source is done through the mentioned information entities of the article. However, its existence is justified for ontology consistency and compliance with BFO standards.
16. Predictive maintenance article title: the instances of these classes are strings that provide the article title given by its authors.
17. Predictive maintenance article identifier: the instances are strings that gather the alphanumeric code used as publication identifier; for example, the DOI, HAL Id, conference Id, among many other options. The DOI will be the privileged one if more than one ID is available.
18. Predictive maintenance article publication year: the instances of this class are recorded as integer values.

5.3.2 Similarity assessment

The similarity assessment is performed in two steps. First, a local similarity is performed for each of the problem attributes, and in the end, all these local similarities are consolidated in a single value called global similarity.

5.3.2.1 Local similarities

Once the case structure is defined for each problem attributes, a similarity measure must be assigned. Table 5.1 summarizes the problem attributes and the similarity functions that have been assigned to each of them. Seven attributes define the problem but the retrieval engine user is able to provide as input six of them. The PdM Article Publication year is automatically computed by the system. An integer similarity function based on discrete points has been assigned to the PdM Article Publication year. It provides a local similarity of 1 to cases that are 5 years old or less. For older cases, the similarity decreases linearly until it reaches the value of 0 at 40 years old. Cases older than 40 years will receive a local similarity of 0 for the PdM Article Publication year attribute.

Table 5.1: Similarity functions assignation

Attribute	Similarity function
PdM Function	Symbol (Ontology)
Maintainable item	String (Levenshtein)
Maintainable item type	Symbol (Equality)
Condition Data	Symbol (Ontology)
Condition Data Type	Symbol (Equality)
Module synchronization	Symbol (Equality)
PdM Article Publication Year	Integer (Points function)

The attribute PdM function has been assigned with a symbol similarity function in MyCBR. The similarity matrix has been filled out using the feature-based ontological similarity presented in [Sán+12]. For this attribute the ontological similarity is computed based on the classes PdM model, PdM module, and PdM function of the ontology (see Figure 5.5). A PdM Module is linked to the PdM model by the relation *isCarrierOf* and at the same time is linked to the PdM function by the relation *hasFunction*. An inferred relation has been obtained between PdM model and PdM function. Using this inferred relation, the similarity is computed based on the models that have been used to fulfil a specific function. For example, a model such as Support Vector Machines (SVM) has been used in several cases for fault detection or fault identification; this increases the similarity between these two PdM functions.

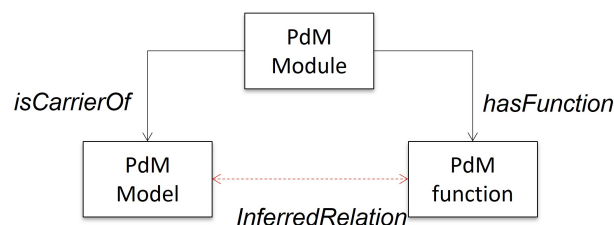


Figure 5.5: Classes and relations used to compute the ontological similarity for PdM function

Similarly, the attribute `condition data` has been assigned with a symbol similarity function whose

matrix has been filled out using the feature-based ontological similarity presented in [Sán+12]. For this attribute, the similarity is built based on an inferred relation between PdM module and condition data (see Figure 5.6). A PdM module of a specific case can have a list of variables that define the condition data (several instances for each case). The user will enter the list of variables of the target case and there will be similarity assessment for each condition data variable against the corresponding lists in the stored cases of the case base. In the end, the local similarity is based on the amount of condition data variables that match between the target case and the stored cases. For example, a target case has the variables “temperature”, and “pressure”, the closest case in the case base has the variables “pressure”, “temperature”, and “spinning speed”, the ontological similarity obtained with equation 5.1 would be approximately 0,70 as there is only one different element between each list. For this similarity function, an additional typo error tolerance has been added. The DSS user enters the condition data variables manually and a typo error is likely to happen. The similarity assessment among the variables is done using the Levenshtein distance [Lev66]. In the first attempt, a limit of two different characters has been selected for typo error tolerance. This means that if a user writes for example “tepmerature” instead of “temperature” the function is still able to perform the similarity assessment correctly.

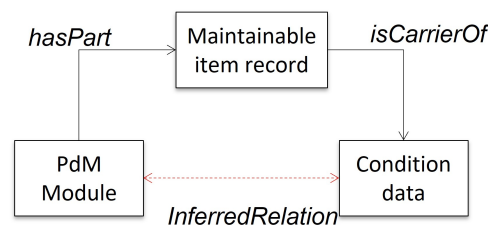


Figure 5.6: Classes and relations used to compute the ontological similarity for condition data

The attributes `Maintainable item type`, `Condition Data Type`, and `Module synchronization` have been assigned a symbol similarity function as provided by MyCBR. For each of these attributes, all the possible options are known and can be directly selected by the retrieval engine user. The equality in the symbol function means that the similarity is binary, if the user selects one option from the available ones for each of these attributes, there will be a local similarity equal to 1 to each case that has the same attribute value and 0 when the case has a different value. No partial similarity has been assigned among the options in the similarity matrix of the symbol function. This can be changed in the future as more relations from the ontology can be used to compute partial similarity among the attribute instances.

The attribute `Maintainable item` has been assigned with a string-based similarity function as provided by MyCBR. Levenshtein distance is used to compare a string entered by the retrieval engine user and the corresponding case attribute in the stored cases.

5.3.2.2 Global similarity: aggregation functions

After the similarity computation for each of the problem attributes, the combination of all these individual similarities in global similarity takes place. Each attribute has a weight and an amalgamation function calculates the final similarity based on local similarities and weights. The weights aim to give special importance to specific problem attributes. In a first attempt, all attributes get the weight of 1, meaning they have the same importance. MyCBR offers two different options to compute the global similarity: weighted sum and euclidean distance.

- **Weighted sum:** as the name says, is the sum of similarities considering the weight of each one. The equation 5.2 shows the calculation performed to find the global similarity.

$$sim_{global} = \sum_{i=1}^n w_i \cdot sim_i \quad (5.2)$$

The values of n , w_i and sim_i are respectively the numbers of variables, the weight and the similarity of variable i .

- **Euclidean distance:** the euclidean distance between two points in an euclidean space is a number, measuring the length of a line segment between the two points. In the scope of the project, the distance is equal to global similarities. The equation 5.3 shows the calculation performed to find the similarity.

$$sim_{global} = \sqrt{\sum_{i=1}^n w_i \cdot sim_i^2} \quad (5.3)$$

In the current research, both amalgamation functions can be used by the user. It is important to point out that to compute the global similarity, only the available attributes are considered. This means that if the architect counts only on two or three attributes out of the seven that describe the problem, the global similarity will be computed based on those two or three attributes.

5.3.3 Retrieval engine user interface

The retrieval engine developed with MyCBR is the core of the DSS for predictive maintenance component selection in this research framework. A Graphical User Interface (GUI) has been developed to help in the verification and validation of the system. Figure 5.7 shows the developed GUI for the developed retrieval engine.

It is important to recall that the CBR system development and the ontology development were done in parallel. The ontology classes that define the problem attributes received different names compared to those used for the CBR system variables. Table 5.2 shows the OPMAD classes and the corresponding names in the retrieval engine code. The distinction of these names has helped to avoid ambiguity in the code.

The GUI has a pull-down menu for those problem attributes with a fixed number of options. Free-text cells have been added for the problem attributes `maintainable item` and `condition data`. The weight for each problem attribute can be directly assigned in the GUI (by default these weights are equal to 1). In the GUI it is possible to selected how many cases to retrieve from the case base and the aggregation (amalgamation) function to compute the global similarity between the target case and the retrieved cases. The list of the most similar cases is shown in the white field just below the attributes that the user can modify. The user triggers the retrieval by clicking on the ***SUBMIT QUERY*** button at the bottom of the GUI.

5.4 Lessons learnt

This chapter addressed the creation of the CBR retrieval engine using the MyCBR platform. The principles of CBR have been introduced including the different phases of the CBR cycle. MyCBR is intended for the retrieval phase of CBR systems. The platform offers a stand-alone application called MyCBR workbench for fast prototyping of retrieval engines. MyCBR also offers an open-source System Development Kit (SDK)

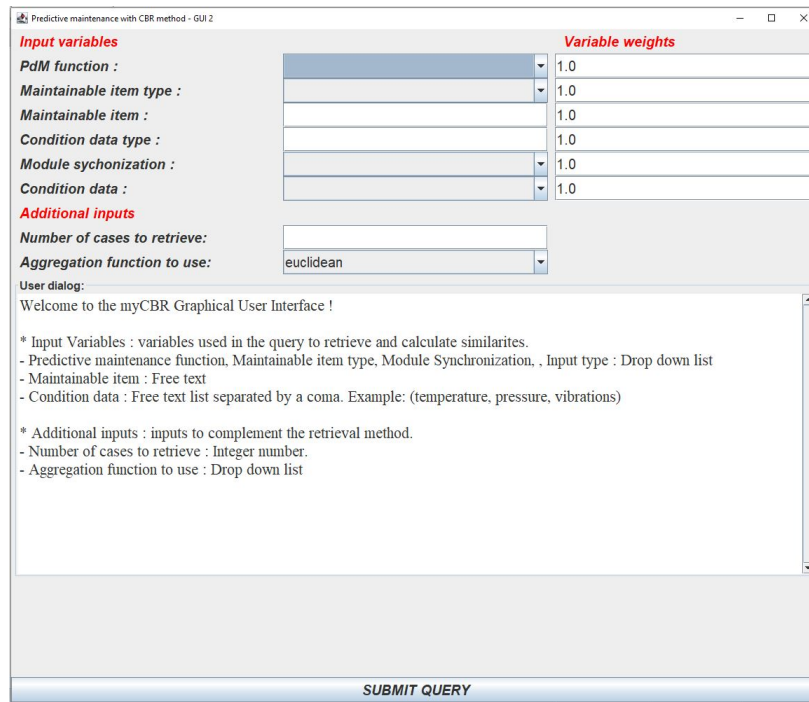


Figure 5.7: Retrieval engine Graphical User Interface (GUI)

Table 5.2: OPMAD classes and corresponding variables in the retrieval engine

OPMAD class	CBR variable
PdM Function	Task
Maintainable item	Case study
Maintainable item type	Case study type
Condition Data	Input for the model
Condition Data Type	Input type
Module synchronization	Online/Off-line
PdM Article Publication Year	Publication year

that allows the integration of the retrieval engines with other systems in a Java-based environment. As the SDK has been selected over the workbench to develop the retrieval engine in this research as the SDK provides more capabilities than the workbench. The case structure and the case base have been defined using the OPMAD classes defined in Chapter 4. The case base corresponds to an instantiated version of OPMAD. The process to instantiate OPMAD from an extensive literature review includes the definition of the different variables to search and the possible options for each variable; this allows a better case base structure and facilitates the retrieval. For each problem attribute, a local similarity is selected among the possible options: integer, symbol, ontology-based, open text. Two different aggregation functions have been added to compute the global similarity between a target case and those cases stored in the case base. A GUI has been developed to facilitate the verification and validation of the retrieval engine.

The developed retrieval engine is the core of the DSS for the selection of predictive maintenance components. The next chapter is oriented on showing the complete framework of the ontology-enabled CBR system for the selection of predictive maintenance components. The verification and validation of the retrieval engine developed in this section are addressed in Chapters 6 and 7. Development details of the retrieval engine are provided in the code guide in Appendix C.

Building a framework for predictive maintenance models selection

“Manufacturing is more than just putting parts together. It’s coming up with ideas, testing principles and perfecting the engineering as well as final assembly.”

James Dyson

Content

6.1	Making the parts work together	97
6.2	Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning (Article 4)	98
6.3	Cross-validation	107
6.4	Lessons learnt	108

6.1 Making the parts work together

Chapters 4 and 5 introduced the two technologies that are used to develop the Decision Support System (DSS) for predictive maintenance component selection. The proposed DSS is intended to overcome the problems encountered when selecting suitable approaches and models in the systems engineering approach to predictive maintenance design presented in Chapter 3. As complex systems may represent an important investment at high risk, the architect can not only rely on an immediate vision of a possible solution (unstructured creativity). The architect needs to explore the solution space looking for suitable components and their possible combinations (structured creativity) to propose a systems architecture able to meet the initial requirements of the system. Exploring the solution space of a new system can be a long-lasting task, especially when the architect has several of options from which they can select logical components. The proposed DSS can help the architect to save time in the exploration of the solution space to find suitable components to fulfill the logical architecture.

This chapter presents the framework for the proposed DSS. It explains how the proposed ontology and the CBR retrieval engine are integrated, and how the DSS fits in the creative work performed by the systems architect. The framework has been consolidated in a conference article. This chapter also includes a complementary section that extends the cross-validation performed to test the DSS capabilities.

6.2 Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning (Article 4)

The content in this section corresponds to a published work in the 7th IEEE International Symposium of Systems Engineering (ISSE) held virtually in 2021. ©IEEE 2021. Reprinted, with permission, from Juan José Montero Jiménez, Rob Vingerhoeds, and Bernard Grabot. “Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning.” In: 7th IEEE Int. Symposium on Systems Engineering 2021, Virtual, 2021 [MVG21].

Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning

Juan José Montero-Jiménez
Tecnológico de Costa Rica
Cartago, Costa Rica
ISAE-SUPAERO
Toulouse, France
Email: juan.montero@itcr.ac.cr

Rob Vingerhoeds
ISAE-SUPAERO, France
Toulouse, France
Email: rob.vingerhoeds@isae-supero.fr

Bernard Grabot
ENIT
Tarbes, France
Email: bernard.grabot@enit.fr

Abstract—A common milestone in systems architecture development is the logical architecture. It provides a detailed overview of the system components and their interfaces but keeps the architecture as generic as possible, meaning that no component is bound to a specific technology. Subsequently, the architect searches for physical/informational components to fulfill the logical architecture and can apply structured creativity to look for innovative solutions. This search can turn out to be a difficult and long-lasting task depending on the system complexity. Too many options may be available to fulfill the logical system components and not always the most suitable ones are identified. This problem is for instance encountered in the design of new predictive maintenance systems, especially when selecting the components to carry out the diagnostics and prognostics. The current study proposes to support the choice of suitable components combining case-based reasoning and ontologies. A domain ontology has been developed as a terminology framework to support the case base, case structure and similarity measures for a case-based reasoning Decision Support System (DSS). The DSS uses attributes of the new problem to solve and suggests the most similar cases from past experiences. The retrieved solutions can be adapted to develop a new predictive maintenance architecture. The decision support system has been tested with data coming from proved predictive maintenance solutions documented in scientific publications.

Index Terms—System architecture, case-based reasoning, predictive maintenance, knowledge reuse, structured creativity, decision-support system.

I. INTRODUCTION

The life cycle of complex systems is composed of several stages starting from the concept phase [1], which can be seen as a very crucial stage. Following a systems engineering approach, the concept phase starts by gathering the needs and desires from all stakeholders, that are then translated into a formal set of requirements to be classified and prioritized to facilitate their analysis. Once the set of stakeholders' requirements is agreed by all parties a creative process starts to find the solution or solutions that meet the requirements. This is then formalized in a system architecture before a detailed design and the system implementation start in the development stage. The system architecture formally describes and represents the system elements and their relationships [1].

Developing a system architecture can be a complex task in which many options are available. Several methods exist to

carry out this architecture process. Very often these methods cover a functional analysis of the system that includes a functional decomposition allowing to handle complexity [1]–[3]. Logical components are proposed to fulfill each sub-function. Once the logical components are defined, physical/informational components are selected and allocated to the logical components to complete the systems architecture.

Proposing suitable components to fulfill the logical architecture is an important challenge for the system architect. Sometimes, the architect may have an immediate vision of a solution, yet this does not always lead to the most efficient one. As the development of complex systems may represent an important investment, structured creativity is increasingly used in the concept stage [2]. This means the architect explores the solution space of the system using a structured methodology, identifying and proposing several possible solutions and performing trade-off analysis to identify the most suitable ones. Exploring the solution space of a new system can be long-lasting task, specially when the architect has many options for suitable components. Also, no explicit design rules exist to perform such component selection.

To overcome this challenge, this study proposes a searching tool that combines Case-Based Reasoning (CBR) and Ontologies. CBR is a problem solving paradigm based on previous experiences that can be used for design purposes. Ontologies are semantic knowledge representations that can model the terminology of a specific domain. The proposed approach aims at allowing the architect to save time when exploring the solution space and at the same time providing inspiration by offering diverse possible solutions to fulfill the logical components. In contrast to other implementations of CBR for systems design [4], [5], the proposed tool incorporates a domain ontology that serves as terminology framework for the CBR system. It helps to model the case structure, store the case base and compute semantic similarity for retrieval purposes in CBR.

This paper is organized as follows: section II explains the context of the research by introducing the building blocks of the proposed framework: Case-Based Reasoning (CBR), ontologies, and ontology-enabled CBR systems. Section III presents the design of predictive maintenance systems and

their architectures, domain to which the proposed approach is applied. Section IV explains the proposed approach of the use of an ontology-enabled CBR system to enhance structured creativity. Section V presents the implementation of this approach for predictive maintenance systems design. Section VI describes the results of the resulting Decision Support System (DSS) and section VII concludes the paper by summarizing the lessons learnt and providing perspectives of future work.

II. BACKGROUND

The current study aims at implementing an ontology-based CBR system that can be used to retrieve suitable components for predictive maintenance (PdM) systems. The objective is to propose to system architects a means to explore the solution space more efficiently so not to miss important potential solutions. This section provides a background of CBR, ontologies, and their combination in recent knowledge modelling and reuse works.

A. Case-based Reasoning (CBR)

Case-based reasoning is a paradigm that leverages past problem solving experience, in form of concrete solving cases, when it comes to solving new problems [6]. Case-based reasoning tries to implement this way of reasoning. The following definition for CBR is used here:

Solve a new problem by remembering a previous similar situation and by reusing information and knowledge of that situation.

Case-based reasoning is a paradigm in which specific knowledge of previously experienced problem situations is being used to solve a new problem, by finding close previous cases, adapting and reusing those previous experiences. It leads to a form of incremental, sustained learning, where information from new situations is kept for future use.

Solving a problem with CBR is performed in a cyclic process of several steps, the so-called CBR-cycle [7], triggered by a new problem: solving the target case. The first phase aims at retrieving the most similar cases from a knowledge base that stores all previous cases. The target case is compared to the stored cases in a the case base using different similarity measurements. The most similar retrieved case is proposed as possible solution in the reuse phase. Some adaptation may be needed to implement the solution for the target case. Subsequently, revision phase takes place to ensure that the suggested solution achieves to solve the problem. Once the solution is validated, in a last phase it is stored in the knowledge base so that it can be reused in future similar problems.

B. Ontology

In information science, an ontology is a formal explicit description of concepts in a domain of discourse, properties of each concept describing its features, attributes and restrictions [8]. One of the goals in developing ontologies is “sharing a common understanding of the structure information among

people and software agents” [9]. This means that the vocabulary used by people in a specific domain of knowledge is “machine readable”. All concepts in an ontology are represented by classes which are linked by properties (also called relations). Ontologies are built using formal languages. One of the most recognized languages is the Web Ontology Language in its second version (OWL2) which is recommended by the World Wide Web Consortium (W3C) [10] because of its importance for the semantic web. The semantic web is an extension of the current web, where the information is well-structured and well-defined so that it can be processed by a machine [11].

Ontologies in OWL2 are compatible with information written in Resource Description Framework (RDF) [10]. In RDF, information is represented in semantic triples: subject-predicate-object. OWL2 ontologies are primarily exchanged as RDF documents; all the knowledge stored in an ontology can be also represented by semantic triples. Two related classes will represent the subject and the object of the triple, while the relation between the two classes represents the predicate of the triple. RDF structure provides a flexible means to model, storage and manage information [10]; other methods requiring variable-length fields would require a more complicated implementation.

Ontologies can help to make the knowledge explicit, defining the relations among different terms. This allows deep analysis on domain knowledge, helping to identify semantic rules among the terms. These rules can be used to develop algorithms that perform automated inferences based on the domain knowledge.

C. State-of-the-art of ontology-enabled CBR

For any CBR application, a very important step is the definition of a domain vocabulary that will serve to model the cases, the case base, the similarity measures and the solution adaptation knowledge [12], [13]. CBR was formalized before ontologies gained importance in the artificial intelligence field. Recent research trends incorporate ontologies to facilitate natural language modelling and processing. For CBR, ontologies can be used to enhance case indexing and retrieval, to improve semantic similarity estimation, to improve case representation, to improve case adaptation, and to improve case retention in a case base [14]. Ontologies provide the terminology framework to better describe the case attributes. In [15], an ontology was used as vocabulary framework for a case-based reasoning system that automates emergency response services. The authors use the ontology in the information extraction process, during the lexical analysis. Later, the ontology is used in the case retrieval by helping to determine the similarities for the case attributes.

Ontologies provide the possibility to compute the similarity between two semantic terms by the use of the ontology-based similarity. As [16] explains, ontology-based similarity can be determined by two different manners: by computing the ontological distance or by comparing the features of the terms. As ontologies can be represented by graphs in which the

terms are in the nodes and the relations among the terms are the edges, the ontological distance between two terms is the shortest edges path from one term to another. Feature-based similarity is obtained by comparing the properties of the classes. This feature based similarity can vary depending on the scope. Selecting the different features will yield to different similarities.

Ontologies can also be used to model the cases and the case base in CBR. The ontology classes can be instantiated with the different cases of the case base. An instantiated ontology can be referred to as a knowledge base of the a specific domain [8]. This knowledge base stores the information in the Resource Description Framework (RDF) which can be easily retrieved and managed. In [17], an ontology is presented to support a CBR system for mechanics tolerance specification in which the case base is built on top of the ontology. Another example uses an ontology to build the case base of a CBR system for decision support on manufacturing process selection [18]. Specifically in systems design, [4] proposes an integrated approach using an ontology and a CBR system to propose design solutions based on the initial requirements and preferences. The authors propose a generic design approach in which the cases are built from requirement attributes and the solution obtained from previous experiences with similar requirements.

Current trends in systems architecture aim at incorporating structured creativity methods to explore the solution space in the concept phase of complex systems. The current study aims at integrating CBR and a domain ontology to facilitate the architect work of retrieving suitable components for predictive maintenance systems.

III. PREDICTIVE MAINTENANCE SYSTEMS

Within the maintenance strategies to trigger maintenance actions, three terms are commonly used: corrective maintenance, preventive maintenance and predictive maintenance. Corrective maintenance triggers the maintenance actions once the failure of a component or system has occurred. Preventive maintenance trigger the maintenance actions by using fixed operation intervals of the component or system; such as time, cycles, kilometres, flights, among others. Predictive maintenance (PdM) aims at determining the right moment to trigger the maintenance actions based on the condition of the maintainable system under consideration [19]. Current fast expanding tools and technologies such as machine learning, internet of things, Industry 4.0, big data, boost predictive maintenance implementation to reach safer, more reliable and more efficient technical systems. Predictive maintenance is often studied within the disciplines of Condition-Based Maintenance (CBM) and Prognostics and Health Management (PHM).

Predictive maintenance is carried out by specialized systems whose main function is to estimate the precise moment to trigger maintenance actions. This main function can be decomposed into six different sub-functions (Figure 1: collect data, pre-process data, detect faults, assess degradation, compute

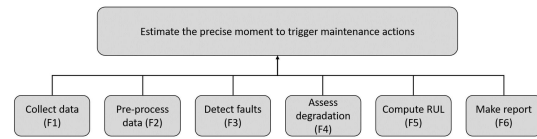


Fig. 1. Functional decomposition for a predictive maintenance system. [20]

remaining useful life and make recommendation report). This functional decomposition is widely used to develop the architecture of predictive maintenance systems. A new predictive maintenance system may require components to fulfill all sub-functions or a smaller subset of them [20]. For example, a system can be intended only to cover a fault detection, then the architecture should have components to collect data (F1), pre-process data (F2), detect the fault (F3) and make a recommendation report (F6).

Specifically for the diagnosis and prognosis sub-functions, there exist several models that can fulfill the requirements. These models can be implemented individually following a single model approach for each sub-function or many of them can be combined in a multi-model approach [19]. These models can be divided into three main families:

- *Knowledge-based models*: these models are built from expert knowledge and experience from which it is possible to explicitly define rules, cases and constraints that can be used to perform reasoning. These explicit models highly rely on the access to experts which can be a challenge for their development.
- *Physics-based models*: these models use the laws of physics to assess the degradation of components. They demand high skills on mathematics and physics of the phenomena for the application.
- *Data-driven models*: these models use data records that often consist of time series. Data-driven models have gained a lot of importance in recent years thanks to the improved availability of computational power and the large amounts of data produced every day by technical systems.

Predictive maintenance systems accurately illustrate the problem for system architects to select suitable components that fulfill the logical architecture. There are no explicit rules that guide the architect throughout the components selection to fulfill the logical architecture when developing a new predictive system. There exist several models that can be used to carry out the diagnosis and prognosis functions of the system; and these models are often combined as one single model hardly addresses a single task of a predictive maintenance system [19]. A decision support system can help the architect to select the model or combination models that have been used in the past accentuates the need for supporting architects for components selection.

IV. ONTOLOGY ENABLED CASE-BASED REASONING FOR SYSTEMS ARCHITECTURE

When developing a new system architecture, the architect often looks for inspiration from previous experiences and related systems that already exist. The idea to integrate Case-Based Reasoning (CBR) and ontologies for systems architecture comes from an analogy between CBR and the structured creativity process, as well as the need to formally model the domain vocabulary that allows the knowledge reuse. This section explains the analogy between CBR and structured creativity. A first concept of the integration of an ontology-enabled CBR Decision Support System (DSS) is introduced as means to help the architect to look for inspiration from previous experiences. The analogy and the concept of the DSS, here intended for predictive maintenance systems, are generic and can be used for the development of other types of complex systems as well.

A. The analogy between structured creativity and CBR for systems architecture

A common milestone in the systems architecture development is the logical architecture. Structured creativity at the logical architecture is based on the combination of different possible physical/informational components that can fulfill the logical components. This allows the exploration of the solution space and may help the architect to identify innovative solutions for a new system. If several possible solutions are identified a trade-off analysis may be necessary to select the most suitable one. The selected architecture serves as a basis for detailed design of the systems. The knowledge gathered from the new implemented system can be used by the architect to develop future systems.

There is an analogy between the structured creativity work performed by the architect to select suitable components to fulfill the logical architecture and the four phases of Case-Based Reasoning: retrieve, reuse, revise and retain. A graphical representation of this analogy is presented in Figure 2 (in Capella notation [3]). All activities before the logical architecture are outside the scope for the current study and for layout simplification, these activities have been summarized in a single box on Figure 2.

- The proposed analogy relates the retrieve phase of CBR with the search performed by the architect on previous related systems that may serve as inspiration for the development of the new system.
- The reuse phase of CBR would be the allocation of the identified components to fulfill the new system architecture.
- The revise phase of CBR would start by the trade-off analysis done by the architect to identify the most suitable components. This phase continues until the verification and validation of the implemented system which are usually performed by other actors than the architect.
- After validation of the new system, the architect may keep the records to be used in the future; this would be the retain phase of CBR.

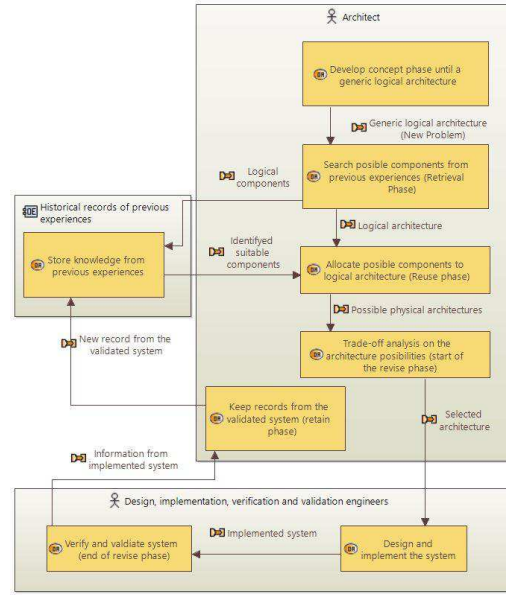


Fig. 2. Analogy between case-based reasoning and the tasks performed by a systems architect, in Capella notation [3]

This analogy remains generic in terms of the system to be developed. All the activities are carried out “manually” by the architect and the records from previous experiences may not be structured to facilitate their reuse. This manual approach may work when the amount of previous experiences is limited so that the solution space to be explored by the architect remains manageable. When the number of previous experiences is high or the requirements complex, the selection of the most suitable components may not be easily achieved. Retrieving knowledge from an important number of previous architectures may consume too much time and important options may be missed.

B. A Decision Support System concept using CBR and ontologies

Considering this analogy between CBR and the structured creativity work, different algorithms can be proposed to support the architect in the different phases of the architecture work. In a first step, the current research focused on the development of a Decision Support System (DSS) able to perform the search and recommendation of suitable physical/informational components. This can help the architect save time in the concept phase, and allows a broader analysis done by machines, analysis that can be hardly addressed by humans.

Figure 3 presents a concept of the DSS and how it fits in the analogy presented on Figure 2. This concept is composed of three main parts: the case base, the retrieve engine, and the domain specific ontology. The case base stores the cases from the past in a structured format to facilitate reuse. The

architect will present to the retrieve engine the information from the new system under development and will obtain a set of the most similar cases retrieved from the case base that will serve as inspiration when selecting suitable components for the logical architecture.

Within this structure the ontology plays a vital role. The cases, attributes of these cases and the similarities among the different text based variables are often described in natural language. Modeling this natural language and making it machine readable is necessary to automate the case retrieval.

V. IMPLEMENTATION ON THE PREDICTIVE MAINTENANCE CASE STUDY

The proposed approach assumes that the logical architecture of the system has been developed. A systematic approach to develop new predictive maintenance systems has been proposed in [20]. The ontology-enabled CBR Decision Support System (DSS) proposed here is for predictive maintenance models (components) selection to fulfill the logical architecture of a new system, especially to fulfill the diagnosis and prognosis functions. The stakeholder requirements for the DSS are as follows:

- 1) The system shall suggest suitable models to fulfill a specific diagnosis and prognosis function.
- 2) The system shall provide a list of diverse potential solutions for each logical component.
- 3) The system shall provide a structured framework to store the knowledge from previous predictive maintenance systems.
- 4) The system shall base the recommendation on known attributes of the new predictive maintenance system such as function to fulfill, the maintainable system and the available condition data from the maintainable system.

- 5) The system shall provide design information from the suitable models such as: performance indicators, input variables and models configuration.

A. Domain ontology development

The ontology for this study was developed using the methodology *Ontology Development 101* [8]. To delimit the scope of the ontology a set of competency questions should be considered. The ontology must be able to answer these questions. In this study, these competency questions have been defined in cooperation with experts in predictive maintenance and aim at gathering useful information that can be used in the design of new predictive maintenance systems. In a first attempt, the scope of the ontology is limited by the following competency questions:

- What are functions of a predictive maintenance system?
- What are the systems on which predictive maintenance has been implemented?
- What models have been used to fulfill each function of the system?
- For a given predictive maintenance case, is the predictive maintenance system implemented online or off-line?
- What performance indicators have been used to assess a model that fulfills a predictive maintenance function?
- What data is analyzed by the predictive maintenance models?
- If several models are being used to fulfill a specific function, what is the models configuration in the system?
- Where is the predictive maintenance implementation documented?

The ontology model was developed using a top-level domain-neutral ontology, the Basic Formal Ontology (BFO) [21]. It also uses a set of mid-level domain-neutral ontologies, the Common Core Ontologies (CCO) [22]. BFO and CCO

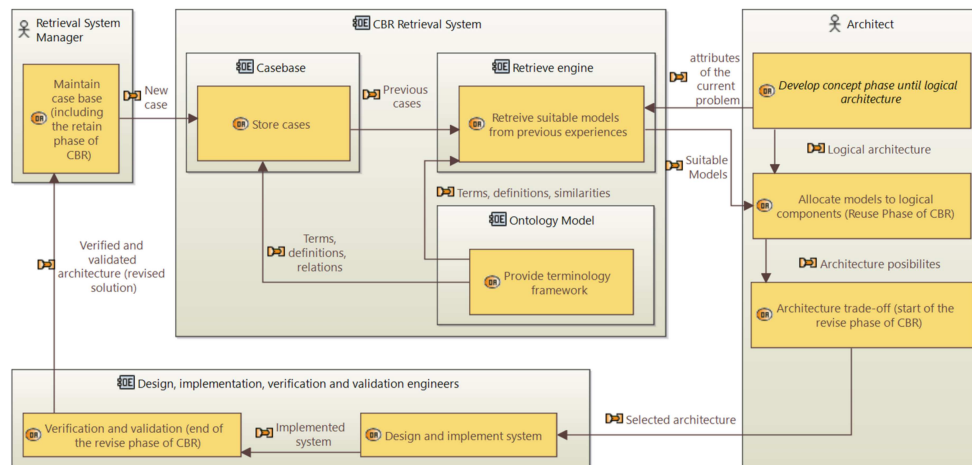


Fig. 3. Incorporation of an CBR retrieve system to facilitate logical components selection in the architecture process

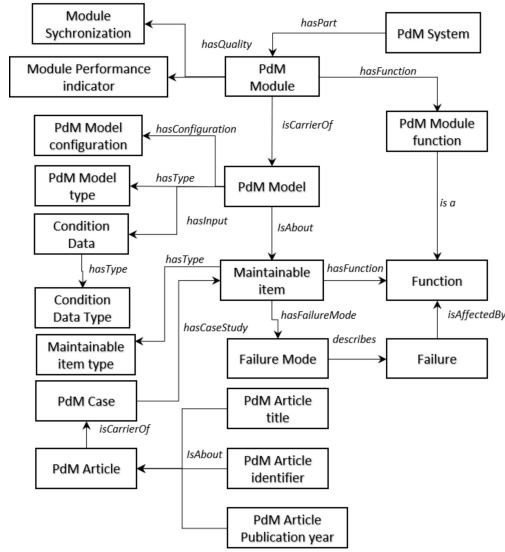


Fig. 4. Classes and relations in the ontology to support CBR for PdM component selection

have also been used in other ontologies in the industrial domain. The standardization on the developed ontology by adopting these upper level ontologies facilitates its future reuse and integration with other ontologies in the industrial domain. The Ontology was created using Protégé® (version 5.5.0) and its consistency was verified using the ontology reasoner HermiT OWL (version 1.4.3.456).

Figure 4 summarizes the most important classes and relations in the ontology for CBR decision support system for predictive maintenance components selection. For the validation of the current approach, the ontology was populated with information coming from the publications considered in a recent structured literature survey about predictive maintenance [19]. The OWL API [23] was used to populate the ontology and to provide the classes and methods for the integration of the ontology and the CBR system.

B. CBR decision support using MyCBR SDK

MyCBR is an open-source similarity-based retrieval tool for Case-Based Reasoning (CBR) [12] that offers two possible options for CBR solutions: a stand-alone application called myCBR Workbench and a Software Development Kit (SDK). The SDK is written in Java and includes all classes and methods to develop CBR applications and integrate them to other systems.

When developing a CBR retrieval system using myCBR, the first step is to determine the case structure. Cases are represented by a coupled vector of attributes Case = [Problem attributes, Solution Attributes]. The problem attributes are used to measure the similarity of a target case and the cases stored in the case base. The solution attributes are stored as part

of the solution information for the cases. Taking as reference the classes in the domain ontology, for predictive maintenance design the case vector will have the following structure:

- **Problem Attributes** = [PdM Function, Maintainable Item, Maintainable Item Type, Condition Data Type, Module synchronization, PdM Article Publication year]
- **Solution Attributes** = [PdM Model, PdM Model Configuration, PdM Model Type, Module Performance Indicator, PdM Article Identifier, PdM Article Title]

Once the case structure is defined, a similarity measure must be assigned to each problem attribute. In myCBR the similarity for each problem attribute is referred to as local similarity. MyCBR offers a complete set of similarity measures to compute local similarities. For the current study three main types of similarities have been used: integer/float similarity, symbol similarity and string similarity [12]. The domain ontology plays a vital role in the computation of some of the local similarities. For example, for the predictive maintenance (PdM) functions an ontological similarity approach based on the ontological features is adopted to obtain the similarity among the options [16]. This similarity is computed based on the models that have been historically used to fulfill each function. All the cases stored in the ontology are used to compute this similarity and if the case base is updated with new cases in the future, the similarity will be automatically updated. This similarity measure was selected for the attributes PdM Function, Maintainable Item Type, Condition Data Type, and Module synchronization.

After the similarity computation for each attribute of the problem, the combination of all these individual similarities in a global similarity takes place. Each attribute has a weight and an amalgamation function calculates the final similarity based on local similarities and weights. MyCBR offers two different options to compute the global similarity: weighted sum and Euclidean distance.

- **Weighted sum**: the sum of similarities considering the weight of each one (equation 1).

$$sim_{global} = \sum_{i=1}^n w_i \cdot sim_i \quad (1)$$

The values of n , w_i and sim_i are respectively the number of variables, the weight and the similarity of variable i .

- **Euclidean distance**: distance between two points in Euclidean space. It represents the length of a line segment between the two points (equation 2).

$$sim_{global} = \sqrt{\sum_{i=1}^n w_i \cdot sim_i^2} \quad (2)$$

For both amalgamation functions there is a re-scaling normalization between 0 and 1, with 1 being the maximum global similarity between the target case and a retrieved case. This

maximum global similarity is achieved when the target case attributes are identical to those of the retrieved case. This maximum global similarity can be achieved independently from the number of problem attributes that are introduced to the DSS. If the architect uses only on two or three attributes to describe the target case, a global similarity of 1 can be obtained based on those two or three attributes.

VI. RESULTS AND DISCUSSION

The case base stored in the domain ontology was instantiated with 263 cases obtained from the 135 research papers consulted in [19]. For a cross-validation of the ontology-enabled CBR Decision Support System (DSS), the cases were randomly divided into two sets. A set of 200 cases was used as case base in the ontology, the other 63 cases were used as test set.

A. Verification

The DSS verification checked how each local similarity performs. For this, several searches were realized using a reduced set of problem attributes. The verification results are described hereafter:

- With searches using one single attribute it was possible to validate the similarity measures assigned to the problem attributes: PdM Function, Maintainable Item, Maintainable Item Type, Condition Data Type and Module synchronization. These are the variables that the user can directly introduce as inputs for the DSS. When presenting only one out of these five possible attributes to define the target case, the DSS retrieved with similarity of 1 every case from the case base that shared the same attribute value as for the target case, independently from the assessed attribute.
- When increasing one by one the number of attributes of the target case, this behaviour was consistent. Several combinations using a reduced set of attributes were tested to confirm the DSS consistency to retrieve the most similar cases.
- The more attributes shown to the DSS, the fewer cases with similarity of 1 or close to one were retrieved. This is an expected behaviour because the stored cases in the case base are diverse. It was rare to find two cases in the case base with the same problem attributes and when it happened, the solution attributes of the retrieved cases were completely different.
- For some of target cases in the testing set, the maximum similarity reached was between 0.7 and 0.8 proving that when no identical cases were found and thus, the maximum global similarity of one can not be reached.
- The searches using a reduced number of attributes also helped verify the similarity measure implemented for the PdM Article Publication year, the only attribute that the user does not specify in the DSS to describe the target case and it is used to compute the global similarity. The current year is automatically retrieved from the operative system and it is compared

against the year of publication of each PdM article. Models applications in newer papers are favored over similar applications from the past. With a reduced number of problem attributes it was possible to confirm that cases from recent publications had a higher similarity to the target case compared to those from older publications.

B. Validation

The cross-validation of the DSS checked that the stakeholder requirements of section V were met.

- Requirement 1 states that the DSS must suggest suitable components to fulfill diagnosis and prognosis functions in predictive maintenance systems. In practice, an architect needs at least one suitable model to develop the architecture but may look for inspiration from the retrieved cases with highest similarity. In the first attempt the acceptance criteria for the DSS is to provide at least 1 suitable model that can fulfill the PdM function of the target case. For each of the 63 cases it was possible to retrieve the similarity to the 200 cases in the case base. A list of the 5 most similar cases for each target case in the testing set was retrieved. For all the test cases at least one suitable model with similarity above 0.7 was retrieved. For each test case, the implemented PdM model is known. In 40% of the tested cases the implemented PdM model matched to one of proposed solutions by the DSS. In 40% of the tested cases all the proposed solutions by the DSS were implemented for the same PdM function. For the other 60% there was at least one recommended PdM model that was originally used for a different PdM function but being possible to adapt it to the target case. For example, for a target case to fulfill a fault detection function, the DSS proposed a classification model that was used for fault identification in the retrieved case. Classification techniques are suitable for both PdM functions. With these results, the acceptance criteria of the first requirement has been fulfilled; the capability of ontology-enabled CBR decision support system to recommend suitable models to fulfill a specific function of a predictive maintenance system has been confirmed.
- The validation highlights an important point of improvement with regard to the diversity in the retrieved cases. This is related to the second requirement. In at least half of the tested cases, within the list of the 5 most similar cases retrieved by the DSS, two or three cases proposed the same PdM model as potential solution. It is important to notice that this is not a problem of repeated cases in the case base, the similarities of the retrieved cases are different among them but sometimes they propose the exact same solution. Diversity in retrieved cases is a common problem in CBR [13], especially when several different potential solutions are expected from the retrieval phase. Diversity in the retrieved cases can help architects to develop innovative solutions. The identified problem narrows down the solution space recommended

by the ontology-enabled CBR system to the architect and represents an improvement of the DSS for future work.

- As the case base is stored in the domain ontology the third requirements is met. The ontology is a formal and structured knowledge representation that stores the predictive maintenance implementations of the past that can be reused.
- The case structured in the DSS was determined in such a way to fulfill requirements 4 and 5. The problem attributes are based on known parameters of the new predictive maintenance system and the solution attributes of the retrieved cases provide useful information for the design of the system.

For the verification and validation of the proposed DSS, both amalgamation functions were tested with no significant differences in the results. For this current study, all the attributes were assumed to have the same importance, so no special weights were assigned to the local similarities. As such they had a default value of 1.

VII. CONCLUSION AND FUTURE WORK PERSPECTIVES

The current study presents the use of a ontology-enabled Case-Based Reasoning (CBR) recommendation system to select suitable components for predictive maintenance systems architecture. The proposed approach benefits from the semantic knowledge modelled using a domain ontology. It allows to build the similarity measures for different attributes of the CBR system. The ontology stores the case base for the CBR that was created using the myCBR platform. The validation shown that the ontology-enabled CBR system is capable to retrieve suitable components to fulfill specific predictive maintenance functions. The retrieval capability was validated with all problem attributes that have been introduced individually and combined to the recommendation system. The retrieved cases can help architects as inspiration to develop innovative solutions for new predictive maintenance systems.

Future work perspectives include the refinement of the case retrieval functions, case base maintenance to improve the diversity in the retrieved cases, and the integration of a trade-off analysis that can help the architect perform the selection among the retrieved components. Future perspective also include the incorporation of algorithms to address other CBR phases such as adaptation of the retrieved cases for the target problem and case base maintenance after retaining newer cases. As the analogy of CBR and structured creativity was proposed as a generic approach, its implementation in other case studies is also considered in the future work perspectives.

ACKNOWLEDGMENT

The authors would like to thank the students Johanna Mazouzi from ISAE-ENSMA, Augusto Miyagawa and Hugo Muñoz-Hernández from ISAE-SUPAERO, for their collaboration in this research.

REFERENCES

- [1] INCOSE, *Systems Engineering Handbook. A guide for system life cycle processes and activities. Fourth Edition.* Wiley, 2015.
- [2] E. Crawley, B. Cameron, and D. Selva, *System Architecture: Strategy and Product Development for Complex Systems.* Pearson Higher Education, Inc., 2015.
- [3] P. Roques, *Systems Architecture Modeling with the Arcadia Method 1st Edition.* ISTE Press, 2018.
- [4] J. C. Romero Bejarano, T. Coudert, E. Vareilles, L. Geneste, M. Aldanondo, and J. Abeille, "Case-based reasoning and system design: An integrated approach based on ontology and preference modeling," *Artificial Intelligence for Engineering Design, Analysis and Manufacturing: AIEDAM*, vol. 28, pp. 49–69, 2014.
- [5] A. Tidemann, F. O. Björnson, and A. Aamodt, "Case-based reasoning in a system architecture for intelligent fish farming," in *Frontiers in Artificial Intelligence and Applications*, 2011.
- [6] J. Kolodner, *Case based reasoning.* Morgan Kaufmann, 1993.
- [7] A. Agnar and E. Plaza, "Case-Based reasoning: Foundational issues, methodological variations, and system approaches," *AI Communications*, vol. 7, no. 1, pp. 39–59, 1994.
- [8] N. F. Noy and D. L. McGuinness, "Ontology Development 101: A Guide to Creating Your First Ontology," Tech. Rep., 2001.
- [9] T. R. Gruber, "A translation approach to portable ontology specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993.
- [10] World Wide Web Consortium, "OWL 2 Web Ontology Language," 2012. [Online]. Available: <http://www.w3.org/TR/2012/REC-owl2-overview-20121211/>
- [11] D. L. Nuñez and M. Borsato, "OntoProg: An ontology-based model for implementing Prognostics Health Management in mechanical machines," *Advanced Engineering Informatics*, vol. 38, pp. 746–759, 2018.
- [12] K. Althoff, T. Roth-Berhofer, K. Bach, and C. Severin, "Documentation: myCBR," 2006. [Online]. Available: http://mycbr-project.org/%0Adownloads/myCBR_3_tutorial_slides.pdf
- [13] R. L. De Mantaras, D. Mcsherry, D. Bridge, D. Leake, B. Smyth, S. Craw, B. Faltings, M. L. Maher, M. T. Cox, K. Forbus, M. Keane, A. Aamodt, and I. Watson, "Retrieval, reuse, revision and retention in case-based reasoning," *The Knowledge Engineering Review*, vol. 20, no. 3, pp. 215–240, 2005.
- [14] J. Prentzas and I. Hatzilygeroudis, "Combinations of case-based reasoning with other intelligent methods," in *CEUR Workshop Proceedings*, 2008.
- [15] K. Amailef and J. Lu, "Ontology-supported case-based reasoning approach for intelligent m-Government emergency response services," *Decision Support Systems*, vol. 55, pp. 79–97, 2013.
- [16] D. Sánchez, M. Batet, D. Isern, and A. Valls, "Ontology-based semantic similarity: A new feature-based approach," *Expert Systems with Applications*, vol. 39, pp. 7718–7728, 2012.
- [17] Y. Qin, W. Lu, Q. Qi, X. Liu, M. Huang, P. J. Scott, and X. Jiang, "Towards an ontology-supported case-based reasoning approach for computer-aided tolerance specification," *Knowledge-Based Systems*, vol. 141, pp. 129–147, 2018.
- [18] M. M. Mabkhot, A. M. Al-Samhan, and L. Hidri, "An ontology-enabled case-based reasoning decision support system for manufacturing process selection," *Advances in Materials Science and Engineering*, 2019.
- [19] J. J. Montero Jimenez, S. Schwartz, R. Vingerhoeds, B. Grabot, and M. Salaün, "Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics," *Journal of Manufacturing Systems*, vol. 56, pp. 539–557, 2020.
- [20] J. J. Montero Jiménez and R. Vingerhoeds, "A System Engineering Approach to Predictive Maintenance Systems: from needs and desires to logical architecture," in *5th IEEE Int. Symposium on Systems Engineering 2019*, Edinburgh, 2019.
- [21] International Organization for Standardization (ISO), *ISO/IEC 21838-2 - Information technology - Top-level ontologies (TLO) - Part 2: Basic Formal Ontology (BFO)*, 2020.
- [22] R. Rudnicki, "An Overview of the Common Core Ontologies 1.3," Buffalo, NY, p. 29, 2020. [Online]. Available: https://www.nist.gov/system/files/documents/2019/05/30/nist-ai-rfi-cubrc_inc_004.pdf
- [23] I. Palmisano, "OWLAPI Documentation," 2020. [Online]. Available: <https://mvnrepository.com/artifact/net.sourceforge.owlapi/owlapi-distribution/5.1.17>

6.3 Cross-validation

This section extends the cross-validation discussion presented in Article 4 included in Section 6.2. The cross-validation of the ontology-enabled Case-Based Reasoning (CBR) system (the Decision Support System) allows demonstrating that this knowledge reuse technique can be used for the component selection of predictive maintenance systems. It is important to recall that in this first step, the validation is oriented to show that a Decision Support System (DSS) is capable to propose suitable components to fulfil specific predictive maintenance functions. No trade-off analysis has been implemented yet to assess the models proposed by the DSS. The validation focuses on the fact that the suggested models can be adapted to fulfil a specific function and that the list of suggested solutions covers all the solution space.

Cross-validation is a technique that can be used to test the effectiveness of trained artificial intelligence models. For the current approach, a train-test split approach is selected to perform the cross validation. For the test set, 63 out of the 263 cases in the case base were randomly extracted and the rest have been left in case base as training set. For each of the 63 extracted cases, the problem attributes were presented to the DSS using a GUI. The attention was centred on the 10 most similar cases when performing the retrieval. Figure 6.1 shows an example of a retrieval test in which all the target case problem attributes are matched with the corresponding attributes of case 35. This full match between the attributes turns out in a global similarity of 1.

Case	Description
Case35 Sim = 1.000	Reference, Similarity and Input variables Reference: 35 Task: Remaining useful life estimation Case study type: Rotary machines Case study: Rolling bearings Online/Off-line: Off-line Input for the model: Time series Models: Convolutional Neural Network Input type: Vibrations Publication Year: 2018 Publication identifier: DOI: 10.1109/ACCESS.2018.2804930

Figure 6.1: Retrieval example using the GUI of the DSS

The solution space of the predictive maintenance models can be divided into two groups as explained in [Mon+20]. The first group is composed of single model approaches, divided into three main categories: knowledge-based models, data-driven models and physics-based models. The second group is composed of multi-model approaches, in which at least two models from any of the three mentioned categories are combined to fulfil a specific function of the PdM system. Predictive maintenance is composed of diagnostics and prognostics tasks. According to the literature review [Mon+20], diagnostics tasks such as fault detection, fault identification, and health state modelling can be addressed by models of the three categories of single model approaches or multi-model approaches. In contrast, for prognostics tasks, it is difficult to find knowledge-based models applications. For example, prognostics functions such as remaining useful life estimation and next state forecast functions are normally fulfilled by physics-based models, data-driven

models, and multi-model approaches combining physics-based and data-driven models. The cross-validation of the DSS allowed to confirm this behaviour. The retrievals for diagnostics tasks proposed single model and multi-model solutions considering models from the three categories of models. The retrievals for prognostics also proposed single model and multi-model solutions, but the proposed models were only from data-driven and physics-based categories. This helps to confirm that the solution space is well covered.

For each test case, the retrieved cases are ranked based on the similarity of problem attributes. For some of the tests, two or more retrieved cases had the same maximum similarity value. From a systems engineering perspective this represents a limitation because no further information is provided to perform the trade-off analysis and selected the most suitable one. Another limitation was identified with regards the diversity in the retrieved cases. For some target cases the DSS retrieved two or more cases that proposed the same model to fulfil a specific PdM function. An architect looking for inspiration to develop innovative solutions will need the DSS to propose a diverse set of models. Retrieving two cases with the exact same solution is not useful for the solution space exploration.

The above mentioned limitations are improvement points for the DSS to be considered in the future. A more detailed analysis can also be performed including the effort to adapt the solution of a retrieved case for the target case. This can help to perform the trade-off analysis on the retrieved options and consequently improve the performance of the DSS.

6.4 Lessons learnt

This chapter presented the Decision Support System framework and can help the architect in the selection of predictive maintenance components and approaches. The chapter explains the integration of OPMAD and the CBR retrieval engine developed with myCBR platform. It also explains how the DSS can fit in the architecture process and how an architect can benefit from it to automate the solution space exploration allowing a more accurate structured creativity by only retrieving the most suitable components for a new predictive maintenance system.

Recalling the refined research questions introduced in Chapter 2:

1. How to address the design of predictive maintenance systems?
2. How to suggest a suitable approach for a predictive maintenance system solution?
3. How to select a suitable model or combination of models given a new predictive maintenance problem to solve?
4. How can a designer benefit from the experience of existing systems to develop new predictive maintenance solutions?

The presented framework attempts to answer the first refined research question. It combines the systems engineering approach for the concept stage presented in Chapter 3 and the use of a DSS for the selection of suitable components for the logical architecture. The DSS implementation attempts to answer the rest of the research questions. It allows to store the past experiences of predictive maintenance implementations and retrieves the most suitable ones depending on the new predictive maintenance system to develop. The DSS not only suggests the models that can fulfil a specific predictive maintenance function but also other important aspects that can be used for the detailed design and implementation. To further validate the proposed framework and the DSS capabilities, the next chapter presents a practical example on which the framework and the DSS are used. An implementation of one of the suggested models by the DSS is used to indirectly validate the accuracy of the DSS recommendations.

Validation approach and discussion

“When you want to know how things really work, study them when they’re coming apart.”

William Gibson

Content

7.1	Validating the proposed Decision Support System in a practical example	109
7.2	Use case example: Design of a predictive maintenance system for aircraft engine run-to-failure data set under real flight conditions	109
7.3	The concept phase of a predictive maintenance system for the N-CMAPSS database . . .	111
7.4	Component selection using an ontology-enabled case-based recommendation system . . .	113
7.5	Discussion	117
7.6	Lessons learnt from the DSS validation using the N-CMAPSS	119

7.1 Validating the proposed Decision Support System in a practical example

As part of the framework validation, a use case example has been developed. This example uses an aircraft engine data set [Cha+21]. The data set contains the run-failure records of 128 aircraft jet engines under real flight conditions and has been generated with the Commercial Modular Aero-Propulsion System Simulation (CMAPSS) model developed by NASA. This data set is called the N-CMAPSS. The purpose is not only to check the capabilities of the proposed ontology-enabled CBR system but also to identify the improvement points for the DSS. The objective for this validation is to implement one of the models proposed by the DSS to fulfil a predictive maintenance function for the N-CMAPSS database. This implementation helps to demonstrate that the DSS is capable to suggest suitable components for predictive maintenance systems, complementing the cross-validation in Chapter 6.

7.2 Use case example: Design of a predictive maintenance system for aircraft engine run-to-failure data set under real flight conditions

The N-CMAPSS data set was published in January 2021 and has an objective to facilitate the development of specialized algorithms for predictive maintenance applications by providing a complete set of run-to-failure data with different failure modes. CMAPSS has been used to simulate other well-known data sets for

prognostics purposes, such as PHM08 data [Sax+08]. The N-CMAPSS data set supposes an improvement in the level of fidelity between the simulated data versus the real-life data. Each flight in the N-CMAPSS data set is fully recorded from take-off to landing. Previous versions of CMAPSS data sets offer a simpler approach that only provides one single discrete measure for each engine flight. Another improvement can be seen in the number of failure modes. Previous CMAPSS data sets only had one or two imposed failure modes, and for those sets with two failure modes, there were no explicit records that could be used to discriminate the failure mode that affected each jet engine. The N-CMAPSS data set has up to seven different failure modes and there is an explicit record of the failure mode that affected each engine. Each failure mode has its own set of symptoms that can be identified by the loss of Flow (F) or Efficiency (E) in the rotary components of the engine such as the fan, the Low-Pressure Compressor (LPC), the High-Pressure Compressor (HPC), the Low-Pressure Turbine (LPT), and the High-Pressure Turbine (HPT). This improvement extends the use of the N-CMAPSS data set, not only for prognostics like the previous CMAPSS data sets but also for diagnostics.

According to [Cha+21], five main steps have been followed to generate the data set (see Figure 7.1):

1. The flight data are defined as recorded onboard real commercial jets.
2. The degradation of the engine components is imposed in the simulation so that it is possible to keep track of the failed component in each run-to failure engine.
3. The resulting degraded flight is simulated using the CMAPSS.
4. The health condition is evaluated and the unit continues flying with increasing degradation until the health index of the engine has reached zero.
5. Sensor noise is added to the simulated engine response in order to make the simulated data closer to real-life data.

The flight conditions have been divided into three flight classes depending on the flight length. Class 1 includes all flights that last from one to three hours. Class 2 includes all flights between three to five hours of duration. Class 3 includes all flights that last more than 5 hours. The data set is divided into 8 sub-sets with different failure modes and a different number of engine units. Table 7.1 shows the overview of the N-CMAPSS data set. Each sub-set has its own failure mode except for the sub-set DS02 which has 2 failure modes which are the ones from the sub-sets SD01 and DS03 correspondingly. The overview shows the symptoms that describe each failure mode. For example, the failure mode of sub-set DS01 can be detected by a decrease of the efficiency in the HPC, while the failure mode of sub-set DS04 can be detected by a decrease of the efficiency and flow of the engine fan.

Each unit in the N-CMAPSS data set has unknown initial degradation, meaning that the records did not start at the same point for all of them. The records for each engine in the N-CMAPSS finish when the engine reaches the failed state. Each engine goes through a normal degradation stage in which the fault symptoms are not evident. If a fault appears in the engine, there will be a transition from normal degradation to abnormal degradation that is faster than normal degradation to reach the failed state. The data is composed of 46 variables divided in 5 different groups:

1. Scenario descriptors: variables that are independent of the engine operation that are used to describe the operational modes of the engine.
2. Measurements: Sensors that describe the engine operation.
3. Virtual sensors: complementary measures offered by CMAPSS to assess the engine operation

4. Model health parameters: complementary variables that show the symptoms of the different failure modes.
5. Auxiliary variables: variables for the data set consistency.

Further explanation about the N-CMAPSS data set can be found in [Cha+21] and [FDL07].

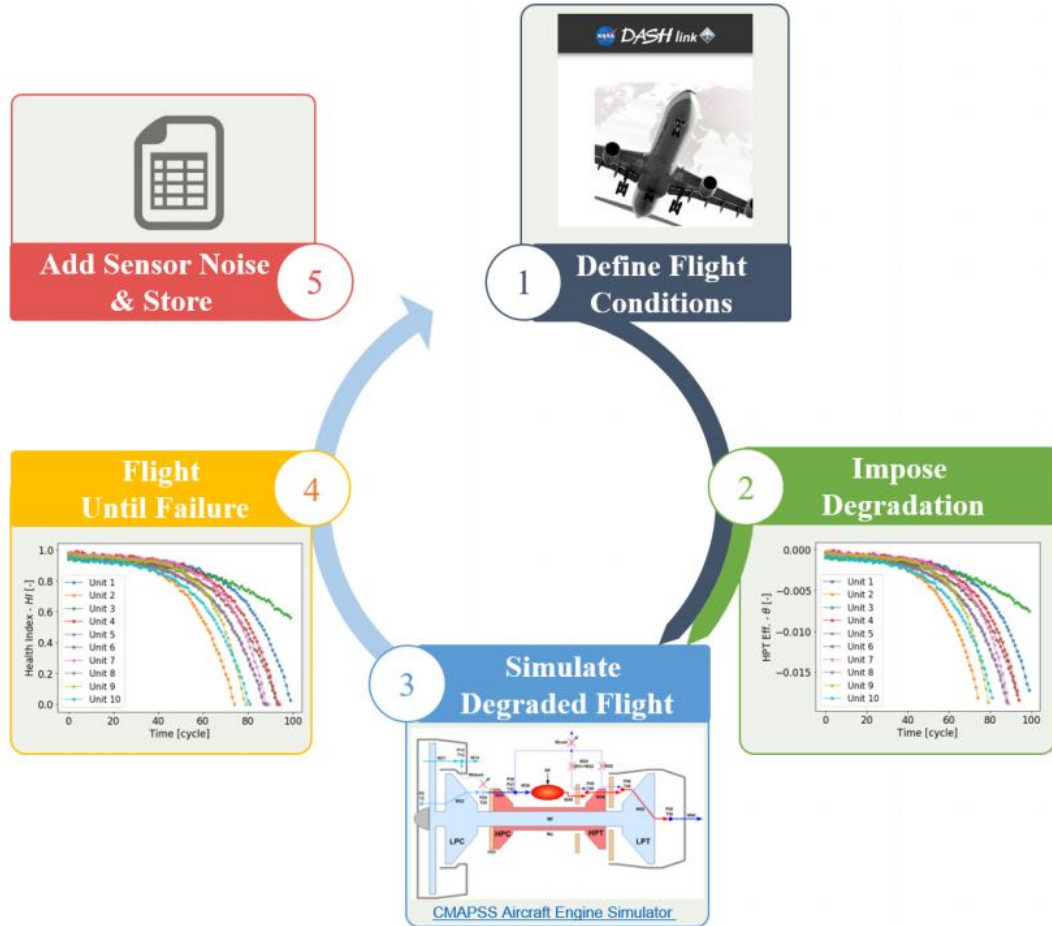


Figure 7.1: N-CMAPSS data creation process [Cha+21]

7.3 The concept phase of a predictive maintenance system for the N-CMAPSS database

Given that N-CMAPSS is intended for research purposes, it is up to the researchers to define the list of needs and desires for the new predictive maintenance system. For the current research framework, the following list of needs and desires is proposed:

1. Read the N-CMAPSS data format.
2. Identify the variables needed to perform the diagnostics and prognostics.

Table 7.1: Overview of N-CMAPSS data set [Cha+21]

Name	# Units	Flight classes	Failure modes	Fan		LPC		HPC		HPT		LPT		Size (measures)
				E	F	E	F	E	F	E	F	E	F	
DS01	10	1, 2, 3	1							✓				7.6M
DS02	9	1, 2, 3	2							✓		✓	✓	6.5M
DS03	15	1, 2, 3	1							✓		✓	✓	9.8M
DS04	10	2, 3	1	✓	✓									10.0M
DS05	10	1, 2, 3	1					✓	✓					6.9M
DS06	10	1, 2, 3	1			✓	✓	✓	✓					6.8M
DS07	10	1, 2, 3	1									✓	✓	7.2M
DS08	54	1, 2, 3	1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	35.6M

3. Pre-process and reduce the N-CMAPSS data set.
4. Detect the transition from normal degradation to abnormal degradation.
5. Identify the failure mode that leads the engine to fail.
6. Determine the current state of a jet engine.
7. Estimate the remaining useful life of a jet engine.
8. Provide a report of the results

The previous list of needs and desires are translated into the following list of functional requirements:

1. The system shall read the N-CMAPSS data set.
2. The system shall model the health of the jet engine.
3. The system shall detect incipient faults in the engine.
4. The system shall assess the health state of the engine.
5. The system shall estimate the remaining useful life of the engine.
6. The system shall provide a report of the predictive maintenance results.

In the first concept of the predictive maintenance systems, the data pre-process and data reduction are not included and they have been performed manually. As can be seen, only functional requirements have been determined. The N-CMAPSS does not provide any performance needs, structural constraints or experiential needs which means that no behavioural, structural or experiential requirements have been added. As the data set comes from an academic case study, no experiential needs or desires have been added. It is important to notice that in practical applications, these three types of requirements are important to be considered as they provide the requirements related to the expected performance of the system, interfaces with other related systems and interface with the system user.

Given the functional requirements, the architecture process presented in Chapter 3 has been followed until obtaining a logical architecture of the new predictive maintenance system. Figure 7.2 presents the logical architecture developed from the functional requirements previously listed. A first component will collect the processed data from the jet engine records. This data will be used to develop the health model of the engine. Later, the engine data and the health model are used for the components in charge of assessing the health state, detect incipient faults, and identify these faults. The outputs of these three components are

the input for the system component in charge of the remaining useful life estimation. The health assessment, fault detection, fault identification and remaining useful life computation will provide the information needed to make the predictive maintenance report by the last system component. The functions of each predictive maintenance system component have been assigned to match with the definitions established in the ontology and the Decision Support System (DSS) so that it can be possible to retrieve the corresponding recommendations.

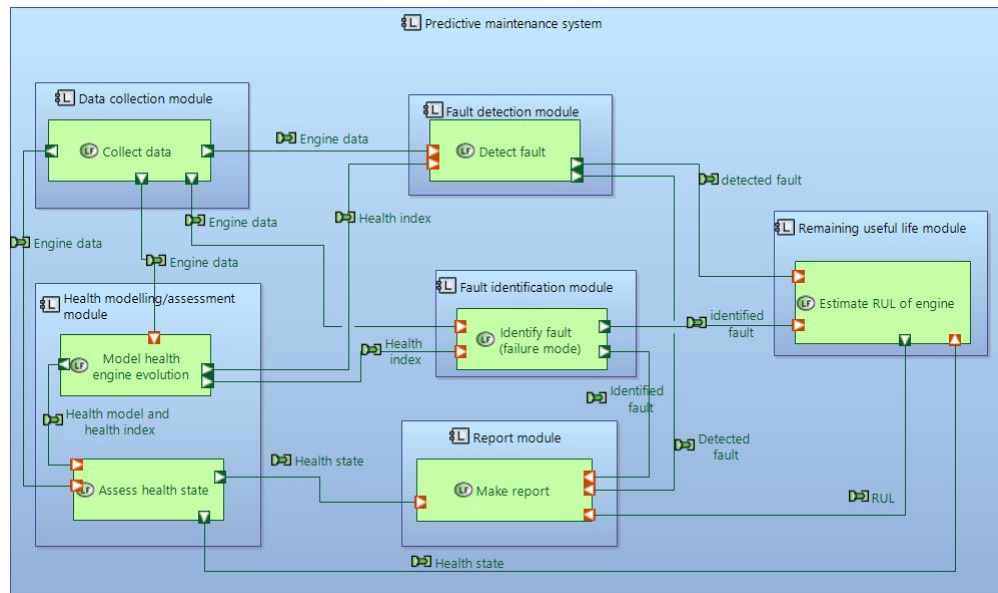


Figure 7.2: The logical architecture of a predictive maintenance system for the N-CMAPSS data set

7.4 Component selection using an ontology-enabled case-based recommendation system

Considering the logical architecture presented in Figure 7.2, there are four different logical components for which the Decision Support System can provide suggestions:

- Health modelling/assessment module
- Fault detection module
- Fault identification module
- Remaining useful life module

For each of these four logical components, a retrieval of possible solutions using the proposed DSS was performed. The results of the retrievals are summarized in Table 7.2. The DSS is capable to provide the similarity to each of the cases in the case base, but the most similar cases will have more relevance. For the current analysis and layout purposes of Table 7.2 only the five highest similarities are considered for each predictive maintenance function.

The results in Table 7.2 demonstrate the capabilities of the DSS to retrieve suitable solutions for each of the logical components. All the proposed models can be adapted to fulfil the intended function. All the

similarities between the target case and retrieved cases were higher than 0.776, meaning that the global similarity between proposed models and the target case was at least of 77.6%. An improvement opportunity for future research can be seen for those retrieved cases with exact same similarity; additional information would be needed to select the most suitable one. For example, for remaining useful life estimation module the DSS proposed five possible models with the same similarity value, no model is ranked higher than the other. An architect will need further information to perform the trade-off analysis and select the most suitable model to be implemented. Further similarity measures can be added to refine the retrieval process. It is important not to forget that in a first attempt the DSS is intended to provide suitable solutions, but there is a fair amount of remaining work so that the DSS is capable to suggest the most suitable among the others.

It is important to notice that the ontological similarity allowed to propose some models originally used for fault identification to fulfil fault detection and vice-versa. This inferred similarity is accurate as both functions are related to classification tasks. By adding more cases to the case base, more semantic similarities like this one can be found. The current values of similarity are low as the diversity in the case base is high compared to the case base size. Further explanations about the semantic similarity can be found in [MHMJV21], a conference publication performed in the framework of the current research, included in Appendix B.

In order to further validate the DSS recommendations, one of the suggested models has been taken as an example and has been implemented to fulfil the corresponding predictive maintenance function. Taking advantage of the research team experience in Self-Organizing Maps (SOM), an implementation of the health modelling component is performed using the SOM. The DSS recommends the SOM and logistic regression for health modelling as the most suitable models (similarity equal to 0.897) for the N-CMAPSS case study. The following section provides further explanations about the SOM example implementation and its preliminary results.

The methodology to implement the Self-Organizing Maps (SOM) for health modelling has been adopted from [Sch+20], (see Appendix A). Self-Organizing Maps are artificial neural networks with unsupervised training that are capable to cluster instances of data depending on the instance attributes. SOM is normally composed of a square layer of neurons and the different clusters after the SOM training can be graphically represented on the maps as well defined regions. It has been successfully used to model the degradation process of different machines such as jet engines [MV18]. In these cases, the neurons represent the health or degradation of the machine at a specific operational mode. The trained SOM will have a single region but a transition from white (optimal state) to black (failed state). When assessing the health or degradation of a machine using the trained SOM, a neuron will be excited on the map showing how advanced the degradation is or how much the health has decreased. This is the actual goal of the current implementation: to obtain a trained SOM capable to show a transition from optimal state to failed state.

In a first attempt, a simplified implementation is selected. To train the SOM, only sub-set of the N-CMAPSS data is used. The DS01 has been selected as it has only one failure mode. The first challenge in the adaptation of the SOM solution to the N-CMAPSS database is related to the operational modes. To train the SOM, the different operational modes must be distinguishable from one another; unfortunately, the N-CMAPSS is not focused on the different operational modes of the engine but in the complete flight envelope. To solve this challenge an operational mode has been defined when the engine reaches 10000 feet of altitude when the aircraft is climbing. At that operational mode, the throttle-resolver angle will be always higher than 70% and the flight Mach number is always in the same interval. These three variables are presented in the N-CMAPSS as scenario descriptors and are independent of the engine operation.

A second challenge in the adaptation of the SOM for the N-CMAPSS was related to the variables selected for the SOM training. The N-CMAPSS is composed by 45 different variables of different natures: scenario descriptors, operational engine measurements, virtual sensors and model health parameters. For the current implementation, only the scenario descriptors and the operational engine measures (see Table 7.3) have been

Table 7.2: Models retrieval for the N-CMAPSS examples

Target case function	Case	PdM Function	Model	Similarity
Fault detection	166	Fault detection	Gaussian process classifier	0.859
	211	Fault detection	Piecewise linear model (PWL), Hybrid Kalman Filter, OBEM model	0.834
	49	Fault identification	Support vector machines	0.776
	48	Fault identification	Deep belief network	0.776
	50	Fault identification	Multi-layer perceptron neural network	0.776
Fault identification	48	Fault identification	Deep belief network	0.859
	50	Fault identification	Multi-layer perceptron neural network	0.859
	49	Fault identification	Support vector machines	0.859
	212	Fault detection	Piecewise linear model (PWL), Hybrid Kalman Filter, OBEM model	0.834
	166	Fault detection	Gaussian process classifier	0.794
Health modelling /assessment	12	Health modelling	Self-organizing maps	0.897
	1	Health modelling	Logistic regression	0.897
	57	Health modelling	Statistical regression model	0.854
	159	Health modelling	Hidden Markov Chains	0.852
	168	Health modelling	Copula based sampling	0.838
Remaining useful life estimation	51	Remaining useful life estimation	LSTM (Long-Short Term Memory Neural Network)	0.895
	53	Remaining useful life estimation	Recurrent Neural Network	0.895
	54	Remaining useful life estimation	Gated recurrent unit network	0.895
	62	Remaining useful life estimation	Relevance vector machine	0.895
	64	Remaining useful life estimation	Bayesian linear regression	0.895

selected for the training as they represent the variables that can be obtained from real aircraft engines. The virtual sensors and the model health parameters are part of the simulation and model consistency but they would not be available in real jet engines.

Table 7.3: Operational Measurements N-CMAPSS

Symbol	Description	Units
Wf	Fuel flow	pps
Nf	Physical fan speed	rpm
Nc	Physical core speed	rpm
T24	Total temperature at LPC outlet	°R
T30	Total temperature at HPC outlet	°R
T48	Total temperature at HPT outlet	°R
T50	Total temperature at LPT outlet	°R
P2	Total pressure at fan inlet	psia
P15	Total pressure in bypass-duct	psia
P21	Total pressure at fan outlet	psia
P24	Total pressure at LPC outlet	psia
Ps30	Static pressure at HPC outlet	psia
P40	Total pressure at burner outlet	psia
P50	Total pressure at LPT outlet	psia

LPC: Low Pressure Compressor
HPC: High Pressure Compressor
LPT: Low Pressure Turbine
HPT: High Pressure Turbine

To increase the convergence changes in the SOM training, it is advisable to add the most representative inputs. A good practice is to delete all variables that present a binary or a constant behaviour as they do not provide any useful information about the degradation. A correlation test of the variables is also recommended to avoid adding redundant variables to the SOM training. Both of these tests have been performed on the operational measurements. No variables presented constants or binary records. Figure 7.3 shows a correlation matrix for the operational measures in which the same variables are in the horizontal and vertical axes; the correlation value is shown in the intersection of a vertical and horizontal axis. A correlation threshold has been set at 0.9. If two or more variables have a correlation higher than the threshold, only one of them will be selected at random as they will provide almost the same information. It is important to point out that before the correlation test, all variables were normalized between 0 and 1 which is also a requirement for the SOM training [Sch+20]. After performing the variables assessment, only four of them were selected to train the map, which represents a reduction in the number of variables compared to the study in Appendix A:

1. Total temperature at Low-Pressure Compressor (LPC) outlet.
2. Total temperature at High-Pressure Turbine (HPT) outlet.
3. Total pressure at fan inlet.
4. Total pressure at Low-Pressure Turbine (LPT) outlet

Once the variables have been selected and normalized, the training of the SOM can start. Given the data size a map of 5x5 neurons in square architecture has been selected. Several SOM's were trained and their convergence and behaviour were confirmed in the results. Figure 7.4 shows an example of a trained

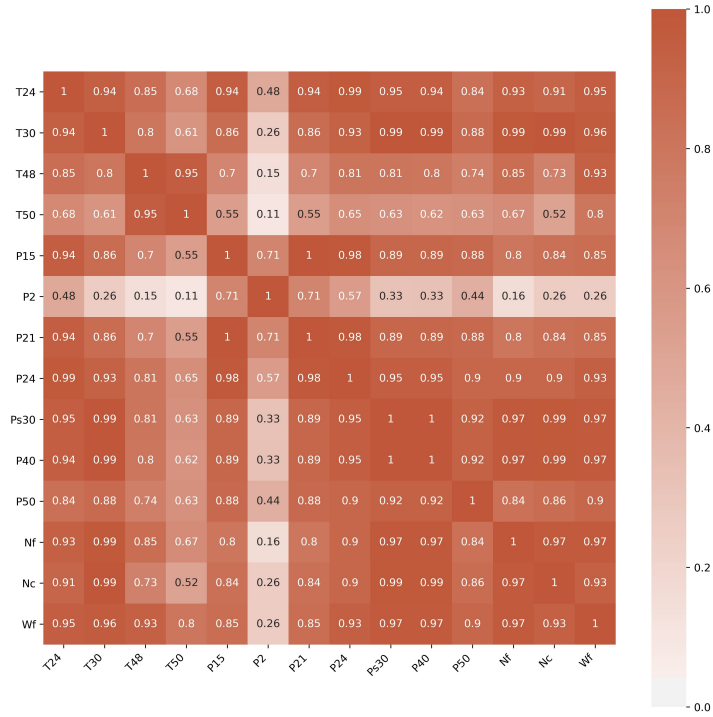


Figure 7.3: Correlation matrix of the sensors of the N-CMAPSS data set to be used as input for the SOM

map using the extracted data from the N-CMAPSS database. The clearest neuron represents the optimal operation condition of the engine and the darkest red neuron represents the condition of the engine just before its failure. An assessed engine with this SOM will excite a neuron between these two limits and its degradation can be estimated using the weights of the neuron. A transition from clear to darks is observable in the graphical representation of the SOM. This represents the degradation of the engine at the selected operational mode. For further explanations about the training and interpretation of the SOM, please read the article in Appendix A.

7.5 Discussion

It is important to recall that the validation of a Decision Support System (DSS) is a non-trivial task. The validation of the implemented model suggested by the DSS can be used to indirectly validate the DSS but further implementations would be required to confirm the capabilities and accuracy of the proposed DSS for predictive maintenance models selection. In the current implementation, the N-CMPASS represents the target case for which a new predictive maintenance system has to be developed. The ontology-enabled CBR recommendation system (also called DSS in this manuscript) proposed different models to fulfil each predictive maintenance function. One of the models proposed by the DSS for the function health modelling was the Self-Organizing Map (SOM). The SOM implementation successfully showed the degradation trend of jet-engines from nominal operation to failed condition using a subset of the N-CMPASS. The trained SOM can be also used to assess the health/ degradation of other engines of the same type and under the same operation conditions. It is important to clarify that this validation aims to demonstrate the suitability of the model proposed by the DSS, but further comparisons are needed among the suggested models are necessary to determine the best model. While performing the cross-validation presented in Chapter 6 and the current validation based on the SOM implementation, several improvement points have been identified:

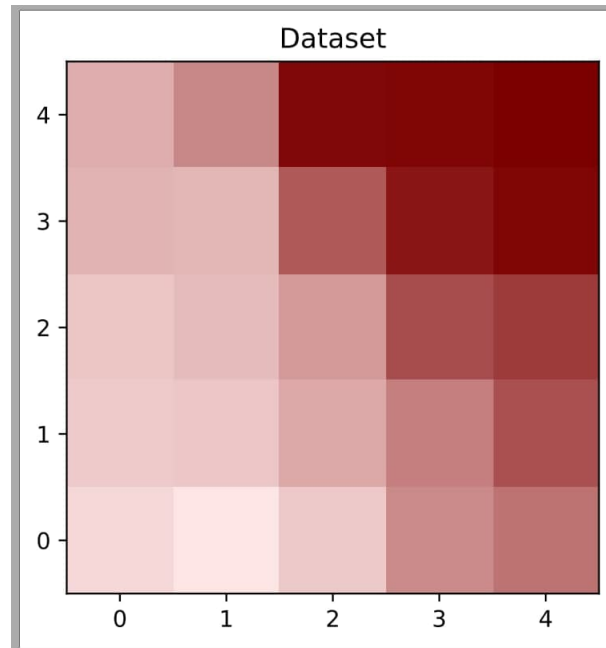


Figure 7.4: Trained SOM for the sub-set DS01

- Problems with the diversity in the retrieved cases: Even if it was not the case for this N-CMAPSS case study, in some retrievals of the cross-validation, the DSS proposed the same model several times with different references in the case base. This is a problem from a systems engineering point of view. An architect looking for innovative solutions needs a diverse list of possible solutions from the DSS. Further work can be done to avoid this diversity problem. Cases generalization and some other case diversity techniques can be applied to overcome this problem [De +05]. Such a situation could also be attributed to a potential problem of the coverage of the solution space. If the stored cases correspond only to a small portion of the solution space, the models suggested by the DSS can be very similar. For the current study this is not the case; models from all over the solution space have been considered. The state-of-the-art study in Chapter 2 allowed determining that the solution space for predictive maintenance systems is composed of data-driven models, knowledge-based models, physics-based models and multi-model approaches that combine at least two models from any of the three mentioned model families. The case base for the DSS was built considering all the solution space but the validation showed that this can be improved. A refinement of the solution space can be done with regards to each predictive maintenance function. For diagnostics functions such as for example “feature extraction” only data-driven models have been identified while creating the the case base. Another example can be seen for the function “remaining useful life estimation” where only data-driven and physics-based models have been identified. A refinement of the solution space considering each predictive maintenance function separately can help to improve the case base and the diversity in the models retrieved with the DSS.
- Additional information to perform the trade-off analysis: Several retrieved models for the same predictive maintenance function have the same similarity. Implement several solutions using different models that have the same similarity may not always be feasible for time and resources limitations. This poses a challenge for the architect to select the most suitable. Some other attributes could be added to the case so that the retrieved information can be used to perform a trade-off analysis so that the DSS can help the architect to make an accurate decision with regard to the model selection. Performance indicators, computational power required, implementation complexity, implementation cost, are some examples of complementary variables that can help to assess the models proposed by

the DSS. The additional attributes can be also used to add the preferences or initial constraint set by the DSS user from the beginning. Some users may be interested only in data-driven models, then the solution could be limited before the retrieval of models.

- Information about the adaptation of the proposed model: The current version of the DSS can be improved by including additional information for the adaptation of the suggested model in the target case. This is not only useful for the trade-off analysis, but also for the detailed design and implementation stage of the predictive maintenance system. The implementation of the SOM for health modelling has shown that some hyper-parameters of the models must be changed. A complementary guide for the adaptation of the suggested models can be added to facilitate its implementation in the target case. An expert system may guide the architect with the adaptation steps for each of the models. For time limitations, this is not possible for the current research.

7.6 Lessons learnt from the DSS validation using the N-CMAPSS

The validation of the proposed DSS for predictive maintenance component selection has been presented in this chapter. The capabilities of the DSS have been tested with the implementation of one of the suggested models for health modelling for the N-CMAPSS case study. This validation helped to demonstrate not only that the proposed DSS is capable to suggest suitable components for predictive maintenance systems, but also it allowed to identify improvement points of the DSS. A DSS is difficult to validate with a limited amount of data and with a limited number of implementations of the suggested models, but the success in the implementation of the SOM for health modelling of the N-CMAPSS case study indirectly validates the DSS capabilities. In the following and last chapter, the lessons learnt in the research are summarized in the conclusions and the perspectives of future work proposed.

Intentionally left blank

Conclusion and perspective for future work

“No tengas miedo de la perfección,
nunca la alcanzarás.”
“Have no fear of perfection,
you’ll never reach it.”

Salvador Dalí

Content

8.1	The journey is coming to an end, but a new one starts...	121
8.2	Lessons learnt	121
8.3	Limitations encountered	123
8.4	Contributions summary	124
8.5	Perspectives of future work	125
8.5.1	Perspectives related to the systems engineering approach to predictive maintenance systems design	125
8.5.2	OMSSA and OPMAD perspectives	125
8.5.3	Ontology-enabled case-based reasoning system perspectives	126
8.6	Epilogue	127

8.1 The journey is coming to an end, but a new one starts...

Research never ends but the current manuscript has already covered a wide field in the design of predictive maintenance systems and how the knowledge reuse from previous experiences can enhance the architecture process of such systems. Acknowledging that for the addressed topics there is still a long way to be covered, this chapter concludes the content of this manuscript by summarizing the lessons learnt, the limitations encountered, the research contributions, and more importantly the perspectives of future work that can serve as inspiration for future research lines.

8.2 Lessons learnt

To summarize the lessons learnt during this research it is necessary to come back to the research questions that initially motivated this work. In the introduction the first research questions was introduced:

1. What are the current trends in diagnostics and prognostics in predictive maintenance?
2. What are the main challenges in predictive maintenance?
3. What are the main research opportunities in the field of predictive maintenance?

These questions motivated an extensive literature review, that allowed to understand that predictive maintenance is carried out by specialized tools that incorporate models able to perform diagnostics and prognostics. The implementation of these models can be divided in two different approaches: single-model approaches and multi-model approaches. Single model approaches can be divided into three main model families: data-driven models, knowledge-based models and physics-based models. Multi-model approaches include at least two models from any of the mentioned model families. Depending on the multi-model approaches configuration, they can be called hybrid models. Recent trends in predictive maintenance push towards the implementation of multi-model approaches as one single model hardly addresses all the predictive maintenance functions for a complex system. The literature review allowed to determine the current challenges in predictive maintenance which include:

- Lack of a systematic method to design predictive maintenance systems.
- The extrapolation of existing solutions to complex system applications.
- The fusion of large and different sources of condition monitoring data.
- The incorporation of external influence data.
- Uncertainty management.

The main part of the challenges are focused in the concept stage and the design of new predictive maintenance systems. This motivated a refinement of the research questions:

1. How to address the design of predictive maintenance systems?
2. How to suggest a suitable approach for a predictive maintenance systems solution?
3. How to select a suitable model or combination of models given a new predictive maintenance system to solve?
4. How can a designer benefit from the experience of existing systems to develop new predictive maintenance solutions?

With regards to the first refined question, a systems engineering approach is proposed in the current research to address the concept stage of new predictive maintenance systems. This systematic approach starts by gathering all initial needs and desires from the stakeholders. The needs and desires are translated into a formal set of stakeholder requirements which are divided into four main categories: functional requirements, behavioural requirements, structural requirements and experiential requirements. The classified requirements are prioritized and later used at different phases of the system architecture process.

While developing this systems engineering approach, a challenge was identified at the logical architecture which also matches with the other refined research questions. The logical architecture shows as much detail as possible of the new system but without engaging it to any specific technology, meaning that the logical architecture still remains generic. The selection of predictive maintenance models to fulfil the logical components became the main scope of the current research. While performing the literature review, hundreds

of publications with successful cases of implemented predictive maintenance systems were consulted; the information from these implementations is not being efficiently used. The hypothesis was proposed: a Case-Based Reasoning (CBR) system can help an architect to retrieve efficiently a suitable model or set of models to perform a structured creativity process and fulfil the logical components in a new predictive maintenance system.

The CBR paradigm was selected as it offers means of reasoning to solve problems based on previous experiences. It is more flexible than rule-based reasoning and it is suitable for the current research as no specific rules exist to link the predictive models to a specific function in the system or to a specific case study. To develop a CBR system one of the most important aspects is a strong vocabulary basis from which the cases and the similarities can be modelled. For the current research, the state-of-the-art in vocabulary development for CBR was considered. It concerns ontologies, which are formal terminology representations that have a wide field of applications, especially in the semantic web. The research in ontologies allowed to identify an opportunity to make a contribution in the topic. Two Ontologies have been developed for the current research, the first one called OMSSA corresponds to a terminology framework to select and assess maintenance strategies. The second one, called OPMAD, is an OMSSA extension and covers the vocabulary involved in the design of predictive maintenance systems. OPMAD is the ontology that is used as a vocabulary framework for the proposed DSS.

It is important to mention that CBR and ontologies not always were implemented together. In recent years, both technologies have been increasingly used together because ontologies boost the use of the vocabulary basis for CBR applications. In the proposed framework, an ontology-enabled CBR recommendation system (a DSS) was developed to help the architect in the selection of suitable predictive maintenance models. A crossed validation helped to demonstrate that the proposed DSS was capable to suggest suitable components for specific predictive maintenance functions. To further validate the DSS, an implementation example was performed using the N-CMAPPs case study. The DSS proposed the Self-Organizing Maps (SOM) among the possible options to address health modelling. During the implementation of the SOM, it was possible not only to confirm the capabilities of the DSS but also to identify its improvement points. These improvement points are part of the perspectives of future works presented in this chapter.

In general, the research allowed to understand the principles of knowledge reuse with the CBR paradigm and how it can be used for architecture and design purposes. The implementation of the DSS allowed suggesting suitable components to fulfil generic components of a logical architecture of a predictive maintenance system.

8.3 Limitations encountered

As for any other research project, several limitations have been found on the way. This section aims at sharing part of the personal experience when performing the current research. Two main limitations have been faced. These are not specific for the current research but rather generic in the current context:

The first one and most important would be the limitation of time. It is understood that research never ends and at the end of any PhD thesis there will always remaining works to be done. For the current research, this is not an exception. Besides the normal workload that a PhD thesis represents, the time frame in which it was developed was also characterized by a global pandemic that forced to close the laboratories in France and all over the world. This affected the research schedule and forced the research objectives refinement. The research topic had to change in the middle of the second year and impacts due to the pandemic were considered to remake the research schedule.

The second limitation was related to the lack of access to practical examples that can be used to further

validate the DSS capabilities. There were conversations with different companies and groups able to provide practical examples that could have been included in this research validation. Bureaucracy and lack of interest from the external actors did not allow to have the case studies on time to be added to the current manuscript. Conversations with public companies in Costa Rica still undergoing so that the research can be continued in the near future.

Having faced these limitations enriches the research experience. It helps the researcher to be resilient and creative when facing problems out of their hands.

8.4 Contributions summary

The current manuscript is composed of four articles that have been published or submitted for being considered for publication in international journals and/or conferences. The main contributions of the PhD research have been consolidated in the articles as follows:

1. The review article summarized the state-of-the-art in diagnostics and prognostics. A proposal to differentiate hybrid models from multi-model approaches has been provided. The tendencies towards the implementation of multi-model approaches have been pointed out. By the time this manuscript was finished, the review article was cited more than thirty times. The literature review can be seen as a scientific contribution as it provides a starting point for those researchers interested in predictive maintenance.
2. The systems engineering approach to predictive maintenance systems design has been the first scientific contribution published in an international conference during the PhD development. In contrast to existing generic architectures for the design of predictive maintenance systems, the proposed systematic approach addresses the concept stage of predictive maintenance systems from the initial needs and desires until the logical architecture. It helps the architect to accurately determine the components of the system and keep traceability with the initial needs and desires obtained from the stakeholders.
3. Even if ontologies were not the main scope of the current research, a fair amount of research work has been performed in the domain and it has produced its own research outcomes. The creation of OMSSA, an ontology model for maintenance strategy selection and assessment provides a terminology framework that can be used by smart agents to automate the complex tasks of maintenance strategies management. An extension to OMSSA (OPMAD) is the ontology used to create the case base and the similarity measures of the DSS for predictive maintenance component selection. The research work performed in creating OMSSA has been consolidated in a journal article accepted for publication at the moment this manuscript was written.
4. The main scope of the current research has produced a decision support system able to query successful cases of predictive maintenance system implementations that can help a systems architect to select suitable predictive maintenance models to fulfil diagnostics and prognostics tasks. The first outcomes of this research have been published in a conference paper which has been published at an international conference while this manuscript was being completed.

8.5 Perspectives of future work

While working in research it is not possible to explain and prove everything. In order to accomplish the research objectives in a fixed time frame, some of the intermediate research steps have to be simplified and sometimes pragmatic decisions are made. When accomplishing the results, it is important to put things in perspective to spot the improvement points and list them as perspectives of future work. Research never ends, and one of the main parts of any PhD manuscript is to indicate what was missing and expected to be done in order to continue the research line. The perspectives of future work are organized in three main groups:

1. Perspectives related to the systems engineering approach to predictive maintenance systems design
2. Perspectives related to the ontologies developed in the current research
3. Perspective related to the ontology-enabled case-based recommendation system for predictive maintenance component selection.

8.5.1 Perspectives related to the systems engineering approach to predictive maintenance systems design

- Refine the list of predictive maintenance system needs, desires and requirements: after publishing the article related to the concept stage of predictive maintenance systems several needs, desires and requirements were identified that were not originally considered. The list of possible needs, desires and requirements for new predictive maintenance systems can be improved. These lists can be stored in a knowledge base that can be used by automated smart agents to help the engineers assess the initial needs and desires for a predictive maintenance system and suggest the corresponding stakeholder requirements. A DSS could be developed to define the correct stakeholder requirements based on the initial needs and desires for the new predictive maintenance system.
- Include trade-off analysis technique in the systematic framework: Trade-off analyses are present in several points during the concept stage. Due to time constraints, it was not addressed in this thesis. The integration of a trade-off analysis tool can help to improve the DSS and facilitate the selection of the components of the architecture.
- Include the formalization of system requirements: an important aspect of a systems engineering approach is the elicitation of system requirements. Even when the creation of these requirements is in theory before the architecture process, in practice these requirements are normally established in parallel with the development of the system architecture or even at the end when all the performances for the different logical components are known. These requirements are important for the detailed design stage. The elicitation of these requirements was out of the scope of the current research but it is an interesting complementary topic for future research.

8.5.2 OMSSA and OPMAD perspectives

- Refine classes and relations: ontology development is a fast evolving topic. There are several initiatives to provide top-level and mid-level ontologies in different domains. The industrial domain is not the exception. Further work will be needed to align OMSSA and OPMAD to these standardized ontologies so that to boost their integration and reuse in other applications. This work includes the refinement of classes and relations among them.

- Extend reasoning in the ontology: OMSSA and OPMAD have been useful in the current research but as ontologies, they have been underutilised. Ontologies have several capabilities that were not exploited. One of them is the implementation of semantic rules. This extends the reasoning options and can be used as a complementary means to the CBR system.

8.5.3 Ontology-enabled case-based reasoning system perspectives

- Expand the use of the ontology for more local similarity functions: as it was already mentioned, the ontology has been useful but underutilised. Some other similarity measures that have been assigned a binary symbol similarity measure can be rearranged to use ontology-based similarities.
- Use the ontology to estimate the weights for the global similarity computation: related to the previous remark. the ontology can also be used to compute the weights of each local similarity when computing the global similarity. In the first attempt of global similarity computation performed in this research, all local similarities received the same weight. An improvement in the global similarity computation could be by giving a rank of importance to each local similarity. The weights can be computed using the populated ontology.
- Expand the Decision Support System by developing other phases of the CBR cycle, such as the adaptation phase: The scope for the DSS was on the retrieval phase of the CBR cycle. The system can be expanded to facilitate the adaptation phase of the suggested components for the new predictive maintenance system. This extension can also include the trade-off analysis on different components suggested by the DSS. The results of the current research shown several improvement opportunities that should be addressed in the future before eventual deployment of the DSS.
- Refine the instantiation process to avoid diversity problems but keeping all the solution space covered: As it was mentioned in the validation chapter, the instantiation of the case base should be refined. An improvement opportunity can be to divide the solution space according to the different predictive maintenance functions and make sure that the solution space is fully covered for each of them. Generalized cases for each predictive maintenance function can solve the problem of diversity in the retrieval cases.
- Trade-off analysis to better select the technique among the proposed ones: it was already mentioned as the perspective of future work for the systematic approach to design predictive maintenance systems. Regarding the DSS, the trade-off analysis can help the architect discriminate among the suggested components by the DSS. In the validation it was mentioned that sometimes the DSS suggested two different components with the same similarity. Adding some extra attributes such as performance indicators can help the architect make the right decision for their problem.
- Compare against other techniques for DSS: In a first attempt, the objective of the current research was to prove that case-based reasoning can be combined with ontologies to develop a Decision Support System for component selection for new predictive maintenance systems and it worked. Further research would be dedicated to assess the performance of the proposed DSS against other technologies that can be used for the same purposes such as machine learning algorithms. Such comparison was not considered as it exceeded the scope of the current research.

8.6 Epilogue

The content of the current manuscript has come to an end. It attempted to summarize the research experiences accumulated over the last three years specifically in the topic of predictive maintenance systems design, an interesting topic that still leaves a lot challenges to be solved. A systematic methodology to address the concept phase of such systems has been proposed and within this framework a more specific challenge was identified for suitable components selection. A Decision Support System composed of an ontology-enabled case-based reasoning retrieval engine has been proposed to overcome the challenge of suitable component selection based on past experiences. The results demonstrated the capabilities of the proposed DSS but also allowed to spot important improvement opportunities that should be considered in future research.

Intentionally left blank

Bibliography

- [AA10] Edgar J. Amaya and Alberto J. Alvares. “SIMPREGAL: An expert system for real-time fault diagnosis of hydrogenerators machinery”. In: *Proceedings of the 15th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2010*. 2010. ISBN: 9781424468508. DOI: 10.1109/ETFA.2010.5641302.
- [AGC18] Panagiotis Aivaliotis, Konstantinos Georgoulas, and George Chrysosouris. “A RUL calculation approach based on physical-based simulation models for predictive maintenance”. In: *2017 International Conference on Engineering, Technology and Innovation: Engineering, Technology and Innovation Management Beyond 2020: New Challenges, New Approaches, ICE/ITMC 2017 - Proceedings*. 2018. ISBN: 9781538607749. DOI: 10.1109/ICE.2017.8280022.
- [AH17] Sylvestre A. Aye and Philippus S. Heyns. “An integrated Gaussian process regression for prediction of remaining useful life of slow speed bearings based on acoustic emission”. In: *Mechanical Systems and Signal Processing* 84.A (2017), pp. 485–498. ISSN: 10961216. DOI: 10.1016/j.ymssp.2016.07.039.
- [AH18] Sylvester A. Aye and Stephan Heyns. “Prognostics of slow speed bearings using a composite integrated Gaussian process regression model”. In: *International Journal of Production Research* 56.14 (2018), pp. 4860–4873. ISSN: 1366588X. DOI: 10.1080/00207543.2018.1470340.
- [AIC18] Ronke M. Ayo-Imoru and Anthonie C. Cilliers. “Continuous machine learning for abnormality identification to aid condition-based maintenance in nuclear power plant”. In: *Annals of Nuclear Energy* 118 (2018), pp. 61–70. ISSN: 18732100. DOI: 10.1016/j.anucene.2018.04.002.
- [AKC15] Dawn An, Nam H. Kim, and Joo Ho Choi. “Practical options for selecting data-driven or physics-based prognostics algorithms with reviews”. In: *Reliability Engineering and System Safety* 133 (2015), pp. 223–236. ISSN: 09518320. DOI: 10.1016/j.res.2014.09.014.
- [AL13] Khaled Amaief and Jie Lu. “Ontology-supported case-based reasoning approach for intelligent m-Government emergency response services”. In: *Decision Support Systems* 55 (2013), pp. 79–97. ISSN: 01679236. DOI: 10.1016/j.dss.2012.12.034.
- [Alt+12] Klaus Althoff et al. *Documentation: myCBR*. 2012. URL: <http://www.mycbr-project.org/3.1-doc/index.html>.
- [Ant+12] Grigoris Antoniou et al. *A Semantic Web Primer*. Third edit. MIT Press, 2012. ISBN: 9780262018289.
- [AP94] Agnar Aamodt and Enric Plaza. “Case-Based reasoning: Foundational issues, methodological variations, and system approaches”. In: *AI Communications* 7.1 (1994), pp. 39–59. ISSN: 09217126. DOI: 10.3233/AIC-1994-7104.
- [ASF14] Pekka Aarnio, Ilkka Seilonen, and Mats Friman. “Semantic repository for case-based reasoning in CBM services”. In: *19th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2014*. 2014. ISBN: 9781479948468. DOI: 10.1109/ETFA.2014.7005195.
- [ASS16] Robert Arp, Barry Smith, and Andrew D. Spear. *Building Ontologies with Basic Formal Ontology*. 2016. DOI: 10.7551/mitpress/9780262527811.001.0001.
- [AX17] Suzan Alaswad and Yisha Xiang. “A review on condition-based maintenance optimization models for stochastically deteriorating system”. In: *Reliability Engineering and System Safety* 157 (2017), pp. 54–63. ISSN: 09518320. DOI: 10.1016/j.res.2016.08.009.

- [Bag+15] Behrad Bagheri et al. “A Stochastic Asset Life Prediction Method for Large Fleet Datasets in Big Data Environment”. In: *ASME 2015 International Mechanical Engineering Congress and Exposition. Volume 14: Emerging Technologies; Safety Engineering and Risk Analysis; Materials: Genetics to Structures*. 2015. ISBN: 978-0-7918-5757-1. DOI: 10.1115/IMECE2015-52458.
- [Bai+15] Chris Bailey et al. “Prognostic and health management for engineering systems: a review of the data-driven approach and algorithms”. In: *The Journal of Engineering* 2015.7 (2015), pp. 215–222. ISSN: 2051-3305. DOI: 10.1049/joe.2014.0303.
- [Baj+09] Gautam Bajracharya et al. “Optimization of maintenance for power system equipment using a predictive health model”. In: *2009 IEEE Bucharest PowerTech: Innovative Ideas Toward the Electrical Grid of the Future*. 2009. ISBN: 9781424422357. DOI: 10.1109/PTC.2009.5281928.
- [BAJ17] Oguz Bektas, Amjad Alfudail, and Jeffrey A. Jones. “Reducing Dimensionality of Multi-regime Data for Failure Prognostics”. In: *Journal of Failure Analysis and Prevention* 17 (2017), pp. 1268–1275. ISSN: 15477029. DOI: 10.1007/s11668-017-0368-2.
- [BB09] Donald W Benbow and Hugh W Broome. *The Certified Reliability Engineer Handbook*. 2009. ISBN: 978-0-87389-721-1. DOI: 10.1017/CBO9781107415324.004. arXiv: arXiv:1011.1669v3.
- [BB+17] Diana Barraza-Barraza et al. “An adaptive ARX model to estimate the RUL of aluminum plates based on its crack growth”. In: *Mechanical Systems and Signal Processing* 82 (2017), pp. 519–536. ISSN: 10961216. DOI: 10.1016/j.ymssp.2016.05.041.
- [BB18] Toufik Berredjem and Mohamed Benidir. “Bearing faults diagnosis using fuzzy expert system relying on an Improved Range Overlaps and Similarity method”. In: *Expert Systems with Applications* 108 (2018), pp. 134–142. ISSN: 09574174. DOI: 10.1016/j.eswa.2018.04.025.
- [BBM18] Marius Baban, Calin Florin Baban, and Benjamin Moisi. “A Fuzzy Logic-Based Approach for Predictive Maintenance of Grinding Wheels of Automated Grinding Lines”. In: *2018 23rd International Conference on Methods and Models in Automation and Robotics, MMAR 2018*. 2018. ISBN: 9781538643259. DOI: 10.1109/MMAR.2018.8486144.
- [Ber+01] Ralph Bergmann et al. “Utility-Oriented Matching: A New Research Direction for Case-Based Reasoning”. In: *Proceedings of the Ninth German Workshop on Case-Based Reasoning*. 2001.
- [Ber+16] Maitane Bercibar et al. “Online state of health estimation on NMC cells based on predictive analytics”. In: *Journal of Power Sources* (2016). ISSN: 03787753. DOI: 10.1016/j.jpowsour.2016.04.109.
- [BKP05] Ralph Bergmann, Janet Kolodner, and Enric Plaza. “Representation in case-based reasoning”. In: *Knowledge Engineering Review* 20.3 (2005), pp. 209–213. ISSN: 02698889. DOI: 10.1017/S0269888906000555.
- [Bos+13] Kosta P. Boshnakov et al. “Predictive maintenance model-based approach for objects exposed to extremely high temperatures”. In: *2013 Signal Processing Symposium, SPS 2013*. 2013. ISBN: 9781467363198. DOI: 10.1109/SPS.2013.6623621.
- [Bra+18] Frances Brazier et al. “Design, Engineering and Governance of Complex Systems”. In: *Projects and People - Mastering success*. Ed. by H.L.M. Bakker and J.P. Kleynen. NAP Foundation Press, 2018, pp. 34–59.
- [But96] Karen L. Butler. “An Expert System Based Framework for an Incipient Failure Detection and Predictive Maintenance System”. In: *Proceeding of the Int Conf on Intelligent Sys Application to Power Sys* (1996). ISSN: 0020-7543. DOI: 10.1080/002075400188933. arXiv: arXiv:1011.1669v3.

- [BW96] Ralph Bergmann and Wolfgang Wilke. “On the role of abstraction in case-based reasoning”. In: *Proceeding of the 3rd European Workshop on Case-Based Reasoning*. Berlin: Springer, 1996, pp. 29–43.
- [CA+17] Vicente Climente-Alarcon et al. “Combined Model for Simulating the Effect of Transients on a Damaged Rotor Cage”. In: *IEEE Transactions on Industry Applications* (2017). ISSN: 00939994. DOI: 10.1109/TIA.2017.2691001.
- [Cas+20] Fernando Castaño et al. “Quality monitoring of complex manufacturing systems on the basis of model driven approach”. In: *Smart Structures and Systems* 26.4 (2020), pp. 495–506. ISSN: 17381991. DOI: 10.12989/sss.2020.26.4.495. URL: <http://techno-press.org/content/?page=article{\&}journal=sss{\&}volume=26{\&}num=4{\&}ordernum=7>.
- [CCN19] Duan Chaoqun, Deng Chao, and Li Ning. “Reliability assessment for CNC equipment based on degradation data”. In: *The International Journal of Advanced Manufacturing Technology* 100 (2019), pp. 421–434.
- [CCS15] Edward Crawley, Bruce Cameron, and Daniel Selva. *System Architecture: Strategy and Product Development for Complex Systems*. Pearson Higher Education, Inc., 2015.
- [Cer+16] Mariela Cerrada et al. “Fault diagnosis in spur gears based on genetic algorithm and random forest”. In: *Mechanical Systems and Signal Processing* 70-71 (2016), pp. 87–103. ISSN: 10961216. DOI: 10.1016/j.ymssp.2015.08.030.
- [CFZ17] Yang Chang, Huajing Fang, and Yong Zhang. “A new hybrid method for the prediction of the remaining useful life of a lithium-ion battery”. In: *Applied Energy* 206 (2017), pp. 1564–1578. ISSN: 03062619. DOI: 10.1016/j.apenergy.2017.09.106.
- [CH11] Jamie B. Coble and Wesley Hines. “Applying the General Path Model to Estimation of Remaining Useful Life”. In: *International Journal of Prognostics and Health Management* 2.1 (2011), p. 13. ISSN: 21532648.
- [Cha+21] Manuel Arias Chao et al. “Aircraft Engine Run-to-Failure Dataset under Real Flight Conditions for Prognostics and Diagnostics”. In: *Data* 2021, Vol. 6, Page 5 6.1 (2021), p. 5. DOI: 10.3390/DATA6010005. URL: <https://www.mdpi.com/2306-5729/6/1/5/htmlhttps://www.mdpi.com/2306-5729/6/1/5>.
- [Che+16] Peter Chemweno et al. “I-RCAM: Intelligent expert system for root cause analysis in maintenance decision making”. In: *2016 IEEE International Conference on Prognostics and Health Management, ICPHM 2016*. 2016. ISBN: 9781509003822. DOI: 10.1109/ICPHM.2016.7542830.
- [Che+17] Zhen Chen et al. “Hidden Markov model with auto-correlated observations for remaining useful life prediction and optimal maintenance policy”. In: *Reliability Engineering and System Safety* 184 (2017), pp. 123–136. ISSN: 09518320. DOI: 10.1016/j.ress.2017.09.002.
- [Chi+19] Juan Chiachío et al. “A knowledge-based prognostics framework for railway track geometry degradation”. In: *Reliability Engineering and System Safety* 181 (2019), pp. 127–141. ISSN: 09518320. DOI: 10.1016/j.ress.2018.07.004.
- [Cho+19] Michael E. Cholette et al. “Degradation modeling and condition-based maintenance of boiler heat exchangers using gamma processes”. In: *Reliability Engineering and System Safety* 183 (2019), pp. 184–196.
- [CNY10] Wahyu Caesarendra, Gang Niu, and Bo Suk Yang. “Machine condition prognosis based on sequential Monte Carlo method”. In: *Expert Systems with Applications* 37.3 (2010), pp. 2412–2420. ISSN: 09574174. DOI: 10.1016/j.eswa.2009.07.014.

- [CRW08] Jiehua Chen, Clive Roberts, and Paul Weston. “Fault detection and diagnosis for railway track circuits using neuro-fuzzy systems”. In: *Control Engineering Practice* 16 (2008), pp. 585–596. ISSN: 09670661. DOI: 10.1016/j.conengprac.2007.06.007.
- [CYC12] Cong Cheng, Ling Yu, and Liu Jie Chen. “Structural nonlinear damage detection based on ARMA-GARCH model”. In: *Applied Mechanics and Materials* 204-208 (2012), pp. 2891–2896. ISSN: 16609336. DOI: 10.4028/www.scientific.net/AMM.204-208.2891.
- [CZMR19] Qiushi Cao, Cecilia Zanni-Merk, and Christoph Reich. “Towards a core ontology for condition monitoring”. In: *Procedia Manufacturing*. 2019. DOI: 10.1016/j.promfg.2018.12.029.
- [De +05] Ramon Lopez De Mantaras et al. “Retrieval, reuse, revision and retention in case-based reasoning”. In: *The Knowledge Engineering Review* 20.3 (2005), pp. 215–240. ISSN: 02698889. DOI: 10.1017/S0269888906000646.
- [De +18] Massimiliano De Benedetti et al. “Anomaly detection and predictive maintenance for photovoltaic systems”. In: *Neurocomputing* 310 (2018), pp. 59–68. ISSN: 18728286. DOI: 10.1016/j.neucom.2018.05.017.
- [DHK13] Nadjet Dendani-Hadiby and M. Tarek Khadir. “A fault diagnosis application based on a combination case-based reasoning and ontology approach”. In: *International Journal of Knowledge-Based and Intelligent Engineering Systems* 17.4 (2013), pp. 305–317. ISSN: 13272314. DOI: 10.3233/KES-130280.
- [DLS17] Dong Dong, Xiao Yang Li, and Fu Qiang Sun. “Life prediction of jet engines based on LSTM-recurrent neural networks”. In: *2017 Prognostics and System Health Management Conference, PHM-Harbin 2017 - Proceedings*. 2017. ISBN: 9781538603703. DOI: 10.1109/PHM.2017.8079264. arXiv: arXiv:1512.04143v1.
- [DMB16] Landon T. Detwiler, Jose L.V. Mejino, and James F. Brinkley. “From frames to OWL2: Converting the Foundational Model of Anatomy”. In: *Artificial Intelligence in Medicine* 69 (2016), pp. 12–21. ISSN: 18732860. DOI: 10.1016/j.artmed.2016.04.003.
- [DMD18] Chaoqun Duan, Viliam Makis, and Chao Deng. “An integrated framework for health measures prediction and optimal maintenance policy for mechanical systems using a proportional hazards model”. In: *Mechanical Systems and Signal Processing* 111 (2018), pp. 285–302. ISSN: 10961216. DOI: 10.1016/j.ymssp.2018.02.029.
- [Dow+19] Austin Downey et al. “Physics-based prognostics of lithium-ion battery using non-linear least squares with dynamic bounds”. In: *Reliability Engineering and System Safety* 182 (2019), pp. 1–12.
- [Dra+09] Otilia Elena Dragomir et al. “Review of prognostic problem in condition-based maintenance”. In: *European Control Conference, (ECC’09)*. 2009. ISBN: 9783952417393. DOI: 10.1128/JB.00591-09.
- [DS18] Samalis Santini De León and Daniel Selva. “A rule-based tool for science traceability of Mars exploration mission architectures”. In: *IEEE Aerospace Conference Proceedings*. 2018. ISBN: 9781538620144. DOI: 10.1109/AERO.2018.8396804.
- [DZD14] Xiaofei Du, Yuanjun Zhou, and Shiliang Dong. “Residual life prediction from statistical features and a GARCH modeling approach for aircraft generators”. In: *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering* 228.1 (2014), pp. 137–146. ISSN: 20413025. DOI: 10.1177/0954410012472838.
- [ECJ14] Omer Faruk Eker, Fatih Camci, and Ian K Jennions. “A Similarity-based Prognostics Approach for Remaining Useful Life Prediction”. In: *Second European Conference of the Prognostics and Health Management Society 2014*. 2014.

- [ECJ16] Omer F. Eker, Fatih Camci, and Ian K. Jennions. “Physics-based prognostic modelling of filter clogging phenomena”. In: *Mechanical Systems and Signal Processing* 75 (2016), pp. 395–412. ISSN: 10961216. DOI: 10.1016/j.ymssp.2015.12.011.
- [EER16] Hatem M. Elattar, Hamdy K. Elminir, and Alaa el-din Mohamed Riad. “Prognostics: a literature review”. In: *Complex & Intelligent Systems* 2.2 (2016), pp. 125–154. ISSN: 2199-4536. DOI: 10.1007/s40747-016-0019-3.
- [ENM18] Ikuobase Emovon, Rosemary A. Norman, and Alan J. Murphy. “Hybrid MCDM based methodology for selecting the optimum maintenance strategy for ship machinery systems”. In: *Journal of Intelligent Manufacturing* 29 (2018), pp. 519–531. ISSN: 15728145. DOI: 10.1007/s10845-015-1133-6.
- [ESE15] Shaker El-Sappagh and Mohammed Elmogy. “Case Based Reasoning: Case Representation Methodologies”. In: *International Journal of Advanced Computer Science and Applications* 6.11 (2015), pp. 192–208. ISSN: 2158107X. DOI: 10.14569/ijacsa.2015.061126.
- [Eur09] European standard NF EN 13306X60-319. *Maintenance — Terminologie de la maintenance*. 2009.
- [Eur17] European Committee for Standardization. *CEN EN 13306: Maintenance-Maintenance terminology*. 2017.
- [FDL07] Dean K Frederick, Jonathan DeCastro, and Jonathan Litt. *User’s guide for the Commercial Modular Aero-Propulsion System Simulation (C-MAPSS)*. Tech. rep. Washington, DC: NASA, 2007.
- [Fer+11] Miriam Fernández et al. “Semantically enhanced Information Retrieval: An ontology-based approach”. In: *Journal of Web Semantics* 9.1 (2011), pp. 434–452. ISSN: 15708268. DOI: 10.1016/j.websem.2010.11.003.
- [Fer+18] Borja Ramis Ferrer et al. “Towards the Adoption of Cyber-Physical Systems of Systems Paradigm in Smart Manufacturing Environments”. In: *Proceedings - IEEE 16th International Conference on Industrial Informatics, INDIN 2018*. 2018. ISBN: 9781538648292. DOI: 10.1109/INDIN.2018.8472061.
- [FGPJ97] Mariano Fernandez, Asuncion. Gómez-Pérez, and Natalia Juristo. “Methontology: from ontological art towards ontological engineering”. In: *Proceedings of the AAAI97 Spring Symposium Series on Ontological Engineering*. 1997.
- [Fou20] The OBO Foundry. *Relation Ontology (RO)*. 2020. URL: <http://www.obofoundry.org/ontology/ro.html> (visited on 07/22/2020).
- [Fra57] Alexander S. Fraser. “Simulation of Genetic Systems by Automatic Digital Computers I”. In: *Australian Journal of Biological Science* 10 (1957), pp. 484–491.
- [Fre91] Bernd Freyermuth. “Knowledge based incipient fault diagnosis of industrial robots”. In: *IFAC Proceedings Volumes (IFAC-PapersOnline)* 24.6 (1991), pp. 369–375.
- [FZ15] Liu Fang and Huang Zhaodong. “System Dynamics Based Simulation Approach on Corrective Maintenance Cost of Aviation Equipments”. In: *Procedia Engineering*. Vol. 99. Elsevier Ltd, 2015, pp. 150–155. DOI: 10.1016/j.proeng.2014.12.519.
- [Gan+15] Ma Gang et al. “A model of intelligent fault diagnosis of power equipment based on CBR”. In: *Mathematical problems in engineering* Article ID 203083 (2015).
- [Ger90] John S. Gero. “Design Prototypes: A knowledge Representation Schema for Design”. In: *AI Magazine* 11.4 (1990), pp. 26–36.
- [GF95] Michael Grüninger and Mark Stephen Fox. “Methodology for the Design and Evaluation of Ontologies”. In: *International Joint Conference on Artificial Intelligence (IJCAI95), Workshop on Basic Ontological Issues in Knowledge Sharing*. 1995. DOI: citeulike-article-id:1273832.

- [GK07] John S. Gero and Udo Kannengiesser. “A function-behavior-structure ontology of processes”. In: *Artificial Intelligence for Engineering Design, Analysis and Manufacturing: AIEDAM*. 2007. DOI: 10.1017/S0890060407000340.
- [GMZ16] Rafael Gouriveau, Kamal Medjaher, and Nouredine Zerhouni. *From Prognostics and Health Systems Management to Predictive Maintenance 1: Monitoring and Prognostics*. 2016. ISBN: 9781119371052. DOI: 10.1002/9781119371052.
- [Goy+16] Deepam Goyal et al. “Intelligent predictive maintenance of dynamic systems using condition monitoring and signal processing techniques-A review”. In: *Proceedings - 2016 International Conference on Advances in Computing, Communication and Automation, ICACCA 2016*. 2016. ISBN: 9781509006731. DOI: 10.1109/ICACCA.2016.7578870.
- [GPH13] Yuan Guo, Yinghong Peng, and Jie Hu. “Research on high creative application of case-based reasoning system on engineering design”. In: *Computers in Industry* 64.1 (2013), pp. 90–103. ISSN: 0166-3615. DOI: 10.1016/J.COMPIND.2012.10.006.
- [Gru93] Thomas R. Gruber. “A translation approach to portable ontology specifications”. In: *Knowledge Acquisition* 5.2 (1993), pp. 199–220. ISSN: 10428143. DOI: 10.1006/knac.1993.1008.
- [Guo+17] Liang Guo et al. “A recurrent neural network based health indicator for remaining useful life prediction of bearings”. In: *Neurocomputing* 240 (2017), pp. 98–109. ISSN: 18728286. DOI: 10.1016/j.neucom.2017.02.045.
- [GV17] Jakub Gajewski and David Vališ. “The determination of combustion engine condition and reliability using oil analysis by MLP and RBF neural networks”. In: *Tribology International* 115 (2017), pp. 557–572. ISSN: 0301679X. DOI: 10.1016/j.triboint.2017.06.032.
- [Han+19] Houman Hanachi et al. “Hybrid sequential fault estimation for multi-mode diagnosis of gas turbine engines”. In: *Mechanical Systems and Signal Processing* 115 (2019), pp. 225–268. ISSN: 10961216. DOI: 10.1016/j.ymssp.2018.05.054.
- [HB11] Matthew Horridge and Sean Bechhofer. “The OWL API: A Java API for OWL ontologies”. In: *Semantic Web 2.1* (2011), pp. 11–21. ISSN: 15700844. DOI: 10.3233/SW-2011-0025.
- [HLC14] Bahareh Rahmanzadeh Heravi, Mark Lycett, and Sergio de Cesare. “Ontology-based standards development: Application of OntoStanD to ebXML business process specification schema”. In: *International Journal of Accounting Information Systems* 15.3 (2014), pp. 275–297. ISSN: 14670895. DOI: 10.1016/j.accinf.2014.01.005.
- [HLP08] Frank van Harmelen, Vladimir Lifschitz, and Bruce Porter. *Handbook of Knowledge Representation*. 2008. ISBN: 9780444522115. DOI: 10.1016/S1574-6526(07)03013-1.
- [Hod+21] Melinda Hodkiewicz et al. “Rethinking Maintenance Terminology for an Industry 4.0 Future”. In: *International Journal of Prognostics and Health Management* 2021.1 (2021), p. 14. URL: <https://www.phmsociety.org/node/2794>.
- [Hoe09] Rinke Hoekstra. *Ontology representation: Design patterns and ontologies that make sense*. IOS Press, 2009. ISBN: 9781607500131. DOI: 10.3233/978-1-60750-013-1-i.
- [HSC18] Moinul Shaidul Haque, Mohammad Noor Bin Shaheed, and Seungdeog Choi. “RUL Estimation of Power Semiconductor Switch using Evolutionary Time series Prediction”. In: *2018 IEEE Transportation and Electrification Conference and Expo, ITEC 2018*. 2018. ISBN: 9781538630488. DOI: 10.1109/ITEC.2018.8450131.
- [HT18] Ahmed Zakariae Hinch and Mohamed Tkouat. “Rolling element bearing remaining useful life estimation based on a convolutional long-short-Term memory network”. In: *Procedia Computer Science* 127 (2018), pp. 123–132. ISSN: 18770509. DOI: 10.1016/j.procs.2018.01.106.

- [Hu+12] Chao Hu et al. “Ensemble of data-driven prognostic algorithms for robust prediction of remaining useful life”. In: *Reliability Engineering and System Safety* 103 (2012), pp. 120–135. ISSN: 09518320. DOI: 10.1016/j.ress.2012.03.008.
- [Hu+18] Ya-Wei Hu et al. “Sequential Monte Carlo Method Toward Online RUL Assessment with Applications”. In: *Chinese Journal of Mechanical Engineering* 31.5. <https://doi.org/10.1186/s10033-018-0205-x> (2018). ISSN: 1000-9345. DOI: 10.1186/s10033-018-0205-x.
- [Hus+15] Akhtar Hussain et al. “An expert system for acoustic diagnosis of power circuit breakers and on-load tap changers”. In: *Expert Systems with Applications* 42.24 (2015), pp. 9426–9433. ISSN: 09574174. DOI: 10.1016/j.eswa.2015.07.079.
- [INC15] INCOSE. *Systems Engineering Handbook. A guide for system life cycle processes and activities. Fourth Edition*. Wiley, 2015.
- [Int03] International Organization for Standardization (ISO). *ISO 13374-1:2003 Condition monitoring and diagnostics of machines — Data processing, communication and presentation — Part 1: General guidelines*. 2003.
- [Int12a] International Organization for Standardization (ISO). *ISO 13372 - Condition monitoring and diagnostics of machines Vocabulary*. 2012.
- [Int12b] International Organization for Standardization (ISO). *ISO 13379-1:2012 - Condition monitoring and diagnostics of machines Data interpretation and diagnostics techniques Part 1: General guidelines*. 2012.
- [Int18] International Electrotechnical Commission (IEC). *IEC60812, Analysis techniques for system reliability- Procedure for failure mode and effects analysis (FMECA)*. 2018.
- [Int20a] International Organization for Standardization (ISO). *ISO/IEC 21838-1 Information technology - Top-level Ontologies (TLO) - Part 1: Requirements*. 2020.
- [Int20b] International Organization for Standardization (ISO). *ISO/IEC 21838-2 - Information technology - Top-level ontologies (TLO) - Part 2: Basic Formal Ontology (BFO)*. 2020.
- [Int97] International Organization for Standardization (ISO). *Information technology — Vocabulary — Part 14: Reliability, maintainability and availability*. 1997.
- [IOF20] IOF. *Industrial Ontology Foundry*. 2020. URL: https://www.industrialontologies.org/?page{_}id=164 (visited on 10/06/2020).
- [ISO11] ISO/IEC/IEE. *42010:2011 Systems and Software Engineering — Architectural Description*. Ed. by ISO/IEC 42010:2011 International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). 2011.
- [JGZ17] Kamran Javed, Rafael Gouriveau, and Nouredine Zerhouni. “State of the art and taxonomy of prognostics approaches, trends of prognostics applications and open issues towards maturity at different technology readiness levels”. In: *Mechanical Systems and Signal Processing* 94 (2017), pp. 214–236. ISSN: 10961216. DOI: 10.1016/j.ymssp.2017.01.050.
- [Jin+15] Wenjing Jin et al. “Development and evaluation of health monitoring techniques for railway point machines”. In: *2015 IEEE Conference on Prognostics and Health Management: Enhancing Safety, Efficiency, Availability, and Effectiveness of Systems Through PHAf Technology and Application, PHM 2015*. 2015. ISBN: 9781479918935. DOI: 10.1109/ICPHM.2015.7245016.
- [JLB06] Andrew K.S. Jardine, Daming Lin, and Dragan Banjevic. “A review on machinery diagnostics and prognostics implementing condition-based maintenance”. In: *Mechanical Systems and Signal Processing* 20.7 (2006), pp. 1483–1510. ISSN: 08883270. DOI: 10.1016/j.ymssp.2005.09.012. arXiv: 0208024 [gr-qc].

- [Kar+19] Mohamed Hedi Karray et al. "ROMAIN: Towards a BFO compliant reference ontology for industrial maintenance". In: *Applied Ontology* 14.2 (2019), pp. 155–177. ISSN: 18758533. DOI: 10.3233/AO-190208.
- [KCL18] Dongdong Kong, Yongjie Chen, and Ning Li. "Gaussian process regression for tool wear prediction". In: *Mechanical Systems and Signal Processing* 104 (2018), pp. 556–574. ISSN: 10961216. DOI: 10.1016/j.ymssp.2017.11.021.
- [KCMZ12] Mohamed Hedi Karray, Brigitte Chebel-Morello, and Noureddine Zerhouni. "A formal ontology for industrial maintenance". In: *Applied Ontology* 7.3 (2012), pp. 1–20. ISSN: 15705838. DOI: 10.3233/AO-2012-0112.
- [KCMZ15] Racha Khelif, Brigitte Chebel-Morello, and Noureddine Zerhouni. "Experience Based Approach for Li-ion Batteries RUL Prediction". In: *IFAC-PapersOnLine*. 2015. DOI: 10.1016/j.ifacol.2015.06.174.
- [Kee18] Maria Keet. *An Introduction to Ontology Engineering*. Texts in computing Vol 20, 2018, p. 289.
- [Kel+01] Kirby Keller et al. "A process and tool for determining the cost/benefit of prognostic applications". In: *AUTOTESTCON (Proceedings)*. 2001. DOI: 10.1109/autest.2001.949432.
- [KH07] Ranganath Kothamasu and Samuel H. Huang. "Adaptive Mamdani fuzzy model for condition-based maintenance". In: *Fuzzy Sets and Systems* 158.24 (2007), pp. 2715–2733. ISSN: 01650114. DOI: 10.1016/j.fss.2007.07.004.
- [Kin+18] Jakob Kinghorst et al. "Hidden Markov model-based predictive maintenance in semiconductor manufacturing: A genetic algorithm approach". In: *IEEE International Conference on Automation Science and Engineering*. 2018. ISBN: 9781509067800. DOI: 10.1109/COASE.2017.8256274.
- [KKL12] Kiyoshi Kobayashi, Kiyoyuki Kaito, and Nam Lethanh. "A statistical deterioration forecasting method using hidden Markov model for infrastructure management". In: *Transportation Research Part B: Methodological* 46.4 (2012), pp. 544–561. ISSN: 01912615. DOI: 10.1016/j.trb.2011.11.008.
- [Kol93] Janet Kolodner. *Case based reasoning*. Morgan Kaufmann, 1993, p. 612.
- [Kol94] Janet L. Kolodner. "Understanding creativity: A case-based approach". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. 1994. ISBN: 9783540583301. DOI: 10.1007/3-540-58330-0_73.
- [Kot+06] Ranganath Kothamasu et al. "System health monitoring and prognostics -a review of current paradigms and practices". In: *International Journal of Advanced Manufacturing Technology* 28.9 (2006), pp. 1012–1024. ISSN: 02683768. DOI: 10.1007/978-1-84882-472-0_14. arXiv: arXiv:1011.1669v3.
- [KRH02] Gregory J. Kacprzynski, Michael J. Roemer, and Andrew J. Hess. "Health management system design: Development, simulation and cost/benefit optimization". In: *IEEE Aerospace Conference Proceedings*. 2002. ISBN: 078037231X. DOI: 10.1109/AERO.2002.1036148.
- [KS12a] Erik E. Kostandyan and John D. Sorensen. "Physics of failure as a basis for solder elements reliability assessment in wind turbines". In: *Reliability Engineering and System Safety* 108 (2012), pp. 100–107. ISSN: 09518320. DOI: 10.1016/j.res.2012.06.020.
- [KS12b] Prakash Kumar and R. K. Srivastava. "An expert system for predictive maintenance of mining excavators and its various forms in open cast mining". In: *2012 1st International Conference on Recent Advances in Information Technology, RAIT-2012*. 2012. ISBN: 9781457706974. DOI: 10.1109/RAIT.2012.6194607.

- [LDS18] Xiang Li, Qian Ding, and Jian Qiao Sun. “Remaining useful life estimation in prognostics using deep convolution neural networks”. In: *Reliability Engineering and System Safety* 172 (2018), pp. 1–11. ISSN: 09518320. DOI: 10.1016/j.ress.2017.11.021.
- [Le +13] Khanh Le Son et al. “Remaining useful life estimation based on stochastic deterioration models: A comparative study”. In: *Reliability Engineering and System Safety* 112 (2013), pp. 165–175. ISSN: 09518320. DOI: 10.1016/j.ress.2012.11.022. arXiv: 1011.1669.
- [Leg+17] Václav Legát et al. “Preventive maintenance models – Higher operational reliability. Maintenance and Reliability”. In: *Eksploracja i Niezawodność* 19.1 (2017), pp. 134–141. ISSN: 15072711. DOI: 10.17531/ein.2017.1.19.
- [Lei+18] Yaguo Lei et al. “Machinery health prognostics: A systematic review from data acquisition to RUL prediction”. In: *Mechanical Systems and Signal Processing* 104 (2018), pp. 799–834. ISSN: 10961216. DOI: 10.1016/j.ymssp.2017.11.016.
- [Lev66] Vladimir Levenshtein. “Binary codes capable of correcting deletions, insertions, and reversals”. In: *Soviet Physics Doklady* 10.8 (1966), pp. 707–710.
- [LFS18] Sai Li, Huajing Fang, and Bing Shi. “Multi-Step-Ahead Prediction with Long Short Term Memory Networks and Support Vector Regression”. In: *Chinese Control Conference, (CCC)*. 2018. ISBN: 9789881563941. DOI: 10.23919/ChiCC.2018.8484066.
- [LGS13] Kaibo Liu, Nagi Z. Gebraeel, and Jianjun Shi. “A Data-level fusion model for developing composite health indices for degradation modeling and prognostic analysis”. In: *IEEE Transactions on Automation Science and Engineering* 10.3 (2013), pp. 652–664. ISSN: 15455955. DOI: 10.1109/TASE.2013.2250282.
- [LHS20] Daniel P Lupp, Melinda Hodkiewicz, and Martin G Skjæveland. “Template libraries for industrial asset maintenance: A methodology for scalable and maintainable ontologies”. In: *CEUR Workshop Proceedings*. Vol. 2757. 2020, pp. 49–64.
- [LHZ18] Huan Luo, Miaohua Huang, and Zhou Zhou. “Integration of Multi-Gaussian fitting and LSTM neural networks for health monitoring of an automotive suspension component”. In: *Journal of Sound and Vibration* 428 (2018), pp. 87–103. ISSN: 10958568. DOI: 10.1016/j.jsv.2018.05.007.
- [Li+12] Sha Li et al. “Health condition-based maintenance decision intelligent reasoning method”. In: *Proceedings of 2012 International Conference on Quality, Reliability, Risk, Maintenance, and Safety Engineering, ICQR2MSE 2012*. 2012. ISBN: 9781467307888. DOI: 10.1109/ICQR2MSE.2012.6246263.
- [Li+16] Qi Li et al. “Remaining useful life estimation for deteriorating systems with time-varying operational conditions and condition-specific failure zones”. In: *Chinese Journal of Aeronautics* 29.3 (2016), pp. 662–674. ISSN: 10009361. DOI: 10.1016/j.cja.2016.04.007.
- [Li+17] Gaoyang Li et al. “Failure Prognosis of High Voltage Circuit Breakers with Temporal Latent Dirichlet Allocation”. In: *Energies* 10.11 (2017), p. 1913. ISSN: 1996-1073. DOI: 10.3390/en10111913.
- [Li+18] Jianfeng Li et al. “Three-dimensional Simulation and Prediction of Solenoid Valve Failure Mechanism Based on Finite Element Model”. In: *IOP Conference Series: Earth and Environmental Science*. 2018. DOI: 10.1088/1755-1315/108/2/022035.
- [Liu+09] Hao Liu et al. “A review on fault prognostics in integrated health management”. In: *ICEMI 2009 - Proceedings of 9th International Conference on Electronic Measurement and Instruments*. 2009. ISBN: 9781424438624. DOI: 10.1109/ICEMI.2009.5274082.
- [Liu+18] Datong Liu et al. “An on-line state of health estimation of lithium-ion battery using unscented particle filter”. In: *IEEE Access* (2018). ISSN: 21693536. DOI: 10.1109/ACCESS.2018.2854224.

- [LK18] Changyong Lee and Daeil Kwon. “A similarity based prognostics approach for real time health management of electronics using impedance analysis and SVM regression”. In: *Microelectronics Reliability* 83 (2018), pp. 77–83. ISSN: 00262714. DOI: 10.1016/j.microrel.2018.02.014.
- [LLS19] Yanting Li, Shujun Liu, and Lianjie Shu. “Wind turbine fault diagnosis based on Gaussian process classifiers applied to operational data”. In: *Renewable Energy* 134 (2019), pp. 357–366. ISSN: 18790682. DOI: 10.1016/j.renene.2018.10.088.
- [LMM18] Yazid Laib dit Leksir, Moufid Mansour, and Abdelkrim Moussaoui. “Localization of thermal anomalies in electrical equipment using Infrared Thermography and support vector machine”. In: *Infrared Physics and Technology* 89 (2018), pp. 120–128. ISSN: 13504495. DOI: 10.1016/j.infrared.2017.12.015.
- [LQ+19] Hu Li-Qiang et al. “Track circuit fault prediction method based on grey theory and expert systems”. In: *Journal of Visual Communication and Image Representation* 58 (2019), pp. 37–45.
- [LWX19] Yuqian Lu, Hongqiang Wang, and Xun Xu. “ManuService ontology: a product data model for service-oriented business interactions in a cloud manufacturing environment”. In: *Journal of Intelligent Manufacturing* 30 (2019), pp. 317–334. ISSN: 15728145. DOI: 10.1007/s10845-016-1250-x.
- [LYKS18] Kenisuomo C. Luwei, Akilu Yunusa-Kaltungo, and Yusuf A. Sha’aban. “Integrated Fault Detection Framework for Classifying Rotating Machine Faults Using Frequency Domain Data Fusion and Artificial Neural Networks”. In: *Machines* 6.59 (2018).
- [LYM16] Lei Lu, Jihong Yan, and Yue Meng. “Dynamic Genetic Algorithm-based Feature Selection Scheme for Machine Health Prognostics”. In: *Procedia CIRP* 56 (2016), pp. 316–320. ISSN: 22128271. DOI: 10.1016/j.procir.2016.10.026.
- [Mab+18] Mohammed M. Mabkhot et al. *Requirements of the smart factory system: A survey and perspective*. 2018. DOI: 10.3390/MACHINES6020023.
- [MAK14] Nadakatti Mahantesh, Parida Aditya, and Uday Kumar. “Integrated machine health monitoring: A knowledge based approach”. In: *International Journal of Systems Assurance Engineering and Management* 5 (2014), pp. 371–382. ISSN: 09764348. DOI: 10.1007/s13198-013-0178-1.
- [MASH19] Mohammed M. Mabkhot, Ali M. Al-Samhan, and Lotfi Hidri. “An ontology-enabled case-based reasoning decision support system for manufacturing process selection”. In: *Advances in Materials Science and Engineering* (2019). ISSN: 16878442. DOI: 10.1155/2019/2505183.
- [Mat+10] Aristeidis Matsokis et al. “An ontology-based model for providing Semantic Maintenance”. In: *IFAC Proceedings Volumes (IFAC-PapersOnline)*. 2010. ISBN: 9783902661784. DOI: 10.3182/20100701-2-pt-4012.00004.
- [MBZ95] Mary Lou Maher, M. Bala Balachandran, and Dong Mei Zhang. *Case-Based Reasoning in Design*. 1st Editio. Psychology Press, 1995.
- [McD+18] Darren McDonnell et al. “Predicting the unpredictable: Consideration of human and organisational factors in maintenance prognostics”. In: *Journal of Loss Prevention in the Process Industries* 54 (2018), pp. 131–145. ISSN: 09504230. DOI: 10.1016/j.jlp.2018.03.008. arXiv: 1612.08814.
- [MD97] Mary Lou Maher and Andrés Gómez De Silva Garza. “Case-based reasoning in design”. In: *IEEE Expert-Intelligent Systems and their Applications* 12.2 (1997), pp. 34–41. ISSN: 08859000. DOI: 10.1109/64.585102.

- [Men+15] Sandeep Menon et al. “Evaluating covariance in prognostic and system health management applications”. In: *Mechanical Systems and Signal Processing* 58-59 (2015), pp. 206–217. ISSN: 10961216. DOI: 10.1016/j.ymssp.2014.10.012.
- [MHMJV21] Hugo Muñoz-Hernández, Juan José Montero-Jiménez, and Rob Vingerhoeds. “Integrating ontologies and case-based reasoning for the development of knowledge-intensive systems”. In: *Proceedings of the 35th annual European Simulation and Modelling Conference*. 2021, pp. 29–36. ISBN: 978-9-492859-18-1.
- [MIM01] MIMOSA. *Open System Architecture for Condition-Based Maintenance (OSA-CBM)*. 2001.
- [MLK17] Josey Mathew, Ming Luo, and Chee Khiang Pang. “Regression kernel for prognostics with support vector machines”. In: *22nd IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2017.
- [MM90] Vidosav D. Majstorovic and Vladimir R. Milacic. “Expert Systems for Maintenance in the CIM Concept”. In: *Computers in Industry* 15 (1990), pp. 83–93.
- [MMZ16] Ahmed Mosallam, Kamal Medjaher, and Nouredine Zerhouni. “Data-driven prognostic method based on Bayesian approaches for direct remaining useful life prediction”. In: *Journal of Intelligent Manufacturing* 27.5 (2016), pp. 1037–1048. ISSN: 15728145. DOI: 10.1007/s10845-014-0933-4.
- [MNK13] Nader Meskin, Esmaeil Naderi, and Khashayar Khorasani. “A multiple model-based approach for fault diagnosis of jet engines”. In: *IEEE Transactions on Control Systems Technology* 21.1 (2013), pp. 254–262. ISSN: 10636536. DOI: 10.1109/TCST.2011.2177981.
- [MO+14] Gabriela Medina-Oliva et al. “Predictive diagnosis based on a fleet-wide ontology approach”. In: *Knowledge-Based Systems* 68 (2014), pp. 40–57. ISSN: 09507051. DOI: 10.1016/j.knosys.2013.12.020.
- [Mon+20] Juan José Montero Jimenez et al. “Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics”. In: *Journal of Manufacturing Systems* 56 (2020), pp. 539–557. ISSN: 02786125. DOI: 10.1016/j.jmsy.2020.07.008.
- [Mon+21] Juan José Montero Jiménez et al. “An Ontology Model for Maintenance Strategy Selection and Assessment”. In: *Journal of Intelligent Manufacturing*. (2021). DOI: 10.1007/s10845-021-01855-3.
- [Mou09] Marcelo Nascimento Moutinho. “Fuzzy diagnostic systems of rotating machineries, some Eletronorte’s applications”. In: *2009 15th International Conference on Intelligent System Applications to Power Systems, ISAP '09*. 2009. ISBN: 9781424450985. DOI: 10.1109/ISAP.2009.5352882.
- [Mou97] John Moubray. *RCM II: Reliability Centered Maintenance*. Industrial Press, 1997, p. 423. ISBN: 978-0831130787.
- [MS95] Salvatore T. March and Gerald F. Smith. “Design and natural science research on information technology”. In: *Decision Support Systems* 15 (1995), pp. 251–266. ISSN: 01679236. DOI: 10.1016/0167-9236(94)00041-2.
- [MU11] Kamran S. Moghaddam and John S. Usher. “Sensitivity analysis and comparison of algorithms in preventive maintenance and replacement scheduling optimization models”. In: *Computers and Industrial Engineering* 61.1 (2011), pp. 64–75. ISSN: 03608352. DOI: 10.1016/j.cie.2011.02.012.
- [Mus92] Mark A. Musen. “Dimensions of knowledge sharing and reuse”. In: *Computers and Biomedical Research* 25.5 (1992), pp. 435–467. ISSN: 00104809. DOI: 10.1016/0010-4809(92)90003-S.

- [MV18] Juan José Montero Jimenez and Rob Vingerhoeds. “Enhancing operational fault diagnosis by assessing multiple operational modes”. In: *Proceedings - International Conference in Modelling, Optimization and Simulation MOSIM 2018, 27th-29th June*. Toulouse, France: MOSIM2018, 2018, pp. 237–244.
- [MV19] Juan José Montero Jiménez and Rob Vingerhoeds. “A System Engineering Approach to Predictive Maintenance Systems: from needs and desires to logical architecture.” In: *5th IEEE Int. Symposium on Systems Engineering 2019*, Edinburgh, 2019. DOI: 10.1109/ISSE46696.2019.8984559.
- [MVG21] Juan José Montero Jiménez, Rob Vingerhoeds, and Bernard Grabot. “Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning”. In: *7th IEEE International Symposium on System Engineering*, 2021. DOI: 10.1109/ISSE51541.2021.9582535.
- [MXJ19] Seyed M.Mehdi Hassani.N, Jin Xiaoning, and Ni Jun. “Physics-based Gaussian process for the health monitoring for a rolling bearing”. In: *Acta Astronautica* 154 (2019), pp. 133–139.
- [MZ18] Jianing Man and Qiang Zhou. “Prediction of hard failures with stochastic degradation signals using Wiener process and proportional hazards model”. In: *Computers and Industrial Engineering* 125 (2018), pp. 480–489. ISSN: 03608352. DOI: 10.1016/j.cie.2018.09.015.
- [NB17a] David Lira Nuñez and Milton Borsato. “An ontology-based model for prognostics and health management of machines”. In: *Journal of Industrial Information Integration* (2017). ISSN: 2452414X. DOI: 10.1016/j.jii.2017.02.006.
- [NB17b] David Lira Nuñez and Milton Borsato. “An ontology-based model for prognostics and health management of machines”. In: *Journal of Industrial Information Integration* 6 (2017), pp. 33–46. ISSN: 2452414X. DOI: 10.1016/j.jii.2017.02.006.
- [NB18] David Lira Nuñez and Milton Borsato. “OntoProg: An ontology-based model for implementing Prognostics Health Management in mechanical machines”. In: *Advanced Engineering Informatics* 38 (2018), pp. 746–759. ISSN: 14740346. DOI: 10.1016/j.aei.2018.10.006.
- [NM01] Natalya F. Noy and Deborah L. McGuinness. *Ontology Development 101: A Guide to Creating Your First Ontology*. Tech. rep. 2001. DOI: 10.1016/j.artmed.2004.01.014.
- [NO17] Vladislavs Nazaruks and Janis Osis. “A survey on domain knowledge representation with frames”. In: *ENASE 2017 - Proceedings of the 12th International Conference on Evaluation of Novel Approaches to Software Engineering*, 2017. ISBN: 9789897582509. DOI: 10.5220/0006388303460354.
- [Nom+18] Mohammed A. Noman et al. *Overview of predictive condition based maintenance research using bibliometric indicators*. 2018. DOI: 10.1016/j.jksues.2018.02.003.
- [NSG12] Fachri P. Nasution, Svein Sævik, and Janne K.Ø. Gjølsteen. “Fatigue analysis of copper conductor for offshore wind turbines by experimental and FE method”. In: *Energy Procedia* 24 (2012), pp. 271–280. ISSN: 18766102. DOI: 10.1016/j.egypro.2012.06.109.
- [Nyu+18] Ladislav Nyulász et al. “Fault Detection and Isolation of an Aircraft Turbojet Engine Using a Multi-Sensor Network and Multiple Model Approach”. In: *Acta Polytechnica Hungarica* 15.2 (2018), pp. 189–209.
- [Oko+14] Caxton Okoh et al. “Overview of Remaining Useful Life prediction techniques in Through-life Engineering Services”. In: *Procedia CIRP* 16 (2014), pp. 158–163. ISSN: 22128271. DOI: 10.1016/j.procir.2014.02.006.
- [One+18] Melis Onel et al. “Simultaneous Fault Detection and Identification in Continuous Processes via nonlinear Support Vector Machine based Feature Selection”. In: *Computer Aided Chemical Engineering* 44 (2018), pp. 2077–2082. ISSN: 15707946. DOI: 10.1016/B978-0-444-64241-7.50341-4.

- [ORM17] Criston. Okoh, Rajkumar Roy, and Jorn Mehnen. “Predictive Maintenance Modelling for Through-Life Engineering Services”. In: *Procedia CIRP*. 2017. ISBN: 22128271 (ISSN). DOI: 10.1016/j.procir.2016.09.033. arXiv: 0208024 [gr-qc].
- [PBG12] Ashok Prajapati, James Bechtel, and Subramaniam Ganesan. “Condition based maintenance: A survey”. In: *Journal of Quality in Maintenance Engineering* 18.4 (2012), pp. 384–400. ISSN: 13552511. DOI: 10.1108/13552511211281552. arXiv: 2014WR016527 [10.1002].
- [PD11] Paul Phillips and Dominic Diston. “A knowledge driven approach to aerospace condition monitoring”. In: *Knowledge-Based Systems* 24.6 (2011), pp. 915–927. ISSN: 09507051. DOI: 10.1016/j.knosys.2011.04.008.
- [PDZ10] Ying Peng, Ming Dong, and Ming Jian Zuo. “Current status of machine prognostics in condition-based maintenance: A review”. In: *International Journal of Advanced Manufacturing Technology* 50.1-4 (2010), pp. 297–313. ISSN: 02683768. DOI: 10.1007/s00170-009-2482-0. arXiv: 0208024 [gr-qc].
- [Pey+09] Flavien Peysson et al. “Expert knowledge impact on damage trajectory analysis based prognostics”. In: *IFAC Proceedings Volumes (IFAC-PapersOnline)*. 2009. ISBN: 9783902661463. DOI: 10.3182/20090630-4-ES-2003.0231.
- [PH08] Jim Prentzas and Ioannis Hatzilygeroudis. “Combinations of case-based reasoning with other intelligent methods”. In: *CEUR Workshop Proceedings*. 2008. DOI: 10.3233/his-2009-0096.
- [Pla95] Enric Plaza. “Cases as terms: A feature term approach to the structured representation of cases”. In: *First International Conference, ICCBR-95*. Vol. 1. Springer Verlag, 1995, pp. 265–276. DOI: 10.1007/3-540-60598-3_24.
- [PMK13] Bahareh Pourbabae, Nader Meskin, and Khashayar Khorasani. “Multiple-Model Based Sensor Fault Diagnosis Using Hybrid Kalman Filter Approach for Nonlinear Gas Turbine Engines”. In: *2013 American Control Conference (Acc)*. 2013. ISBN: 9781479901760. DOI: 10.1109/TCST.2015.2480003.
- [Pro05] Philip E. Protter. *Stochastic Integration and Differential Equations, Second Edition*. Ed. by B Rozovski and M Yor. Springer, 2005.
- [PS04] Sankar Pal and Simon Shiu. *Foundations of soft case-based reasoning*. Wiley, 2004, p. 299.
- [PSD14] Panče Panov, Larisa Soldatova, and Sašo Džeroski. “Ontology of core data mining entities”. In: *Data Mining and Knowledge Discovery* 28 (2014), pp. 1222–1265. ISSN: 13845810. DOI: 10.1007/s10618-014-0363-0.
- [QA+17] Santiago Quintana-Amate et al. “A new knowledge sourcing framework for knowledge-based engineering: An aerospace industry case study”. In: *Computers and Industrial Engineering* 104 (2017), pp. 35–50. ISSN: 03608352. DOI: 10.1016/j.cie.2016.12.013.
- [Qin+16] Feiwei Qin et al. “An ontology-based semantic retrieval approach for heterogeneous 3D CAD models”. In: *Advanced Engineering Informatics* 30 (2016), pp. 751–768. ISSN: 14740346. DOI: 10.1016/j.aei.2016.10.001.
- [Qin+18] Yuchu Qin et al. “Towards an ontology-supported case-based reasoning approach for computer-aided tolerance specification”. In: *Knowledge-Based Systems* 141 (2018), pp. 129–147. ISSN: 09507051. DOI: 10.1016/j.knosys.2017.11.013.
- [RA19] Fabrício Henrique Rodrigues and Mara Abel. “What to consider about events: A survey on the ontology of occurrents”. In: *Applied Ontology* 14.4 (2019), pp. 343–378. ISSN: 18758533. DOI: 10.3233/AO-190217.
- [Ram14] Emmanuel Ramasso. “Investigating computational geometry for failure prognostics”. In: *Int. Journal on Prognostics and Health Management* 5.005 (2014), p. 18. ISSN: 21532648.

- [Ram15] Luis Ramos. “Semantic Web for manufacturing, trends and open issues: Toward a state of the art”. In: *Computers and Industrial Engineering* 90 (2015), pp. 444–460. ISSN: 03608352. DOI: 10.1016/j.cie.2015.10.013.
- [Ras03] Carl Edward Rasmussen. “Gaussian Processes in Machine Learning”. In: *Advanced Lectures on Machine Learning*. Ed. by Olivier Bousquet, Ulrike von Luxburg, and Gunnar Rätsch. Springer, 2003, pp. 63–71.
- [RC15] Joe Raad and Christophe Cruz. “A survey on ontology evaluation methods”. In: *IC3K 2015 - Proceedings of the 7th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management*. 2015. ISBN: 9789897581588. DOI: 10.5220/0005591001790186.
- [Red12] Timothy Redmond. *SPARQL Query tab for Protégé*. 2012. URL: https://protegewiki.stanford.edu/wiki/SPARQL\{_\}Query (visited on 10/22/2020).
- [RFG14] Paula Potes Ruiz, Bernard Kamsu Foguem, and Bernard Grabot. “Generating knowledge in maintenance from Experience Feedback”. In: *Knowledge-Based Systems* 68 (2014), pp. 4–20. ISSN: 09507051. DOI: 10.1016/j.knosys.2014.02.002.
- [RG10] Emmanuel Ramasso and Rafael Gouriveau. “Prognostics in switching systems: Evidential Markovian classification of real-time neuro-fuzzy predictions”. In: *2010 Prognostics and System Health Management Conference, PHM '10*. 2010. ISBN: 9781424447565. DOI: 10.1109/PHM.2010.5413442.
- [RN12] Stuart Russel and Peter Norvig. *Artificial intelligence—a modern approach 3rd Edition*. 2012. ISBN: 9780136042594. DOI: 10.1017/S0269888900007724. arXiv: 9809069v1 [arXiv:gr-qc].
- [Rom+14] Juan Camilo Romero Bejarano et al. “Case-based reasoning and system design: An integrated approach based on ontology and preference modeling”. In: *Artificial Intelligence for Engineering Design, Analysis and Manufacturing: AIEDAM* 28 (2014), pp. 49–69. ISSN: 14691760. DOI: 10.1017/S0890060413000498.
- [Roq18] Pascal Roques. *Systems Architecture Modeling with the Arcadia Method 1st Edition*. ISTE Press, 2018. ISBN: 9781785481680.
- [RS89] Christopher K Riesbeck and Roger C Schank. *Inside case-based reasoning*. 1989. ISBN: 0-89859-767-6 (Hardcover).
- [Rud20a] Ron Rudnicki. *An Overview of the Common Core Ontologies 1.3*. Buffalo, NY, 2020. URL: https://www.nist.gov/system/files/documents/2019/05/30/nist-ai-rfi-cubrc\{_\}inc\{_\}004.pdf.
- [Rud20b] Ron Rudnicki. *Common core ontologies*. 2020. URL: <https://github.com/CommonCoreOntology/CommonCoreOntologies> (visited on 07/22/2020).
- [RW06] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian process for machine learning*. the MIT Press, 2006.
- [SA08] Abdel-Badeeh M Salem and Marco Alfonse. “Ontology versus semantic networks for medical knowledge representation”. In: *Proceedings of the 12th WSEAS international Conference on COMPUTERS*. 2008. ISBN: 9789606766855.
- [SA17] Christina M. Steiner and Dietrich Albert. “Validating domain ontologies: A methodology exemplified for concept maps”. In: *Cogent Education* 4.1 (2017). ISSN: 2331186X. DOI: 10.1080/2331186X.2016.1263006.
- [Sán+12] David Sánchez et al. “Ontology-based semantic similarity: A new feature-based approach”. In: *Expert Systems with Applications* 39 (2012), pp. 7718–7728. ISSN: 09574174. DOI: 10.1016/j.eswa.2012.01.082.

- [Sax+08] Abhinav Saxena et al. “Damage propagation modeling for aircraft engine run-to-failure simulation”. In: *2008 International Conference on Prognostics and Health Management, PHM 2008*. 2008. ISBN: 9781424419357. DOI: 10.1109/PHM.2008.4711414.
- [SC15] Barry Smith and Werner Ceusters. “Aboutness: Towards foundations for the information artifact ontology”. In: *CEUR Workshop Proceedings*. 2015.
- [Sch+20] Sebastien Schwartz et al. “A fault mode identification methodology based on self-organizing map”. In: *Neural Computing and Applications* 32 (2020), pp. 13405–13423.
- [SHM11] Joanna Z. Sikorska, Melinda Hodkiewicz, and Lin Ma. “Prognostic modelling options for remaining useful life estimation by industry”. In: *Mechanical Systems and Signal Processing* 25.5 (2011), pp. 1803–1836. ISSN: 08883270. DOI: 10.1016/j.ymssp.2010.11.018.
- [Shu+17] Jin Shuangshuang et al. “The Remaining Life Prediction of the Fan Bearing Based on Genetic Algorithm and Multi-parameter Support Vector Machine”. In: *5th International Conference on Mechanical, Automotive and Materials Engineering*. 2017.
- [Si+11] Xiao Sheng Si et al. “Remaining useful life estimation - A review on the statistical data driven approaches”. In: *European Journal of Operational Research* 213.1 (2011), pp. 1–14. ISSN: 03772217. DOI: 10.1016/j.ejor.2010.11.018.
- [SKY19] Emilio M. Sanfilippo, Yoshinobu Kitamura, and Robert I.M. Young. “Formal ontologies in manufacturing”. In: *Applied Ontology* 14.2 (2019), pp. 119–125. ISSN: 18758533. DOI: 10.3233/AO-190209.
- [SMS17] Karanvir Singh, Hasmat Malik, and Rajneesh Sharma. “Condition monitoring of wind turbine gearbox using electrical signatures”. In: *2017 International Conference on Microelectronic Devices, Circuits and Systems, ICMDCS 2017*, 2017. ISBN: 9781538617168. DOI: 10.1109/ICMDCS.2017.8211718.
- [SRC10] Abhinav Saxena, Indranil Roychoudhury, and Jose R Celaya. “Requirements Specifications for Prognostics : An Overview”. In: *Proceedings of AIAA Infotech@Aerospace 2010*. 2010. ISBN: 9781600867439 (ISBN). DOI: 10.2514/6.2010-3398.
- [Sta20] Stanford University. *Protégé website*. 2020. URL: <https://protege.stanford.edu/> (visited on 07/22/2020).
- [Sul+10] Greg P. Sullivan et al. *Operations & Maintenance Best Practices: A Guide to Achieving Operational Efficiency*. U. S. Department of Energy, Federal energy management program, 2010. ISBN: 18773373463. DOI: 10.2172/1034595.
- [SW15] Bernard Schmidt and Lihui Wang. “Predictive Maintenance: Literature review and future trends”. In: *Conference: Proceedings of the 25th International Conference on Flexible Automation and Intelligent Manufacturing* 1 (2015), pp. 232–239.
- [SW18] Nazmus Sakib and Thorten Wuest. “Challenges and Opportunities of Condition-based Predictive Maintenance: a Review”. In: *6th CIRP Global Web Conference: “Envisaging the future manufacturing, design, technologies, and systems in innovation era”*. Elsevier B.V., 2018, pp. 267–272.
- [Tal+19] Asma Talhi et al. “Ontology for cloud manufacturing based Product Lifecycle Management”. In: *Journal of Intelligent Manufacturing* 30 (2019), pp. 2171–2192. ISSN: 15728145. DOI: 10.1007/s10845-017-1376-5.
- [TL14] Tiedo Tinga and Richard Loendersloot. “Aligning PHM, SHM and CBM by understanding the physical system failure behaviour”. In: *Proceedings of the European Conference of the Prognostics and Health Management Society*. 2014. ISBN: 978-1-936263-16-5.

- [TSY18] Diyin Tang, Wubin Sheng, and Jinsong Yu. “Dynamic condition-based maintenance policy for degrading systems described by a random-coefficient autoregressive model: A comparative study”. In: *Eksploracja i Niezawodność* 20.4 (2018), pp. 590–601. ISSN: 15072711. DOI: 10.17531/ein.2018.4.10.
- [Vac+07] George Vachtsevanos et al. *Intelligent Fault Diagnosis and Prognosis for Engineering Systems*. 2007. ISBN: 047172999X. DOI: 10.1002/9780470117842.
- [VBD17] Kim Verbert, Robert Babuška, and Bart De Schutter. “Bayesian and Dempster–Shafer reasoning for knowledge-based fault diagnosis—A comparative study”. In: *Engineering Applications of Artificial Intelligence* 60 (2017), pp. 136–150. ISSN: 09521976. DOI: 10.1016/j.engappai.2017.01.011.
- [VD18] Wim J.C. Verhagen and Lenjaert W.M. De Boer. *Predictive maintenance for aircraft components using proportional hazard models*. 2018. DOI: 10.1016/j.jii.2018.04.004.
- [VDB15] Kim Verbert, Bart De Schutter, and Robert Babuška. “Reasoning under uncertainty for knowledge-based fault diagnosis: A comparative study”. In: *IFAC-PapersOnLine* 48 (2015), pp. 422–427. ISSN: 24058963. DOI: 10.1016/j.ifacol.2015.09.563.
- [Vep91] Rajan Vepa. “Introduction to fuzzy logic and fuzzy sets”. In: *Application of artificial intelligence in process control*. (L. Boullart, A. Krijgsman and R.A. Vingerhoeds; editors). 1991, pp. 146–163.
- [Vin+95] Rob A. Vingerhoeds et al. “Enhancing off-line and on-line condition monitoring and fault diagnosis”. In: *Control Engineering Practice* 3.11 (1995), pp. 1515–1528. ISSN: 09670661. DOI: 10.1016/0967-0661(95)00162-N.
- [VM18] David Vališ and Dariusz Mazurkiewicz. “Application of selected Levy processes for degradation modelling of long range mine belt using real-time data”. In: *Archives of Civil and Mechanical Engineering* 18.4 (2018), pp. 1430–1440. ISSN: 16449665. DOI: 10.1016/j.acme.2018.05.006.
- [Von+18] Alexander Von Birgelen et al. “Self-Organizing Maps for Anomaly Localization and Predictive Maintenance in Cyber-Physical Production Systems”. In: *Procedia CIRP* 72 (2018), pp. 480–485. ISSN: 22128271. DOI: 10.1016/j.procir.2018.03.150.
- [VWD14] Gregory W. Vogl, Brian Weiss, and Alkan Donmez. “Standards for Prognostics and Health Management (PHM) Techniques within Manufacturing Operations”. In: *Annual Conference of the Prognostics and Health Management Society*. 2014. ISBN: 9781936263172.
- [Wan+08] Tianyi Wang et al. “A similarity-based prognostics approach for remaining useful life estimation of engineered systems”. In: *2008 International Conference on Prognostics and Health Management, PHM 2008*. 2008. ISBN: 9781424419357. DOI: 10.1109/PHM.2008.4711421.
- [Wan+14] Yuanhang Wang et al. “A corrective maintenance scheme for engineering equipment”. In: *Engineering Failure Analysis* 36 (2014), pp. 269–283. ISSN: 13506307. DOI: 10.1016/j.engfailanal.2013.10.006.
- [Wan+18] Anping Wan et al. “Prognostics of gas turbine: A condition-based maintenance approach based on multi-environmental time similarity”. In: *Mechanical Systems and Signal Processing* 109 (2018), pp. 150–165. ISSN: 10961216. DOI: 10.1016/j.ymssp.2018.02.027.
- [Wel92] Gordon Wells. “An introduction to neural networks”. In: *Application of artificial intelligence in process control*. (L. Boullart, A. Krijgsman and R.A. Vingerhoeds; editors). 1992, pp. 164–200.
- [WHF18] Zhao Qiang Wang, Chang Hua Hu, and Hong Dong Fan. “Real-Time Remaining Useful Life Prediction for a Nonlinear Degrading System in Service: Application to Bearing Data”. In: *IEEE/ASME Transactions on Mechatronics* 23.1 (2018), pp. 211–222. ISSN: 10834435. DOI: 10.1109/TMECH.2017.2666199.

- [Wor12] World Wide Web Consortium. *OWL 2 Web Ontology Language*. 2012. URL: <http://www.w3.org/TR/2012/REC-owl2-overview-20121211/> (visited on 02/01/2020).
- [WTM17] Dong Wang, Kwok Leung Tsui, and Qiang Miao. "Prognostics and Health Management: A Review of Vibration Based Bearing and Gear Health Indicators". In: *IEEE Access* 6 (2017), pp. 665–676. ISSN: 21693536. DOI: 10.1109/ACCESS.2017.2774261.
- [Wu+18] Zhenyu Wu et al. "K-PdM: KPI-Oriented Machinery Deterioration Estimation Framework for Predictive Maintenance Using Cluster-Based Hidden Markov Model". In: *IEEE Access* Vol 6. DOI: 10.1109/ACCESS.2018.2859922 (2018), pp. 41676–41687. ISSN: 21693536. DOI: 10.1109/ACCESS.2018.2859922.
- [Yan+04] Bo Suk Yang et al. "Case-based reasoning system with Petri nets for induction motor fault diagnosis". In: *Expert Systems with Applications* 27.2 (2004), pp. 301–311. ISSN: 09574174. DOI: 10.1016/j.eswa.2004.02.004.
- [YN18] Ali Yahyatabar and Amir Abbas Najafi. "Condition based maintenance policy for series-parallel systems through Proportional Hazards Model: A multi-stage stochastic programming approach". In: *Computers and Industrial Engineering* 126 (2018), pp. 30–46. ISSN: 03608352. DOI: 10.1016/j.cie.2018.09.014.
- [YSJ17] Kam Chuen Yung, Bo Sun, and Xiaopeng Jiang. "Prognostics-based qualification of high-power white LEDs using Lévy process approach". In: *Mechanical Systems and Signal Processing* 82 (2017), pp. 206–216. ISSN: 10961216. DOI: 10.1016/j.ymssp.2016.05.019.
- [ZBD16] Deyi Zhang, Andrew D. Bailey, and Dragan Djurdjanovic. "Bayesian Identification of Hidden Markov Models and Their Use for Condition-Based Monitoring". In: *IEEE Transactions on Reliability* (2016). ISSN: 00189529. DOI: 10.1109/TR.2016.2570561.
- [Zer+17] Noureddine Zerhouni et al. "Prognostics and Health Management for Maintenance Practitioners-Review, Implementation and Tools Evaluation". In: *Article in International Journal of Prognostics and Health Management* 8.60 (2017), p. 31. ISSN: 22129685. DOI: 10.1016/j.euprot.2015.07.015.
- [Zha+17] Rui Zhao et al. "Machine Health Monitoring Using Local Feature-based Gated Recurrent Unit Networks". In: *IEEE Transactions on Industrial Electronics* 65.2 (2017), pp. 1539–1548. ISSN: 0278-0046. DOI: 10.1109/TIE.2017.2733438.
- [Zha+18a] Zhengxin Zhang et al. "Degradation data analysis and remaining useful life estimation: A review on Wiener-process-based methods". In: *European Journal of Operational Research* 271.3 (2018), pp. 775–796. ISSN: 03772217. DOI: 10.1016/j.ejor.2018.02.033.
- [Zha+18b] Shuai Zhao et al. "Evaluation of Reliability Function and Mean Residual Life for Degrading Systems Subject to Condition Monitoring and Random Failure". In: *IEEE Transactions on Reliability* 67.1 (2018), pp. 13–25. ISSN: 00189529. DOI: 10.1109/TR.2017.2779322.
- [Zho+10] Zhi Jie Zhou et al. "A model for real-time failure prognosis based on hidden Markov model and belief rule base". In: *European Journal of Operational Research* 207.1 (2010), pp. 269–283. ISSN: 03772217. DOI: 10.1016/j.ejor.2010.03.032.
- [Zho+14] Zhi Jie Zhou et al. "A model for online failure prognosis subject to two failure modes based on belief rule base and semi-quantitative information". In: *Knowledge-Based Systems* (2014). ISSN: 09507051. DOI: 10.1016/j.knosys.2014.06.026.
- [ZM07a] Yimin Zhan and Chris K. Mechefske. "Robust detection of gearbox deterioration using compromised autoregressive modeling and Kolmogorov-Smirnov test statistic-Part I: Compromised autoregressive modeling with the aid of hypothesis tests and simulation analysis". In: *Mechanical Systems and Signal Processing* 21.5 (2007), pp. 1953–1982. ISSN: 08883270. DOI: 10.1016/j.ymssp.2006.11.005.

- [ZM07b] Yimin Zhan and Chris K. Mechefske. “Robust detection of gearbox deterioration using compromised autoregressive modeling and Kolmogorov-Smirnov test statistic. Part II: Experiment and application”. In: *Mechanical Systems and Signal Processing* 21.5 (2007), pp. 1983–2011. ISSN: 08883270. DOI: 10.1016/j.ymssp.2006.11.006.
- [ZYX17] Qiang Zhou, Ping Yan, and Yang Xin. “Research on a knowledge modelling methodology for fault diagnosis of machine tools based on formal semantics”. In: *Advanced Engineering Informatics* 32 (2017), pp. 92–112. ISSN: 14740346. DOI: 10.1016/j.aei.2017.01.002.
- [ZYZ15] Anmei Zhou, Dejie Yu, and Wenyi Zhang. “A research on intelligent fault diagnosis of wind turbines based on ontology and FMECA”. In: *Advanced Engineering Informatics* 29.1 (2015), pp. 115–125. ISSN: 14740346. DOI: 10.1016/j.aei.2014.10.001.

A fault mode identification methodology based on self-organizing maps

The content in this appendix corresponds to a published work in the Neural Computing & Applications journal, under the license number 5139260695204 © Springer 2020. Reprinted, with permission, from Sebastien Schwartz, Juan José Montero Jimenez, Michel Salaun, Rob Vingerhoeds. “A fault mode identification methodology based on self-organizing map”. Neural Computing & Applications Vol.32 (2020), pp. 13405-13423 [Sch+20].

Intentionally left blank



A fault mode identification methodology based on self-organizing map

Sébastien Schwartz^{1,2} · Juan José Montero Jimenez^{2,3} · Michel Salaün² · Rob Vingerhoeds²

Received: 14 January 2019 / Accepted: 18 December 2019 / Published online: 1 January 2020
© Springer-Verlag London Ltd., part of Springer Nature 2020

Abstract

One of the main goals of predictive maintenance is to be able to trigger the right maintenance actions at the right moment in time building upon the monitoring of the health status of the concerned systems and their components. As such, it allows identifying incipient faults and forecasting the moment of failure at the earliest stage. Many different data-driven methods are used in such approaches (Naderi and Khorasani in 2017 IEEE 30th Canadian conference on electrical and computer engineering (CCECE), Windsor, ON, IEEE, pp 1–6, 2017. <https://doi.org/10.1109/ccece.2017.7946715>; Sarkar et al. in J Eng Gas Turbines Power 1338(8):081602, 2011. <https://doi.org/10.1115/1.4002877>; Svärd et al. in Mech Syst Signal Process 45(1):170–192, 2014. <https://doi.org/10.1016/j.ymssp.2013.11.002>; Pourbabaee et al. Mech Syst Signal Process 76–77:136–156, 2016. <https://doi.org/10.1016/j.ymssp.2016.02.023>). This work uses the self-organizing maps (SOMs) or Kohonen map, thanks to its ability to emphasize underlying behavior such as fault modes. An automatic fault mode detection is presented based on a SOM network and the kernel density estimation with as less as possible prior knowledge. The different SOM development steps are presented and the suitable solutions proposed to structure the approach are accompanied by mathematical methods. The generated maps are then used with kernel density analysis to isolate fault modes on them. Finally, a methodology is presented to identify the different fault modes. The work is illustrated with an aircraft jet engines case study.

Keywords Diagnostic · Fault identification · Predictive maintenance · Self-organizing map

1 Introduction

Maintenance departments are confronted with three types of maintenance: corrective maintenance (i.e., correcting systems that break down or have a deteriorated functional behavior), preventive maintenance (i.e., maintenance actions at regular intervals, to avoid break down or deterioration) and predictive maintenance (i.e., performing specific maintenance actions based on indications derived

from fine analysis on data, crew reports, etc.). Predictive maintenance has seen a huge rise over the last years, essentially due to the application of neural networks to identify incipient faults and to forecast the moment of failure through the diagnostic phase. Depending on the monitored data, there are different types of classification for diagnostic system (Fig. 1). Machine fault diagnostic approaches are grouped into model-based [1–3] and data-driven [4–7] techniques.

In this paper, a hybrid approach for a “process history-based” diagnosis with quantitative data through the combination of a neural network (NN) with a probability density function (PDF) is scoped. In particular, an unsupervised neural network (UNN) type, the self-organizing map (SOM), or Kohonen map, is used. This NN has already shown its effectiveness in the past [9] but requires substantial knowledge on the neural network type itself, making it complex to comprehend and frequently requiring “manual” rework. Therefore, the presented novel approach

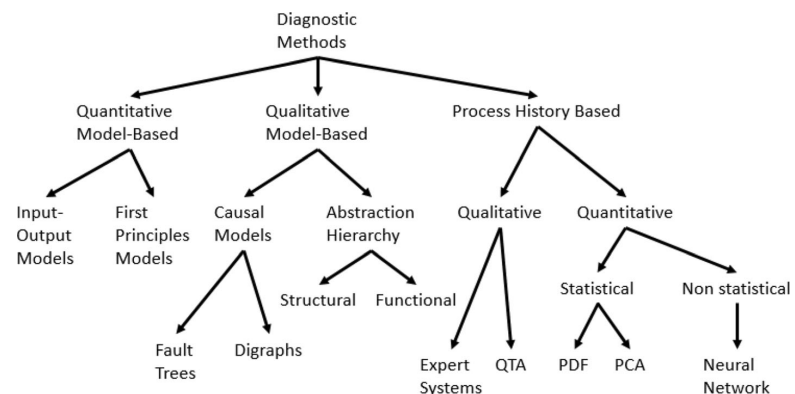
✉ Sébastien Schwartz
sebastien.schwartz@isae-supaero.fr

¹ SOGETI High Tech, R&D Dpt., Aeropark, 3 Chemin de Laporte, 31100 Toulouse, France

² ISAE-SUPAERO, Université de Toulouse, 10 Avenue Edouard Belin, 31400 Toulouse, France

³ Tecnológico de Costa Rica Institute of Technology, Calle 15, Avenida 14., 1 km Sur de la Basílica de los Ángeles, Provincia de Cartago, Cartago 30101, Costa Rica

Fig. 1 Classification of diagnostic methods based on [8]. *QTA* qualitative trend analysis, *PDF* probability density function, *PCA* principal component analysis



aims at automating as much as possible for the development of the SOM. The first objective is to reduce to the minimum level of the interaction between the expert knowledge and the network itself. The complete process is reviewed, and, for each step in the process, mathematical methods are proposed. It leads to a more structured and automatable procedure with an intent to make the method more accessible and autonomous. The second objective is to manage automatically the output of the SOM with PDFs to identify fault modes.

The goal of this paper is to present a fully autonomous toolchain that identifies the fault modes from input sensors by reducing as much as possible the prior knowledge and the expert intervention.

The paper is organized as follows: In the second section, predictive maintenance is introduced. In the third section, the self-organizing map is presented, as well as the different possibilities to enhance the approach and a methodology to identify the fault modes. The fourth section exposes and applies the methodology on the case study of aircraft engines. The paper concludes with some general observations and indications for future work.

2 Predictive maintenance

Keeping a technical system in optimal operational conditions is key for a successful and an efficient use of the system. Interruptions of operations not only have a negative impact (e.g., delayed flights, internet connections not being available, etc.), but may also have worst consequences (e.g., image of the company impacted, people will tend to privilege other suppliers, loss of income, etc.). Maintaining a system in optimal state of operation also means having timely maintenance actions to ensure the intended functionality and to avoid potential failures to occur. A good combination of corrective, preventive and

predictive maintenance is required [10]. Whereas corrective maintenance is based on alarm handling, troubleshooting for corrective actions, etc., predictive maintenance relies on offline diagnostic task analyses such as recorded data, crew reports, maintenance logs and other data recordings on the actual health state of the system. Such information is then used by operation departments to assess the health state and derive necessary maintenance actions and their planning if necessary.

Condition monitoring is used to assess the current health state of the system at hand. It uses pattern recognition in time series of monitored data and classifies those patterns as known conditions. While this is used to be done by human experts [9], requiring great skill and experience from the expert, software tools have appeared to support engineers in such activities.

As predictive maintenance aims to define the best possible moment to trigger maintenance actions [9], it gained more and more attention over the last few years. One of the solutions discussed in the literature for early detection and classification of failures during the diagnostic phase is the self-organizing map (SOM) originally proposed by Kohonen [11]. This approach allows to detect degradation patterns and the nature of the problem and to derive the remaining useful life [9, 10, 12]. This network has been widely used on various application fields for its particular abilities [13–18].

Visualization helps humans to understand diagnostic tasks. According to [19], the visualization of the data helps to gain an understanding of an unknown dataset. For limited amounts of dimensions in data, humans can do it, but the perception is limited to three dimensions. High-dimensional data visualization with more than three features is therefore unreachable. To address this issue, several visualization techniques were developed such as principal component analysis (PCA) [20], self-organized maps (SOMs) [11] or Sammon mapping [21]. Those techniques

rely on dimension reduction to visualize on 2D or 3D plots. This dimension reduction therefore generates new knowledge to be labeled for the analysis. Labeling data using prior knowledge or human reasoning become complicated on high dimensions [22], which could induce errors. Approaches relying on unsupervised learning are interesting thanks to their abilities to deal with high-dimensional data without prior knowledge. Therefore, a SOM network answers to the requirements: visualize high-dimensional data on 2D maps and obtain knowledge generated from these maps.

The successful implementation of SOM for diagnostic tasks requires an in-depth analysis of data obtained from the system at hand. It involves the assessment of data interdependency, the analysis on how many different faults/failures can be identified in the data, the distinction of eventual operational modes (if necessary) and, finally, successful training and subsequent validation of the SOM. Such analysis requires a good knowledge on the application domain itself and the measured data, in addition to strong knowledge on SOMs. In the next section, the fault mode diagnosis approach is presented. Each step of the SOM neural network is revisited and analyzed to see “whether and how” improvements may be obtained to automatize the use of SOM neural networks and to reduce the need for prior knowledge. Then, probability density function on the neural network output is used to identify faults of the supervised system.

3 Fault mode diagnosis using self-organizing maps

3.1 Overview

As presented previously, fault diagnosis requires human intervention and prior knowledge. The proposed methodology (Fig. 2) attempts to perform an automatic fault mode diagnosis with as less as possible prior knowledge. The self-organizing map neural network is the core of the methodology as a tool to emphasize the input data through a map representation. The underlying information such as faults becomes more accessible. The approach has been structured into three phases.

The first step is “input data management”. Input data (raw monitored sensors) have to be formatted and used with the neural network. This sensor management involves the choice of useful variables to decrease the complexity and the size the neural network. In addition, the data are normalized to facilitate the network training.

The second step is “system map.” The formatted input data are presented to the neural network. For the training phase, the network generates maps representing the input

data. During the testing phase, the network outputs a localization on the previously trained map.

The last step is the “fault mode identification”. The localization on the map (output of the previous stage) enables the identification of the fault mode thanks to a mathematical procedure based on the probability density function.

In the following sections, each step (Fig. 2) will be described more in detail.

3.2 Input data management

As presented previously, a selection among raw monitored data is performed. This procedure is called feature selection (see [23–26]). It is used on structured data to select features that explain most of the system behaviors by eliminating inappropriate and redundant data [23]. This paper will only focus on time-series data. Some basic approaches provide good means to address the feature selection. For example, the variance is a good way to eliminate features with little or no evolution. The correlation coefficient is powerful to identify feature that have the same behavior. Visual analysis highlights features that have unusual trend.

For better analysis purpose, a normalization of the input data is performed. It provides a common scale for the features. Two main methods are used: rescaling and standardizing. The rescaling method is the simplest one and consists to scale data on the range $[0, 1]$ with the following formula:

$$\bar{x}_{ij} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}} \quad (1)$$

where x_{ij} and $\bar{x}_{ij}$ are, respectively, the original and normalized data values for a sample j of a feature i from the input dataset, with $i = 1 \dots p$, where p is the number of network input.

The standardizing method uses the following formula:

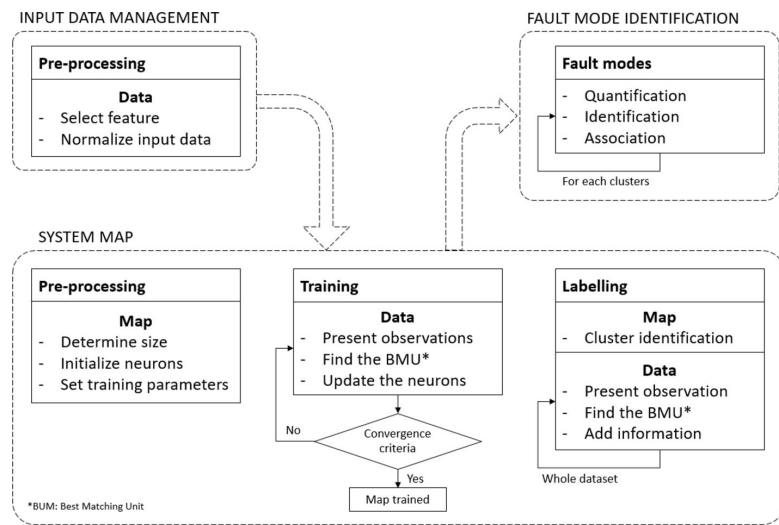
$$\bar{x}_{ij} = \frac{x_{ij} - \mu_i}{\sigma_i} \quad (2)$$

where μ_i and σ_i are, respectively, the mean and the standard deviation of a feature i . Even if this method provides a uniform scale, normalized inputs data do not belong to the same common scale. That is why the rescaling method will be used to have all input data on the same range $[0, 1]$.

3.3 System map

3.3.1 Overview

Self-organizing maps are neural networks using unsupervised learning inspired from human brain way [11]. They

Fig. 2 Fault mode diagnosis methodology

are suitable to produce a low-dimensional representation of the input space of training samples, called a map, to visualize high-dimensional data [27]. SOMs are therefore useful for dimensionality reduction and representation in which the similarity relations between input data are preserved [10]. Its competitive learning capability and the use of the neighborhood function preserve the topological properties of the inputs. The competitive approach aims to put output neurons in competition with each other to be activated, and the winning neuron is the only one that can be activated. As most artificial neural networks, SOMs are developed in two subsequent phases: a training phase and a testing phase. Thanks to the characteristics of this particular neural network, the testing phase can also be used as a labeling part, which is the assignment of information to specific clusters on the map, such as specific faults, or system operational conditions. The goal of the training phase is to teach the algorithm with the dataset in such a way that similar data features are clustered on specific topological regions on the map [28]. Then, the generated map provides clusters, and a health index (HI) is estimated for each node, depicting the degradation status of the studied system.

The SOM neural network building is divided into three stages: preprocessing, training and labeling. A lot of manual work done by experts is needed to perform these tasks.

3.3.2 Preprocessing

SOM topologies can be in one, two or even three dimensions [29–33]. The neurons are localized at lattice nodes. The original SOM [11] is a 2D hexagonal map. Then,

successively, 1D lines, 2D rectangular grids or more complex structures, such as star lattices [34] (Fig. 3), have been created. For our case study, a 2D square lattice is used to visualize it as picture.

A square lattice has $n \times n$ neurons of m weights. The number of weights per group (i.e., m) corresponds to the number of inputs to the network. According to [35], a size of the map can be determined by calculating the number of neurons from the number of observations in the dataset such as:

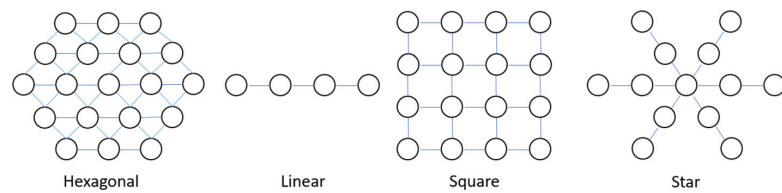
$$M \cong 5\sqrt{N} \quad (3)$$

where M is the number of neurons, and N is the number of observations. A square lattice will have $n = \sqrt{M}$. For example, with about 10,000 observations, Eq. (3) leads to a size $M \cong 500$. For a square lattice, the closest dimension would be a 23×23 matrix.

The next step is the map initialization. There are various ways to set initial weights, such as input vectors randomly selected [36], principal components of the input space [36], large hypercube [37] or random values. A uniform distribution in the range $[0, 1]$ with a probability density function of 1 is considered for usefulness to set neuron weight vectors $w_{ij} = (w_{ij1}, w_{ij2}, \dots, w_{ijm})$ with $i, j = 1 \dots n$.

3.3.3 Training phase

After the input data are processed and the map is initialized, the map is trained using preprocessed data. Training algorithms related to SOMs are various. Stochastic training as Algorithm 1 is one of the most classical algorithms [38]. Alternatives such as fast batch SOM [39] or growing self-

Fig. 3 Examples of lattice structures

organizing map (GSOM) [40] can be faster but are more complex to use. They all rely on the determination of the best matching unit (BMU), which is the smallest distance between the input vector and the weight vector of the map nodes.

The learning process is iterative, until a stopping criterion is met. Examples of criteria are an error estimation such as the quantization error [36] or the so-called “rule of thumb” where the number of steps must be at least 500 times the number of neurons in the map [38]. This last

Algorithm 1. Randomly input selection for self-organizing map

```

For  $t$  from 1 to Number_Of_Iteration do
  | Pick up a random input vector among the input dataset
  | For each node in the map do
  | | Calculate the distance between input vector and map node weight vector
  | | Track the node with the smallest distance (i.e. BMU)
  | End
  | Update the weight vector of the neighborhood of the BMU node and itself
End

```

The weight vector is updated at each iteration as follows:

$$\begin{cases} w_{ij}(t+1) = w_{ij}(t) + h_{kl,ij}(t)[x(t) - w_{ij}(t)] & \forall n_{ij} \in \mathbb{E}_{BMU} \\ w_{ij}(t+1) = w_{ij}(t) & \forall n_{ij} \notin \mathbb{E}_{BMU} \end{cases} \quad (4)$$

where w_{ij} is the vector weight, t is the t^{th} iteration, $h_{kl,ij}$ is the neighborhood function, $x(t)$ is the input observed, n_{ij} is the node on the map, ij are the node coordinates on a 2D lattice, kl are the BMU node coordinates on the same 2D lattice and $\mathbb{E}_{BMU}$ is the space of BMU neighborhood node and itself. This space is defined by the width of the neighborhood function, also called the BMU radius. A smooth Gaussian kernel is mostly used for the neighborhood function [36, 41]:

$$h_{kl,ij}(t) = \eta(t) \cdot e^{-\frac{w_{kl}(t) - w_{ij}(t)}{2\sigma^2(t)}} \quad (5)$$

where $\eta(t)$ and $\sigma(t)$ are, respectively, the learning rate and the width of the kernel, which are the decreasing functions of time [36, 38]. This function decreases through the time to improve the neighborhood identification.

The BMU node n_{kl} is defined by:

$$\left\{ n_{kl} | x(t) - w_{kl}^2 = \arg \min_{ij} x(t) - w_{ij}(t)^2 \right\} \quad (6)$$

criterion will be used in this paper.

The SOM training speed is linked to the map size, the number of inputs and the number of samplings. The number of weights could be large, which leads to a slow convergence due to the amount of weight updates involved in each iteration. Several mechanisms have been developed to address this problem, such as optimizing the width of the neighborhood function or learning rate function. According to [36, 41], the Gaussian kernel is a good candidate. The width of the neighborhood function $\sigma(t)$ is chosen as:

$$\sigma(t) = \sigma_0 \cdot e^{-t/\tau_1} \quad (7)$$

where σ_0 is an initial variance set to the map size divided by two [38], and τ_1 is a positive constant. The learning rate function $\eta(t)$ is chosen as:

$$\eta(t) = \eta_0 \cdot e^{-t/\tau_2} \quad (8)$$

where η_0 is an initial learning rate set to 0.9 [38], and τ_2 is a positive constant. The function is limited to a minimum set at 0.01 [38].

For convenience, τ_1 and τ_2 are equal and follow the relation:

$$\tau_i = t_{\max} / \ln \sigma_0 \quad \text{with } i = 1, 2 \quad (9)$$

where $t_{\max}$ is the maximal number of iterations. Those constants lead the exponential decay function radius to 1 when t reaches its maximum value, which is the maximal number of iterations [42].

With this proposed training, the network is able to adapt automatically to the presented input data. There is no need for an objective function as in supervised learning. The main disadvantage is related to the map size. For the training phase, the computational needs increase exponentially with the map size. The inference phase has lower computational needs compared to the training phase. The output is a map that depicts the used dataset as a representation in lower dimension.

3.3.4 Labeling phase

The goal of the labeling phase is to attribute additional information, such as the name, the color or the number of a cluster. Additional information relies on user's needs and is linked to the application. The generated map has observable clusters. Instead of identifying them manually, an automatic cluster identification phase has been created to do so.

The cluster identification phase wants to identify nodes that make up clusters and assign an information to the cluster to which they belong. The classification of map nodes is performed with Algorithm 2. It generates automatically clusters surrounded by boundaries, and an identifier is assigned to them. For example, if the node 43 with the coordinate (4,3) is localized inside the cluster "2," then this value is attributed to the node.

Algorithm 2. Cluster identification

```

For each Node on the Map not seen do
  | While Node is not a Boundary And has NeighborhoodNodes which are not Boundary do
  | | Attribute ClusterLabel
  | End
  | Increment ClusterLabel
End

```

Algorithm 3. Diagnostic of input data

```

For each Sample in Input Data do
  | Present the sample to the map and localize the exited BMU node
  | Association to the sample : the cluster number and HI of the BMU node
End

```

A *Node* is considered as seen if it has been assigned to a cluster identifier, called *ClusterLabel*. *NeighborhoodNodes* are nodes that touch it in all four directions: up, down, left and right.

In the end, a map is generated in which cluster regions appear and are defined by boundaries surrounding them. Let us recall that weight vectors are associated with each node of the map. Then, for each cluster, a health index is built for nodes (i,j) that belong to cluster C by Eq. (10)

$$H_{ij} = \frac{w_{ij2} - \min_{(i,j) \in C} w_{ij2}}{\max_{(i,j) \in C} w_{ij2} - \min_{(i,j) \in C} w_{ij2}} \quad (10)$$

in which scale node values of each cluster are in the range $[0, 1]$. Those values represent the current state of the studied system. Each cluster has a degradation trend. For a node, a high HI value represents a healthy condition, whereas a low HI indicates a high degradation or a failure.

When a database with more than one fault mode is used to train the network, several subregions could appear on some clusters. Those subregions are linked to different fault modes related to part failures. To identify fault modes automatically, input data need to be labeled through the diagnostic phase.

The diagnostic phase aims at creating knowledge for the fault identification phase. The input data from the training set are again presented to the map. BMU searching provides the cluster number to which they belong and the associated HI. It leads to the input association using Algorithm 3. The building of the fault mode indication becomes possible by knowing exactly which sample appears in which clusters to localize occurring faults.

3.4 Fault modes identification

The identification of the fault modes gives an insight on the system state, such as the probability that a specific fault occurs, or its evolution through the time. Without prior knowledge about datasets, the number of faults is estimated through the fault quantification phase. Their area on the map is approximated thanks to a straightforward methodology based on probabilistic theory during the fault

subregions identification phase. Then, by presenting iteratively datasets with one associated fault to a map, that has one or more unknown fault, the fault modes association phase identifies the unknown faults.

3.4.1 Fault quantification

The fault quantification phase attempts to evaluate automatically the number of different system faults from a dataset (e.g., pressure drop and over-temperature are two different errors). Within a given time series of data referred to as cycles of a specific system, it can be assumed that the last cycle before non-recoverable error corresponds to the error state and can be used to identify faults [43]. At this cycle, the system is considered to be in a defect state and is taken out of service for maintenance actions. Before the stopping of the system, the advanced degradation of parts that were about to fail took place. This should be visible in the dataset. So, the last cycles of the system before breakdown are presented to the SOM and the labeling phase provides the best matching unit (BMU) (i.e., the hit node on the SOM map). The hit BMU can be the same for several instances of the system (for example, different aircraft jet engines belonging to the same family). The quantity of instances hitting this BMU indicates the hit number.

Two ways to estimate the hit number are now introduced:

- H1: using the last cycle of the system
- H2: using the last cluster hit of the system

In the first case, H1, there is only one last hit on a specific cluster for the system. For example, in a dataset with 249 systems, there are 249 last hits, shared by all map clusters, representing the final (most likely) faulty condition in which the system is found to be itself before it was stopped for maintenance. It represents a “sure” faulty condition.

In the second case, H2, the last hit for each system in each cluster is taken into consideration. Then, in the dataset with 249 systems, there are 249 last hits on each cluster. In the case of a six clusters map, this leads to 1494 last hits, representing faulty conditions for those operational conditions.

In the next section, it is shown that H2 provides more information and reliability than H1, and it is a decent approximation. H2 is used for the case study.

The frequency of those hits over each map cluster is linked to the fault number. Indeed, they tend to gather in areas that can be distinguished separately. Those hits are managed with tools from probability theory to build a representation of those subregions. A good candidate is the

probability density function (PDF). The kernel density estimator [44] provides the estimation of the PDF such as:

$$\hat{f}_h(\vec{x}) = \frac{1}{n \cdot h} \sum_{i=1}^N K\left(\frac{\vec{x} - \vec{x}_i}{h}\right) \quad (11)$$

where $\vec{x} = (x^1, x^2, \dots, x^p)$ are real values, $\vec{x}_i$ are random samples from an unknown distribution, N is the number of observation, K is the kernel smoothing function, which is a Gaussian kernel, and h is the bandwidth. In this study, the bandwidth has been selected at 1% of the SOM map size, leaving out of consideration the boundary nodes between fault clusters. For a map of 25×25 nodes without cluster, $h = 0.25$. According to the targeted application, the rule could evolve. Other kernel parameters are automatically estimated by the algorithm [45]. The PDF represents the probability distribution using the data samples where the kernel distribution sums the smoothing functions for each data value to produce a smooth, continuous probability curve. A 3D-PDF generation is used for each subregion, with node coordinate (x, y) and the hit number as a frequency as z coordinate. The generated function can be estimated at any (x, y) point. The number of peaks of the PDF leads to the number of faults inside each map cluster. Therefore, if a dataset has one or two fault modes, the method should lead, respectively, to one and two peaks for each cluster on the map. The goal of this approach is to be able to get an overview on the number of fault modes that are present in a dataset, without relying on a priori information.

3.4.2 Fault subregion identification

A cluster is surrounded by boundaries, and several small subregions can be estimated inside, related to fault modes. Fault subregions are the extracted regions from a cluster. They are generated from the separation of PDF peaks for a cluster and are used to define each fault area. Indeed, PDF uses Gaussian functions, which can be separated geometrically. However, all cluster nodes are not necessarily classified in a fault area. This is the case for nodes with weak PDF value, far away from the peak center. To address this problem, the PDF of a cluster is estimated at every cluster node. A custom threshold is applied on each estimated PDF, and fault areas are then generated with their own self-defined boundaries. The remaining nodes inside each fault area, after applying the threshold, represent the failure. So, if the node (4,3) is inside the subregion Failure 1, then this node is attributed to it.

3.4.3 Fault modes association

The association of fault modes (i.e., subregions of cluster) with a physical part is performed with similar data, which

present a known defect mode. Prior knowledge about fault modes, which is previously identified, is used. For example, a dataset with one fault mode is presented to a generated map that has been trained with a dataset with two fault modes. The presented dataset will excite nodes from one of the two subregions previously determined. This subregion will correspond to the known fault mode of the presented dataset.

4 Case study on aircraft jet engines

4.1 Overview

To illustrate the present work, a case study on diagnosing jet engines is used. Engine condition monitoring (ECM) allows for regular assessment of the jet engine health state, based on in-flight measured variables on the engine itself, as well as its environment (the aircraft) in its flight conditions. Specific parameter trend evolutions have shown to be early indications for engine degradations, failures and/or malfunctions [9]. Engine condition monitoring consists of a wide range of activities assessing the jet engine health, from the mounting on-wing until its removal. After every flight, performance engineers evaluate the evolution of engine critical parameters and derive from those analyses to anticipate or to avoid incidents, to evaluate the effects of incidents or to provide a clear “no problem for the next few flights” indication. Whenever an engine gets into a much deteriorated health state, no longer allowing operation within regulatory limits, the performance engineer recommends its removal and a precise planning. Actions of the performance engineer aim not only to keep the engine in its optimum operational condition, but also to correct in an

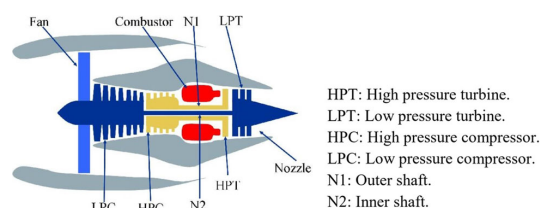


Fig. 4 Simplified diagram of the 90 K engine [46]

early stage any detected malfunction, allowing for staying within safe operation and also reducing fuel consumption and increasing operational punctuality. Therefore, early fault mode identification provides relevant information for the maintenance program. The use of the SOM neural network for this application is particularly interesting for its ability to map the input data without prior knowledge on fault modes. In general, the monitored system does not provide labeled data related to fault modes, whereas in the case of only one fault mode, the situation is straightforward. In the case of multiple fault modes, it becomes more complicated without a proper monitoring to identify them.

4.2 Input data

In this paper, datasets are generated [43] by using the C-MAPSS software [46]. C-MAPSS is a tool for the simulation of a realistic large commercial turbofan engine (Fig. 4) for the 90,000 lb thrust class. Thanks to editable input parameters, it is possible to specify operational profile, closed-loop controllers, and environmental conditions such as altitude. Furthermore, various degradations can be managed in different sections of the engine system.

Using this simulation environment, five datasets were generated by [43]. One of them was used for the prognostics challenge competition at International Conference on Prognostics and Health Management in 2008 (PHM08). In those datasets, the simulated engines have one or six operational conditions (flight phases such as Take-off, Cruise, etc.) driven by engine control settings (altitude, Mach number and Throttle Resolver Angle) and one or two fault modes. In PHM08 (Table 1), there are three datasets with one fault (i.e., #1, #2, and #5) and two with two faults (i.e., #3 and #4). The fault, corresponding to a failed system part, is, respectively, the HPC and the HPC and the fan (see Fig. 4). All dataset characteristics are summarized in Table 1.

Each dataset (i.e., #1 to #5) consists of multivariate time series and is divided into a training set and a testing set, generated by [43]. The database provides those sets in separate files: five training files and five testing files. The training subset is only used for training of the neural network (the learning), whereas the testing subset, with

Table 1 C-MAPSS dataset characteristics

Id	Name	Operational conditions	Fault modes	Failed system part	Number of engines
#1	FD001	1	1	HPC	100
#2	FD002	6	1	HPC	260
#3	FD003	1	2	HPC, Fan	100
#4	FD004	6	2	HPC, Fan	549
#5	FD005	6	1	HPC	218

Table 2 Output variables from C-MAPSS tool

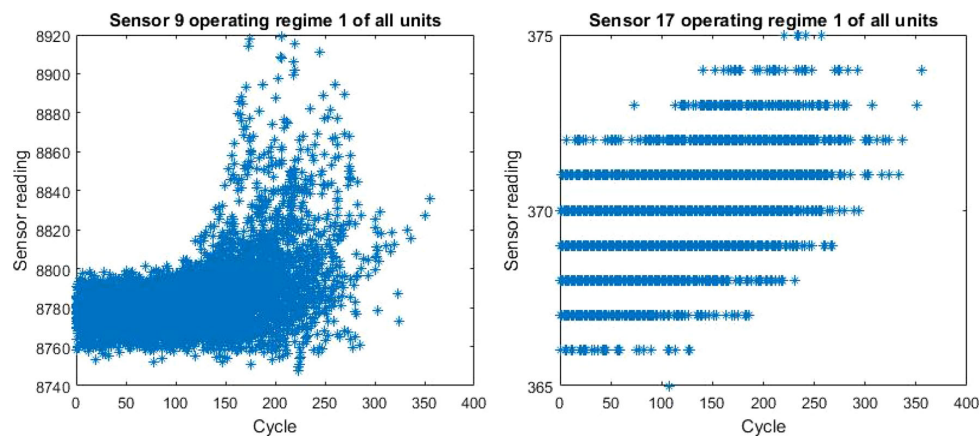
Sensor id	Symbol	Description	Units
1	T2	Total temperature at fan inlet	°R
2	T24	Total temperature at LPC outlet	°R
3	T30	Total temperature at HPC outlet	°R
4	T50	Total temperature at LPT outlet	°R
5	P2	Pressure at fan inlet	psia
6	P15	Total pressure in bypass duct	psia
7	P30	Total pressure at HPC outlet	psia
8	Nf	Physical fan speed	rpm
9	Nc	Physical core speed	rpm
10	epr	Engine pressure ration (P50/P2)	–
11	Ps30	Static pressure at HPC outlet	psia
12	Phi	Ratio of fuel flow to Ps30	pps/psi
13	NRf	Corrected fan speed	rpm
14	NRc	Corrected core speed	rpm
15	BPR	Bypass ration	–
16	farB	Burner fuel–air ratio	–
17	htBleed	Bleed enthalpy	–
18	Nf_dmd	Demanded fan speed	rpm
19	PCNfR_dmd	Demanded corrected fan speed	rpm
20	W31	HPT coolant bleed	lbm/s
21	W32	LPT coolant bleed	lbm/s

Table 3 Extract of the dataset #4

Engine	Cycle	CS1	CS2	CS3	S1	S2	S3	S4	S5	S6	S7	S8	S9
1	1	42.0049	0.84	100	445	549.68	1343.43	1112.93	3.91	5.7	137.36	2211.86	8311.32
1	2	20.002	0.7002	100	491.19	606.07	1477.61	1237.5	9.35	13.61	332.1	2323.66	8713.6
1	3	42.0038	0.8409	100	445	548.95	1343.12	1117.05	3.91	5.69	138.18	2211.92	8306.69
1	4	42	0.84	100	445	548.7	1341.24	1118.03	3.91	5.7	137.98	2211.88	8312.35
1	5	25.0063	0.6207	60	462.54	536.1	1255.23	1033.59	7.05	9	174.82	1915.22	7994.94
1	6	34.9996	0.84	100	449.44	554.77	1352.87	1117.01	5.48	7.97	193.82	2222.77	8340
1	7	0.0019	0.0001	100	518.67	641.83	1583.47	1393.89	14.62	21.58	552.45	2387.92	9050.5
1	8	41.9981	0.84	100	445	549.05	1344.16	1110.77	3.91	5.69	137.13	2211.92	8307.28
1	9	42.0016	0.84	100	445	549.55	1342.85	1101.67	3.91	5.7	138.02	2211.9	8307.81
1	10	25.0019	0.6217	60	462.54	536.35	1251.91	1041.37	7.05	9.01	174.7	1915.23	8005.83
1	11	20.0016	0.7	100	491.19	606.88	1478.02	1233.07	9.35	13.61	333.22	2323.7	8709.62
1	12	34.9993	0.84	100	449.44	554.53	1365.99	1122.73	5.48	7.98	193.67	2222.78	8337.46
1	13	24.9986	0.62	60	462.54	536.32	1257.84	1040.87	7.05	9.01	174.53	1915.28	8000.07
1	14	20.0056	0.7008	100	491.19	607.32	1470.33	1242.41	9.35	13.61	333.71	2323.72	8714.35
Engine	Cycle	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21
1	1	1.01	41.69	129.78	2387.99	8074.83	9.3335	0.02	330	2212	100	10.62	6.367
1	2	1.07	43.94	312.59	2387.73	8046.13	9.1913	0.02	361	2324	100	24.37	14.6552
1	3	1.01	41.66	129.62	2387.97	8066.62	9.4007	0.02	329	2212	100	10.48	6.4213
1	4	1.02	41.68	129.8	2388.02	8076.05	9.3369	0.02	328	2212	100	10.54	6.4176

Table 3 (continued)

Engine	Cycle	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21
1	5	0.93	36.48	164.11	2028.08	7865.8	10.8366	0.02	305	1915	84.93	14.03	8.6754
1	6	1.02	41.44	181.9	2387.87	8054.1	9.3346	0.02	330	2223	100	14.91	8.9057
1	7	1.3	46.94	520.48	2387.89	8127.92	8.396	0.03	391	2388	100	38.93	23.4578
1	8	1.01	41.6	129.65	2387.97	8075.99	9.3679	0.02	329	2212	100	10.55	6.2787
1	9	1.02	41.44	129.65	2388	8071.13	9.3384	0.02	328	2212	100	10.63	6.3055
1	10	0.94	36.24	164.08	2028.13	7869.41	10.9141	0.02	305	1915	84.93	14.34	8.6119
1	11	1.07	43.86	312.96	2387.83	8050.06	9.1667	0.02	363	2324	100	24.63	14.6705
1	12	1.02	41.45	181.71	2387.86	8056.31	9.3041	0.02	332	2223	100	14.68	8.8752
1	13	0.94	36.42	163.67	2028.14	7865.15	10.8388	0.02	305	1915	84.93	14.41	8.6062
1	14	1.07	43.92	313.3	2387.85	8051.34	9.2272	0.02	364	2324	100	24.3	14.7105

**Fig. 5** Trend of sensors in a selected regimes. (Left) Inconsistent end-life trends. (Right) Piecewise trends

different data, is only used for the network validation. In this case study, a dataset contains three input variables representing the engine operational settings, that generate one or six operational conditions and 21 output sensors (Table 2). Dataset is comparable between themselves whether they have the same operation settings, generating

the same number of operational conditions. Thus, FD001 and FD003 are comparable as well as FD002 and FD004. However, the FD005 dataset cannot be compared with FD002 or FD004 because the values of the operational settings are not compatible, even if it has six operational conditions.

Table 3 shows an extract of available data for the dataset #4. The other datasets follow the same format.

A reduction in the number of sensors could lead to a drastic reduction in the computational time for the training of the neural network. Following the work of [10, 22, 47] for the PHM08 dataset number #5, the only relevant seven sensors were found to be: 2, 3, 4, 7, 11, 12 and 15.

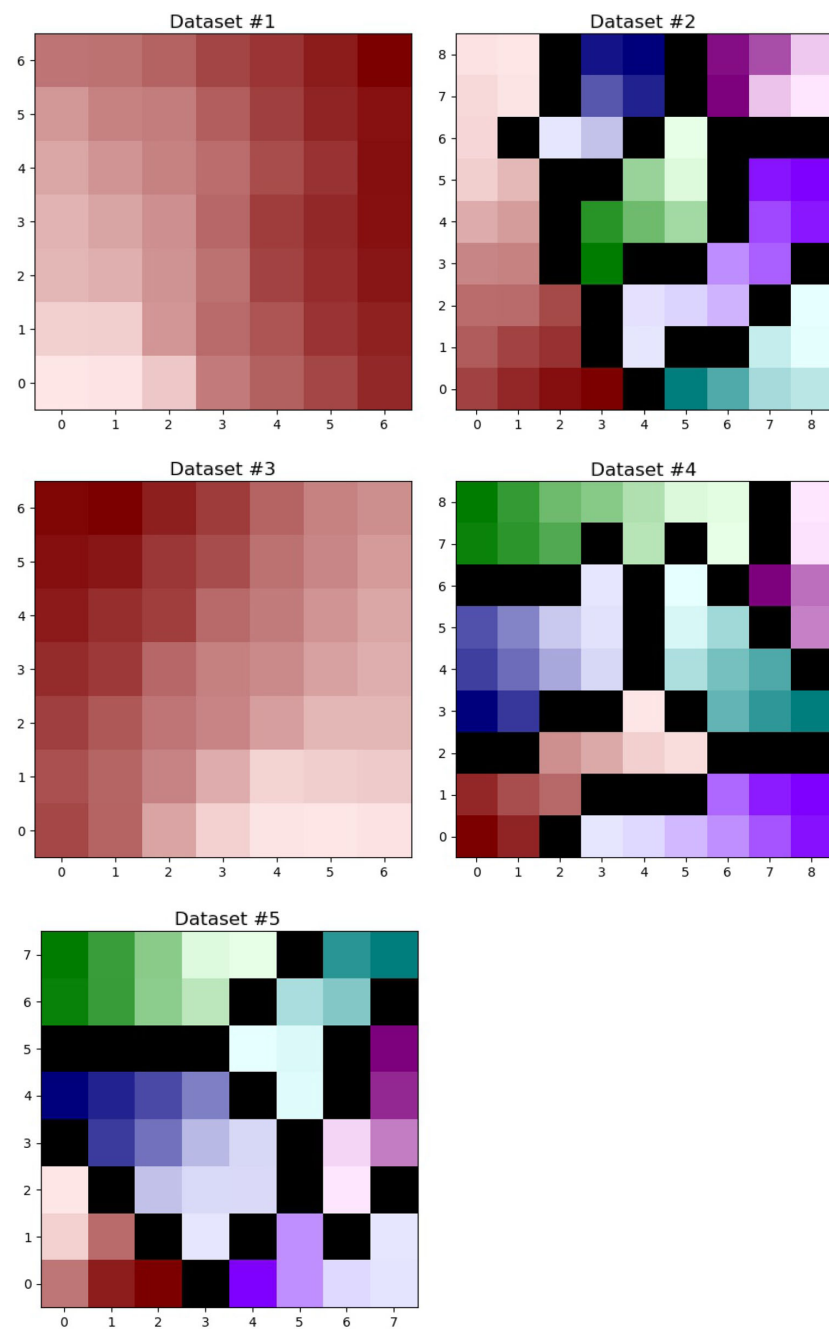
In fact, among the 21 sensors, constant or binary trend is observed on several sensors that do not provide degradation behaviors. Sensors 1, 5, 6, 10, 16, 17, 18 and 19 are concerned and not considered from the selection. Others sensors provide similar information such as sensors 8 and 13

Table 4 SOM information for all datasets

Id	Number of observations	Features	Map size	Iterations ^a
#1	20,631	3	7	24,500
#2	53,759	3	9	40,500
#3	24,720	3	7	24,500
#4	61,249	3	9	40,500
#5	45,918	3	8	32,000

^aEstimated following the rule of thumb

Fig. 6 SOM maps from datasets #1 to #5 with three operation conditions as features



with the sensor 11 by looking at the correlation coefficient with a threshold of 85%. Sensors 9 and 14 show inconsistent end-life trends among the engines (Fig. 5, left), and the sensor 17 is a piecewise constant function (Fig. 5,

right). Finally, the sensors 20 and 21 do not bring a clear trend throughout the unit's life according to [22]. Through those steps, the final seven sensors are determined. The

Table 5 Training SOM information for all datasets

Id	Number of observations	Features	Map size	Iterations ^a
#1	20,631	7	19	180,500
#2	53,759	7	24	288,000
#3	24,720	7	20	200,000
#4	61,249	7	25	312,500
#5	45,918	7	23	264,500

^aEstimated following the rule of thumb

same selected sensors have been taken into account for the datasets #1, #2, #3 and #4.

4.3 Operational mode labeling

The case study contains five datasets generated by [43] (see Table 1), where it is known that one or six different operational conditions are used. According to the complexity of the case study, manual operational mode labeling may not be possible by hand. To demonstrate the power of the automated SOM, they are performed following the system map process introduced in the previous section (see Fig. 2).

The three operational settings (altitude, Mach number and Throttle Resolver Angle) in the dataset are used as inputs to the SOM. Due to the number of inputs, the map size is determined with Eq. (3) and the result is divided by four, custom factors established through multiple empirical experiments. A bigger (or smaller) map leads to an increase (and decrease) in the cluster numbers and may lead to an inconsistency in the representation of information. Further development will be done to address this empirical estimation. The convergence criteria of the “rule of thumb” are used, leading to a number of iterations of 500 times the number of neurons. Table 4 summarizes SOM information used.

The training phase generates maps in Fig. 6.

The maps reveal one cluster for the datasets #1 and #3, whereas the datasets #2, #4 and #5 show six clusters. This means that there is one operational condition for #1 and #3 and six operational conditions for #2, #4 and #5, which is in line with Table 4. The diagnostic phase from the system map process provides exactly the same operational mode labeling as what was obtained manually.

4.4 System map generation

4.4.1 Preprocessing and training

The SOM will now be trained with the seven sensors identified in the previous section. The number of neurons is

determined with Eq. (3) and divided by two to reduce the computational time. Table 5 summarizes SOM information used.

The training phase generates the maps in Fig. 7.

For each dataset in Fig. 7, the number of clusters is identical to Fig. 6, corresponding to the number of operational modes. The cluster identification phase provides the same cluster information with the seven selected sensors, compared to the operational mode labeling in Sect. 4.3. It provides reliability in the unsupervised approach.

It appears that the maps for the datasets #3 and #4 show two darker colors on each cluster, which means that there are two fault modes in each cluster. This matches the information of Table 1. The color degradation corresponds to the evolution of the HI (i.e., degradation status). Lighter colors correspond to a healthy system, whereas the darker colors mean an advanced degradation. Other datasets have only one dark colors on each cluster; they have one fault mode. To confirm that, mathematical tools are now introduced.

4.5 Fault mode identification

4.5.1 Fault quantification

The kernel density estimator Eq. (11) is applied on each cluster of each map under H2. It generates a PDF to identify the number of fault modes. Thanks to the SOM map (Fig. 7), the minimum probability density function kernel bandwidth without cluster boundaries can be obtained (see Table 6).

Figure 8 presents the PDF generated for a particular cluster. On those figures, PDF values of node coordinates (x, y) and hit numbers z are normalized. The number of fault modes is easily identifiable visually as well as automatically. This procedure is performed for each generated cluster on each map, and results are compared with the information provided by the datasets. It results that the fault number is well identified, with two fault modes for the database #3, #4 and one for the others.

In Sect. 3.4.1, two ways to evaluate the hit number were presented. Here, we would like to evaluate whether H1 is more pertinent than H2. H1 is relevant in terms of interpretation. However, the dataset used provides few engines (Table 1). Table 7 summarizes the number of samples according to hypotheses presented in Sect. 3.4.1.

Under hypothesis H1, due to a lack of samples, only one out of two fault modes is identified. For dataset #4, it has two fault modes and six operational conditions; the 249 samples of H1 represent around 21 samples per fault per operational conditions. Following the philosophy of H2, around 100 samples per fault per operational conditions are needed for proper identification.

Fig. 7 SOM map from datasets #1 to #5 with 7 physical sensor data as features

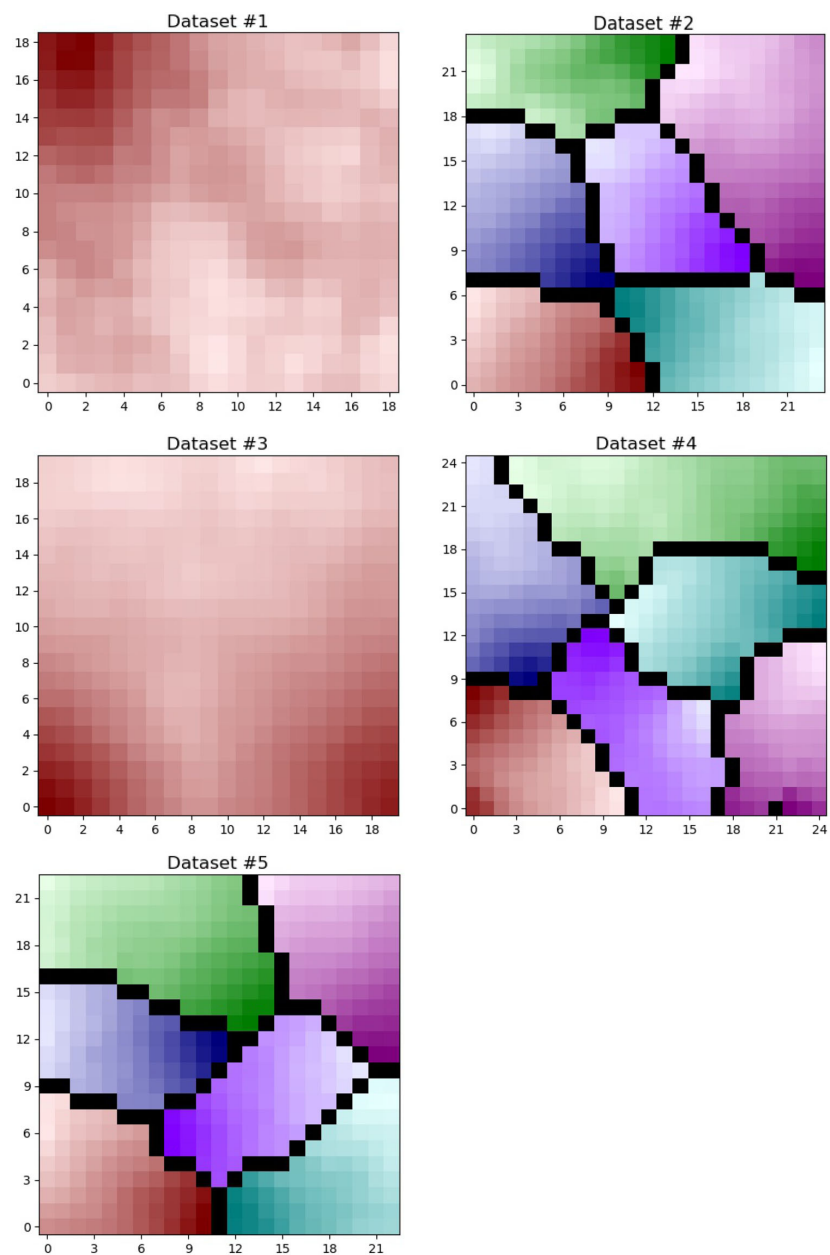


Table 6 PDF kernel bandwidth

Id	Map size	Bandwidth h
#1	19	0.19
#2	24	0.22
#3	20	0.20
#4	25	0.22
#5	23	0.21

For each engine, the last flight cycle spent on each operational condition (H2) is compared to the last flight cycle before a fault occurs (H1) to quantify the reliability of H2. For example, the engine 12 (Table 8) has six operational conditions. Table 8 summarizes the flight cycle for both cases and the error of H2 compared to H1. The

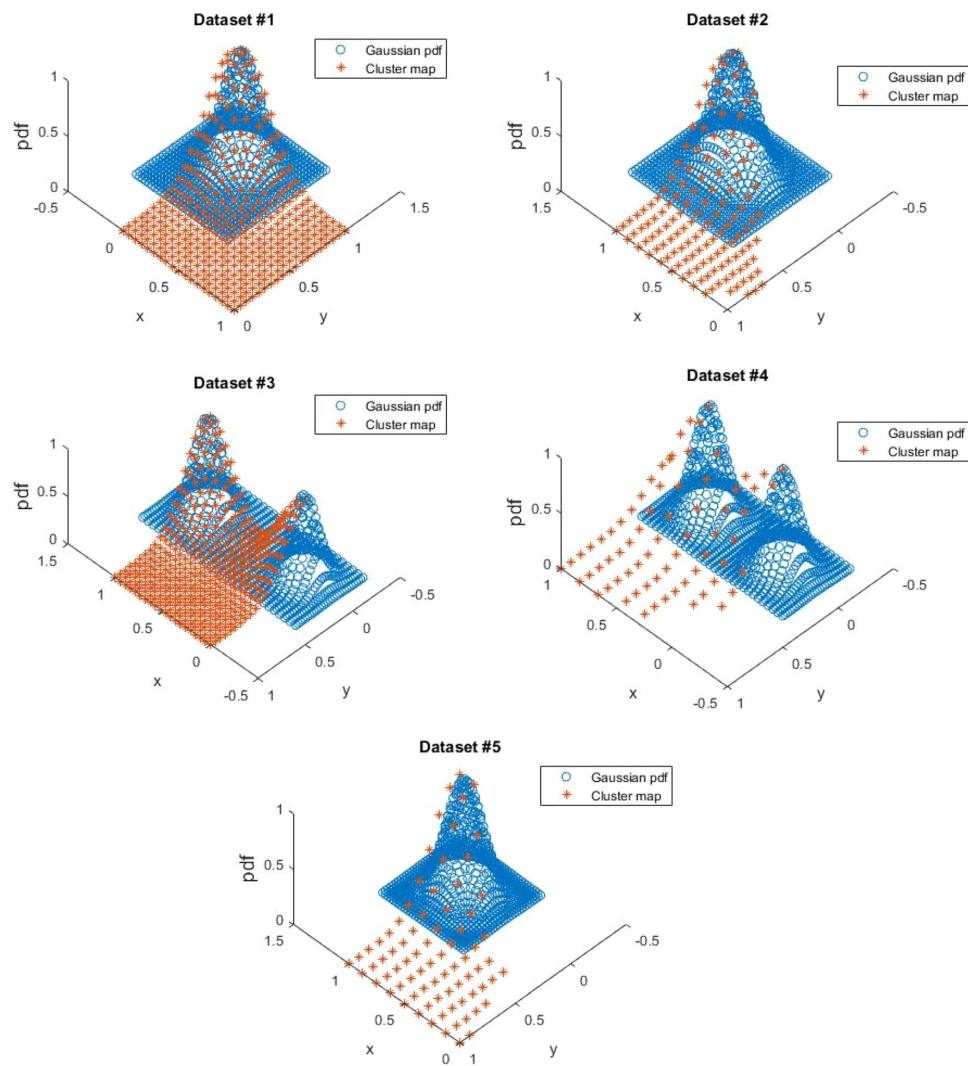


Fig. 8 PDF on a cluster for datasets #1 to #5

Table 7 Number of sample for datasets #1 to #5

Id	H1	H2
#1	100	100
#2	260	1560
#3	100	100
#4	249	1494
#5	218	1308

Table 8 Example of comparison for engine 12

Engine 12			
Cluster	H1	H2	Error (%)
1	320	260	18.75 ^a
2		320	0.00
3		300	6.25
4		289	9.69
5		277	13.44 ^a
6		310	3.13

^aError above 10%

error is above 10% (custom threshold) for the clusters 1 and 5. That means engine 12 belongs to the group of engines, where H1 is more relevant than H2.

Fig. 9 Failure modes for datasets #1 to #5

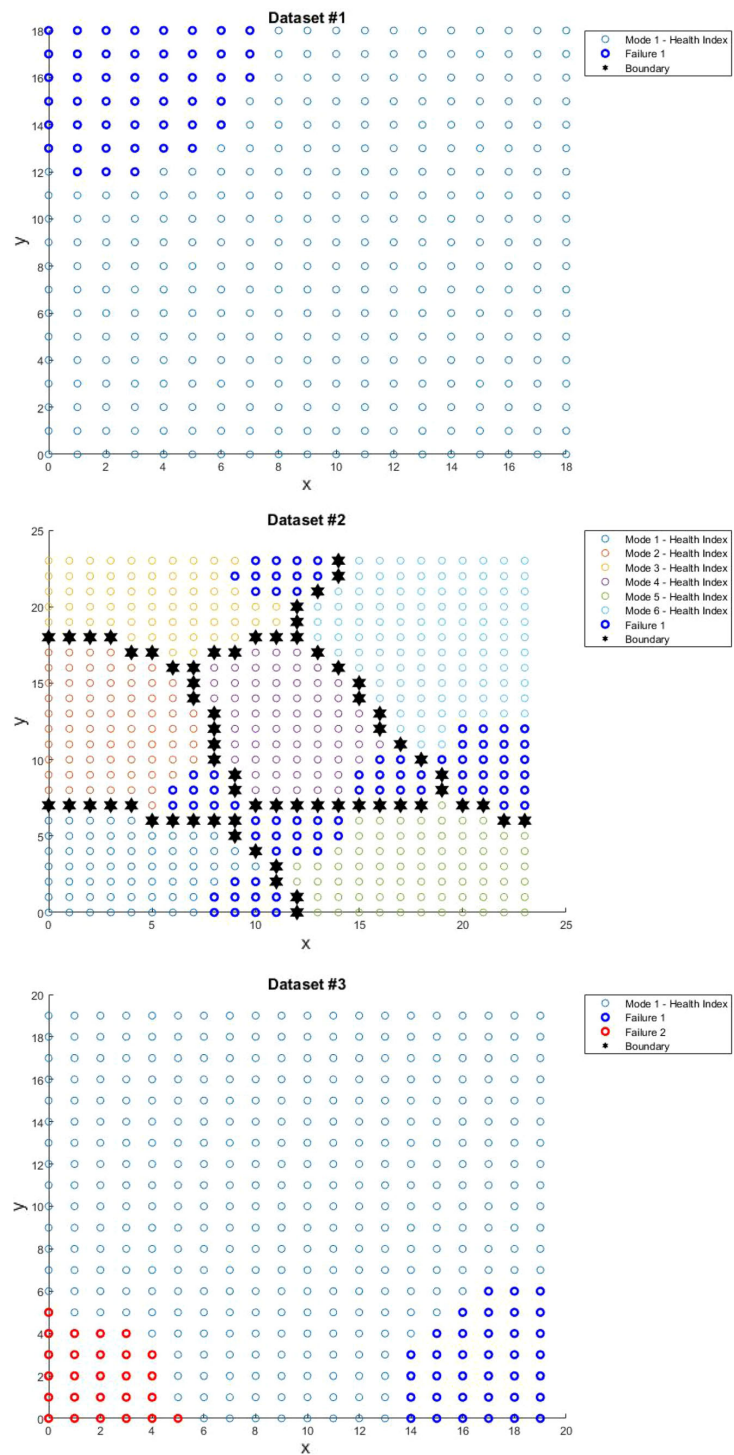
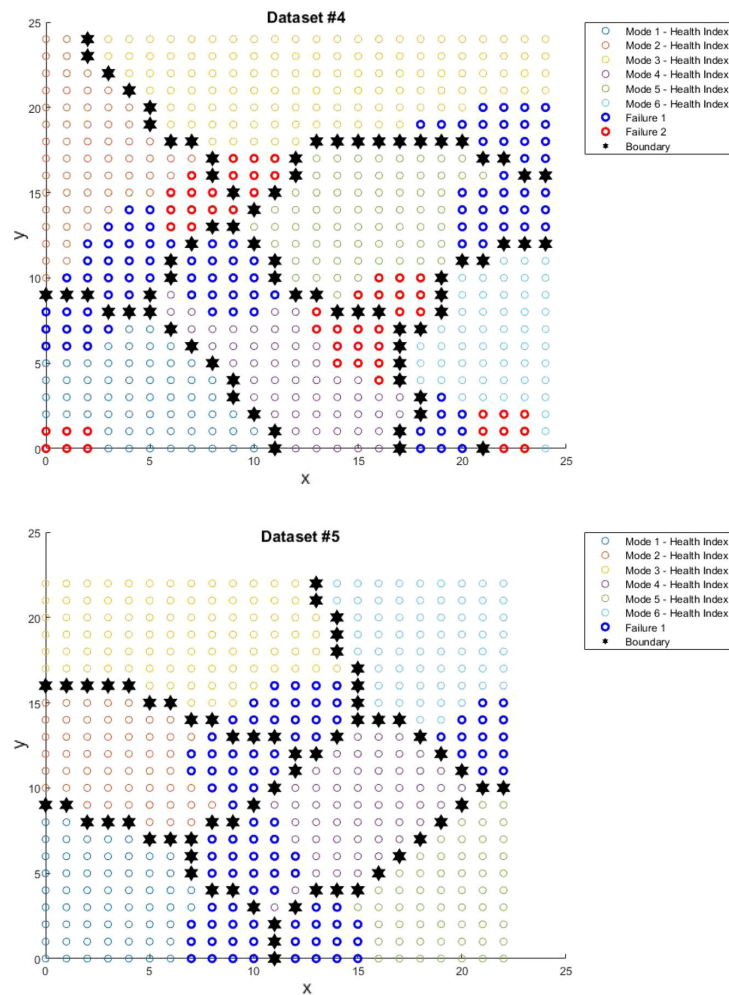


Fig. 9 continued



This evaluation is performed for all datasets with a custom threshold of 10%. It results that there are around 3% of engines where H2 is not relevant. For a dataset of 249 engines, H1 is more relevant than H2 for only eight engines. H2 is therefore acceptable for this case study.

4.5.2 Fault subregions identification

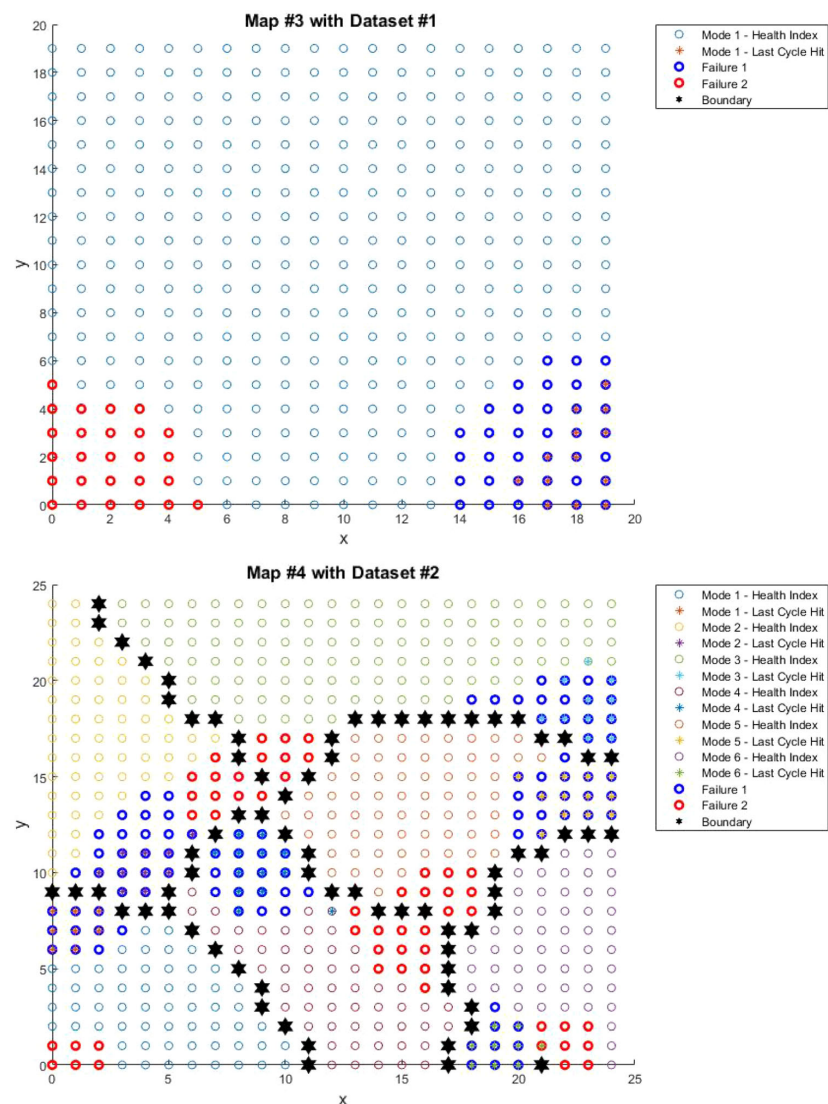
For the fault subregions identification, only map nodes that are included in the PDF shape are retained. The user customizes the threshold according to the required precision. This is linked to the probability of training engines of which their last flight cycle has hit the fault mode area. A threshold of 0.40 was used, meaning that all nodes with a probability density lower than 40% are removed. The results are shown in Fig. 9.

When an engine degrades up to a point a failure mode is likely to happen, fault mode area is crossed on each operational mode. Based on this observation, the fault mode is considered to be the same on each map cluster and labeled as failure in Fig. 9.

4.5.3 Fault modes association

The fault modes association phase can only be performed on similar datasets as explained in Sect. 4.2. Thus, the datasets #1 and #3 are similar (similar operational settings and operation conditions) as well as the datasets #2 and #4. The use of datasets with one fault mode (i.e., the datasets #1 and #2) to identify the same failure on the maps trained with two failures (i.e., the datasets #3 and #4) results in a

Fig. 10 Fault modes identification for maps #3 and #4



clear and unambiguous identification. The found fault mode is associated with the corresponding failure.

As shown in Fig. 9, the datasets #3 and #4 have both two fault modes, named, respectively, failures 1 and 2. Yet, the faulty system part is unknown. As mentioned, datasets #1 and #2 are used to identify one of the two failures on maps trend with datasets #3 and #4.

Figure 10 shows that all engines from the datasets #1 and #2 are, respectively, in the cluster failure 1 of the datasets #3 and #4, following the hit number estimation 'H2' (see Sect. 3.4.1). Knowing that #1 and #2 have HPC fault modes (Table 1), failures 1 and 2 are, respectively,

identified as HPC and fan fault mode. With this knowledge, the fault modes of each engine in the datasets #3 and #4 can be determined.

However, on the map #4 (Fig. 10), some engines are out of a failure area, such as in modes 3 and 4, or misclassified, such as in mode 6. With the hit number estimation 'H2,' it represents 0.19% of error, whereas with the hit number estimation 'H1,' all engines are perfectly classified for this study. The failure identification procedure is then considered satisfactory.

5 Conclusion

This study addresses early detection and classification of faults through an unsupervised learning approach, without prior knowledge, based on self-organizing maps (SOMs). This neural network is at the core of the automatized approach. A complete process to comprehend the concept and to use SOM has been presented. The SOM has some advantage such as unsupervised training and is useful for dimensionality reduction and representation of complex and large datasets thanks to the map visualization. A methodology has been described to build and to configure the SOM according to the case study. The article highlights the possibility to identify the operational mode and fault modes inside generated maps with a methodology relying on the kernel density estimation. The methodology has been illustrated on a case study for diagnosing jet engine datasets. Without prior knowledge on the faults, the proposed algorithm was able to identify the number of operational modes as well as the fault mode number for the five datasets. Furthermore, the fault subregions identification estimates fault mode areas on the map, leading to failures identification on each cluster.

The unsupervised classifier used is a SOM neural network. There exist different types of unsupervised clustering techniques such as hierarchical or Bayesian clustering that could provide different results according to the case study [48]. Another candidate for future work could be a network based on restricted Boltzmann machines (RBM) [49] that will provide a probability distribution over its set of inputs.

In the current study, the automatic fault mode identification was experimented up to two fault modes. For future work, the study should be extended to more than two failure modes on the same case study. Two ways to evaluate the hit number have been explored. Other hypothesis could be examined. The used feature for #1 to #4 was supposed to be same than for #5. A generic approach to get automatically the best set of feature for different types of data will be a good solution to consider, as well as for the custom factor for the map size reduction. A study could be performed to explore the use of the presented approach with a different case study.

Acknowledgements The author affiliated to Sogeti High Tech and ISAE-SUPAERO gratefully acknowledges his colleagues who provided insight and expertise through this paper.

Compliance with ethical standards

Conflict of interest Sébastien Schwartz, Juan José Montero Jimenez, Michel Salaün and Rob Vingerhoeds declare that they have no conflict of interest.

References

1. Simon DL, Rinehart AW (2014) A model-based anomaly detection approach for analyzing streaming aircraft engine measurement data. In: Volume 6: Ceramics; controls, diagnostics and instrumentation; education; manufacturing materials and metallurgy, Düsseldorf, Germany. ASME, p V006T06A032. <https://doi.org/10.1115/gt2014-27172>
2. Naderi E, Meskin N, Khorasani K (2012) Nonlinear fault diagnosis of jet engines by using a multiple model-based approach. *J Eng Gas Turbines Power* 134(1):011602. <https://doi.org/10.1115/1.4004152>
3. Zeng D, Zhou D, Tan C, Jiang B (2018) Research on model-based fault diagnosis for a gas turbine based on transient performance. *Appl Sci* 8(1):148. <https://doi.org/10.3390/app8010148>
4. Naderi E, Khorasani K (2017) Data-driven fault detection, isolation and estimation of aircraft gas turbine engine actuator and sensors. In: 2017 IEEE 30th Canadian conference on electrical and computer engineering (CCECE), Windsor, ON. IEEE, pp 1–6. <https://doi.org/10.1109/ccece.2017.7946715>
5. Sarkar S, Jin X, Ray A (2011) Data-driven fault detection in aircraft engines with noisy sensor measurements. *J Eng Gas Turbines Power* 133(8):081602. <https://doi.org/10.1115/1.4002877>
6. Svärd C, Nyberg M, Frisk E, Krysander M (2014) Data-driven and adaptive statistical residual evaluation for fault detection with an automotive application. *Mech Syst Signal Process* 45(1):170–192. <https://doi.org/10.1016/j.ymssp.2013.11.002>
7. Pourbabaee B, Meskin N, Khorasani K (2016) Robust sensor fault detection and isolation of gas turbine engines subjected to time-varying parameter uncertainties. *Mech Syst Signal Process* 76–77:136–156. <https://doi.org/10.1016/j.ymssp.2016.02.023>
8. Venkatasubramanian V (2005) Prognostic and diagnostic monitoring of complex systems for product lifecycle management: challenges and opportunities. *Comput Chem Eng* 29(6):1253–1263. <https://doi.org/10.1016/j.compchemeng.2005.02.026>
9. Vingerhoeds RA, Janssens P, Netten BD, Aznar Fernández-Montesinos M (1995) Enhancing off-line and on-line condition monitoring and fault diagnosis. *Control Eng Pract* 3(11):1515–1528. [https://doi.org/10.1016/0967-0661\(95\)00162-N](https://doi.org/10.1016/0967-0661(95)00162-N)
10. Montero Jimenez JJ, Vingerhoeds R (2018) Enhancing operational fault diagnosis by assessing multiple operational modes. In: MOSIM'18—Conférence Internationale de Modélisation, Optimisation et Simulation, Toulouse
11. Kohonen T (1982) Self-organized formation of topologically correct feature maps. *Biol Cybern* 43(1):59–69. <https://doi.org/10.1007/BF00337288>
12. Germen E, Başaran M, Fidan M (2014) Sound based induction motor fault diagnosis using Kohonen self-organizing map. *Mech Syst Signal Process* 46(1):45–58. <https://doi.org/10.1016/j.ymssp.2013.12.002>
13. Côme E, Cottrell M, Verleysen M, Lacaille J (2010) Aircraft engine health monitoring using self-organizing maps. In: Perner P (ed) *Advances in data mining*. Springer, Berlin, pp 405–417. https://doi.org/10.1007/978-3-642-14400-4_31
14. Cottrell M, Gaubert P, Eloy C, François D, Hallaux G, Lacaille J, Verleysen M (2009) Fault prediction in aircraft engines using self-organizing maps. In: Principe JC, Miikkulainen R (eds) *Advances in self-organizing maps*, vol 5629. Springer, Berlin, pp 37–44. https://doi.org/10.1007/978-3-642-02397-2_5
15. Katunin A, Amarowicz M, Chrzanowski P (2015) Faults diagnosis using self-organizing maps: a case study on the

- DAMADICS benchmark problem, pp 1673–1681. <https://doi.org/10.15439/2015f26>
16. Yu H, Khan F, Garaniya V (2015) Risk-based fault detection using self-organizing map. *Reliab Eng Syst Saf* 139:82–96. <https://doi.org/10.1016/j.res.2015.02.011>
 17. Chen X, Yan X (2012) Using improved self-organizing map for fault diagnosis in chemical industry process. *Chem Eng Res Des* 90(12):2262–2277. <https://doi.org/10.1016/j.cherd.2012.06.004>
 18. Dharshini R, Hemanandhini S (2016) Brain tumor segmentation based on self organising map and discrete wavelet transform. In: 2016 international conference on computer communication and informatics (ICCCI), Coimbatore, India. IEEE, pp 1–9. <https://doi.org/10.1109/iccci.2016.7479960>
 19. Peel L (2008) Data driven prognostics using a Kalman filter ensemble of neural network models. In: 2008 international conference on prognostics and health management. <https://doi.org/10.1109/phm.2008.4711423>
 20. Jolliffe I (2011) Principal component analysis. Springer, Berlin. <https://doi.org/10.1007/b98835>
 21. Sammon JW (1969) A nonlinear mapping for data structure analysis. *IEEE Trans Comput* 100(5):401–409. <https://doi.org/10.1109/t-c.1969.222678>
 22. Wang T, Jianbo Y, Siegel D, Lee JA (2008) Similarity-based prognostics approach for remaining useful life estimation of engineered systems. In: 2008 international conference on prognostics and health management. IEEE, pp 1–6. <https://doi.org/10.1109/phm.2008.4711421>
 23. Guyon I, Elisseeff A (2003) An introduction to variable and feature selection. *J Mach Learn Res* 3:1157–1182
 24. Jovic A, Brkic K, Bogunovic N (2015) A review of feature selection methods with applications. In: 2015 38th international convention on information and communication technology, electronics and microelectronics (MIPRO), Opatija, Croatia. IEEE, pp 1200–1205. <https://doi.org/10.1109/mipro.2015.7160458>
 25. Saeys Y, Inza I, Larranaga P (2007) A review of feature selection techniques in bioinformatics. *Bioinformatics* 23(19):2507–2517. <https://doi.org/10.1093/bioinformatics/btm344>
 26. Visalakshi S, Radha V (2014) A literature review of feature selection techniques and applications: review of feature selection in data mining. In: 2014 IEEE international conference on computational intelligence and computing research. IEEE, Coimbatore, India, pp 1–6. <https://doi.org/10.1109/icci.2014.7238499>
 27. Kohonen T (1997) Springer series in information sciences, vol 30. Springer, Berlin. <https://doi.org/10.1007/978-3-642-97966-8>
 28. Kohonen T (2014) Unigrafia, Helsinki, Finland
 29. Zin ZM (2014) Cluster and visualize data using 3D self-organizing maps. In: 2014 11th international conference on ubiquitous robots and ambient intelligence (URAI). IEEE, pp 163–168. <https://doi.org/10.1109/urai.2014.7057523>
 30. Azcarraga A, Manalili S (2011) Design of a structured 3D SOM as a music archive, pp 188–197. https://doi.org/10.1007/978-3-642-21566-7_19
 31. Gorricha J, Lobo V (2012) Improvements on the visualization of clusters in geo-referenced data using self-organizing maps. *Comput Geosci* 43:177–186. <https://doi.org/10.1016/j.cageo.2011.10.008>
 32. El Tobely T, Salem A (2005) Position detection of unexploded ordnance from airborne magnetic anomaly data using 3-D self organized feature map. In: Proceedings of the fifth IEEE international symposium on signal processing and information technology, 2005. IEEE, pp 322–327. <https://doi.org/10.1109/isspit.2005.1577117>
 33. Fujimura K, Masuda K, Fukui Y (2006) A consideration on the multi-dimensional topology in self-organizing maps. In: 2006 international symposium on intelligent signal processing and communications, IEEE, pp 825–828. <https://doi.org/10.1109/ispacs.2006.364772>
 34. Côme E, Cottrell M, Verleysen M, Lacaille J (2010) Self organizing star (SOS) for health monitoring. In: European conference on artificial neural networks, pp 99–104
 35. Tian J, Azarian MH, Pecht M (2014) Anomaly detection using self-organizing maps-based k-nearest neighbor algorithm. In: European conference of the prognostics and health management society
 36. Engelbrecht AP (2007) Computational intelligence: an introduction, 2nd edn. Wiley, Hoboken. <https://doi.org/10.1002/9780470512517>
 37. Su M-C, Liu T-K, Chang H-T (1999) An efficient initialization scheme for the self-organizing feature map algorithm. In: IJCNN'99. International joint conference on neural networks. Proceedings (Cat. No. 99CH36339), vol 3. IEEE, pp 1906–1910. <https://doi.org/10.1109/ijcnn.1999.832672>
 38. Kohonen T (2001) Springer series in information sciences, vol 30, 3rd edn. Springer, Berlin. <https://doi.org/10.1007/978-3-642-56927-2>
 39. Kaski S, Venna J, Kohonen T (2000) Coloring that reveals cluster structures in multivariate data. *Aust J Intell Inf Process Syst* 6:82–88
 40. Alahakoon D, Halgamuge SK, Srinivasan B (2000) Dynamic self-organizing maps with controlled growth for knowledge discovery. *IEEE Trans Neural Netw* 11(3):601–614. <https://doi.org/10.1109/72.846732>
 41. Natita W, Wiboonsak W, Dusadee S (2016) Appropriate learning rate and neighborhood function of self-organizing map (SOM) for specific humidity pattern classification over Southern Thailand. *Int J Model Optim* 6(1):61–65. <https://doi.org/10.7763/IJMO.2016.V6.504>
 42. Zhang W, Wang J, Jin D, Oreopoulos L, Zhang Z (2018) A deterministic self-organizing map approach and its application on satellite data based cloud type classification. In: Conference IEEE Big Data
 43. Saxena A, Goebel K, Simon D, Eklund N (2008) Damage propagation modeling for aircraft engine run-to-failure simulation. In: 2008 international conference on prognostics and health management. IEEE, pp 1–9. <https://doi.org/10.1109/phm.2008.4711414>
 44. Kafadar K, Bowman AW, Azzalini A (1999) Applied smoothing techniques for data analysis: the kernel approach with S-PLUS. *J Am Stat Assoc* 94(447):982. <https://doi.org/10.2307/2670015>
 45. Bowman AW, Azzalini A (1997) Applied smoothing techniques for data analysis: the kernel approach with S-Plus illustrations, vol 18. Oxford University Press, Oxford
 46. Frederick DK, DeCastro JA, Litt JS (2007) User's guide for the commercial modular aero-propulsion system simulation (C-MAPSS)
 47. Hu C, Youn BD, Wang P, Taek Yoon J (2012) Ensemble of data-driven prognostic algorithms for robust prediction of remaining useful life. *Reliab Eng Syst Saf* 103:120–135. <https://doi.org/10.1016/j.res.2012.03.008>
 48. Fusco G, Perez J (2019) Bayesian network clustering and self-organizing maps under the test of Indian Districts. A comparison. *Cybergeogeo*. <https://doi.org/10.4000/cybergeogeo.31909>
 49. Zhang X, Yao L, Wang X, Monaghan J, Mcalpine D, Zhang Y (2019) Cs Eess Q-Bio. [arXiv:1905.04149](https://arxiv.org/abs/1905.04149)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Intentionally left blank

Ontology and CBR integration article

The content in this appendix corresponds to work accepted for publication in The 35th annual European Simulation and Modelling Conference, 2021. Reprinted, with permission, Hugo Muñoz Hernández, Juan José Montero Jiménez, Rob Vingerhoeds. Integrating ontologies and case-based reasoning for the development of knowledge intensive intelligent systems. Accepted for publication.

Intentionally left blank

INTEGRATING ONTOLOGIES AND CASE-BASED REASONING FOR THE DEVELOPMENT OF KNOWLEDGE-INTENSIVE INTELLIGENT SYSTEMS.

Hugo Muñoz-Hernández
Rob Vingerhoeds
ISAE-SUPAERO

10 Avenue Edouard Belin 31055 Toulouse, France
hugo.munoz-hernandez@student.isae-supaero.fr
rob.vingerhoeds@isae-supaero.fr

Juan José Montero-Jiménez
TEC-Tecnológico de Costa Rica
Calle 15, Avenida 14

1 km al sur Basílica de los Ángeles
Provincia de Cartago, Cartago, 30101, Costa Rica
juan.montero@itcr.ac.cr

KEYWORDS

Ontology, Case-Based Reasoning (CBR), similarity function.

ABSTRACT

Case-Based Reasoning (CBR) allows emulating the human inference of solutions to problems profiting from previous experience. The integration of CBR with ontologies, structured organization of semantic knowledge, has been in the attention for some time, aiming to create powerful knowledge-intensive systems capable of proposing appropriate solutions to problems. This entails having collected an appropriate number of previous cases as well as having established a suitable ontology for the application domain. This paper focuses on the integration of CBR and ontologies to support the case representation, case base storage, and semantic similarity estimation. Different alternatives for such integration are explored and the approach has been tested in the creation of a Decision-Support System for the design of predictive maintenance systems. This work opens the window to significantly improve the capabilities of a CBR system by using the knowledge materialization and the reasoning features of a specific domain ontology.

INTRODUCTION

Since a long time ago, man has been fascinated with making a machine that had the same capabilities as human beings (think, speak, move, ...). That is the origin of the vast domain of Artificial Intelligence, which definition for this paper is considered as the following:

Artificial intelligence supplies a collection of techniques to manipulate knowledge in such a manner that new results emerge and new inferences that were not explicitly programmed.

Broadly, two areas can be distinguished in AI: symbolic AI and sub-symbolic AI. The latter comprises neural networks, data-driven techniques, that have had a lot of attention since the last 20 years. The former, symbolic AI, concerns knowledge-based systems and may involve techniques such as rule-based, fuzzy logic, Case-Based Reasoning where the knowledge is

contained in well-defined blocks of previous experience, etc (Vingerhoeds et al. 1995).

In this paper, the focus is on Case-Based Reasoning and ontologies. Case-Based Reasoning was developed under the philosophy that human beings think and reason using analogies and examples, rather than rules (Kolodner 1993). The idea is that one may recall previous similar situations when being confronted with a new problem. Starting from this previous experience (knowledge) ideas can be derived for addressing the new situation. Case-Based Reasoning is therefore an approach in which specific knowledge of previously experienced problem situations is being used to solve a new problem. This is being done by finding similar previous cases and reusing previous experiences. Allowing continuously to add new cases, new pieces of knowledge, Case-Based Reasoning supports incremental, sustained learning, where information from new situations is kept for future use.

Ontologies are formal explicit descriptions of concepts in a domain of discourse, properties of each concept describing its features, attributes and restrictions (Noy and McGuinness 2000). One of the most common goals in developing ontologies is "sharing a common understanding of the structure information among people and software agents" (Gruber 1993). However, there are also several other motivations to create ontology models such as enabling reuse and analysis of domain knowledge or making domain assumptions explicit. Ontologies are powerful tools for knowledge representation.

In this paper, the integration of ontologies with Case-Based Reasoning is being addressed. The goal of such an integration is to make use of the advantages of both approaches. For example, ontologies enable the processing and sharing of knowledge that can be used at different tasks of Case-Based Reasoning systems, such as representing the input problem, enhancing similarity assessments, case representation, case abstraction and case adaptation.

The paper is organised as follows. In the next section, the context is presented, including an overview of CBR, ontologies, and their integration. After that, an-

other section is focused on explaining the use case for which the integration of CBR and ontologies has been developed. Then, the Development section shows how the integration was carried out, followed by some results and a discussion. The paper concludes by summarizing the lessons learnt and providing indications for future work

CONTEXT

The trigger for the work presented in this paper can be found in the development of a Decision-Support System for the design of predictive maintenance systems (Montero Jimenez et al. 2021). Ontologies have been used as formal knowledge representations that support a CBR system. This section provides the theoretical background of both technologies.

Case-Based Reasoning (CBR)

Case-Based Reasoning consists of proposing solutions to problems within a certain domain profiting of the knowledge derived from previous cases, representing previous knowledge. Indeed, one of the characteristics of human learning is to use past experiences as a reference for the future. A complete reasoning cycle in CBR is divided into four phases (Aamodt and Plaza 1994): retrieval, reuse, revise and retain.

The development of CBR systems starts by defining the case structure, a set of variables that will be used to describe the problem and its corresponding solution from the past. The problem attributes or features are used to estimate the similarity between the target case and the cases stored in a case base. Once the features have been established, the next step is to define similarity functions. These are algorithms, with diverse levels of complexity, that when applied to a pair of values of the corresponding variable they return a similarity value number, normally between 0 and 1. Each problem attribute has its own local similarity. An aggregation function is used to consolidate all local similarities in a global similarity value. Each local similarity can receive special weights that multiply the local similarity values when calculating the global similarity. Case-based reasoning is a flexible reasoning paradigm. It is capable to compute similarities and retrieve similar cases from a case base even when only partial information is known from the target case.

In this work, the software *myCBR*¹ was used. The choice for this software was due to the availability of the *Java* source code, which allowed for adapting the system to the needs of the study. Different types of case attributes for similarity definition are available in *myCBR*, of which three have been used in the current research: *String*, *Symbol* and *Integer*. Both *String* and *Symbol* attributes have textual values, whereas for *Sym-*

bol the values are limited to a list of allowed values and fixed similarities among them, and *String* similarity is based on free text. By default, the *String* similarity function assigns a binary similarity to each string but this function can also include further analysis by applying the *Levenshtein* string comparison method that can tolerate typing errors in the string and still compute a similarity value. The function applied to *Integer* attributes is very flexible, and the user can control the shape of the function $\mathbb{Z} \rightarrow \mathbb{R}$. Once the case concept is created and the attributes are defined, a case base can be stored as a list of instances of the concept with values assigned to each one of the attributes. Then, the system will be ready for querying and retrieving cases. A deeper explanation of how to model knowledge in *myCBR* can be found in (Bach et al. 2014).

Ontologies

An ontology defines a set of representational primitives with which to model a domain of knowledge or discourse (Gruber 2009). In a practical sense, ontologies represent a vocabulary of concepts whose basic structure is a hierarchy of classes and subclasses. Instances can be defined as individuals belonging to classes in ontology, so as they will match the common features defining such classes. Important components of ontologies are the *Object Properties* which can establish links between pairs of instances or classes, and *Data Properties*, which can assign information tokens to particular instances. These properties are defined within a certain domain and range, which means that they are restricted, by definition, to particular classes or data types. In general, when referring to ontology entities, that includes classes, individuals and properties. The *Protégé* ontology editor (Stanford University 2016-2020) was used for the development of the ontology for this research.

The initiative of the Semantic Web by the World Wide Web Consortium (OWL Working Group 2012) is to gather as much knowledge as possible into the internet in a format that is readable for computers. For such purposes, the *OWL2* ontology language is used. Standard ontologies like the BFO (Basic Formal Ontology) (International Organization for Standardization 2020) and CCO (Common Core Ontologies) (Rudnicki 2019) are developed and published to be the roots of all the domain-specific ontologies that may be proposed.

Other for the current study relevant aspects of ontologies concern reasoning and queries. A reasoning process can be performed on an ontology to check its logical consistency. This allows verifying, for example, if the instancing of individuals reveals no class contradictions or if the *Object Properties* declarations respect the restrictions of domain and range. Another important aspect of ontology reasoning is the inference of relations; some links could be established between certain entities in the ontology that was not initially explicit but logically deduced from the original ontology

¹Available online. URL: <http://www.mycbr-project.org/>. Last visited June 2021.

structure. Various reasoners for *OWL* language exist, amongst which the *HermiT* reasoner (Glimm et al. 2014), which was used for this study. Queries allows extracting information from ontologies; they are questions that can be posed to get a specific information out of the ontology. Three different procedures or languages to query *OWL* ontologies exist: *SPARQL*, *Description Logics (DL)* and *SQWRL*. The *SPARQL* language allow extracting table structured information from an ontology by querying for entities that match certain conditions and have some kind of relation between them. This type of query does not need a reasoning process. A plug-in is available for *Protégé* to execute *SPARQL* queries. In this study, for the *Java* implementation, the *Jena API* has been used. *Description Logics* allow executing simple queries by using a reasoner and checking at first the ontology consistency. They are available by default in *Protégé* and also accessible with the *OWL API* implementation for *Java*. The *SQWRL* language is based on the *SWRL* rule language, and it allows to execute very accurate and specific queries using a reasoner. Again, a *Protégé* plug-in is available. In *Java* it may be used the *SWRL API* with a *drools* reasoning engine implementation available.

An ontology for the selection and assessment of predictive maintenance models was developed to support the CBR approach (Montero Jimenez et al. 2021). The *OMSSA* (Ontology model for Maintenance Strategy Selection and Assessment) includes all the necessary concepts of the maintenance domain to describe predictive maintenance systems architecture and application cases. The ontology was built as an extension of the standard BFO and CCO ontologies in order to be as general as possible and to possibly be re-used in the future by other ontology developers.

Related work: integration CBR-ontologies

Ontologies and Case-Based Reasoning have been increasingly used together in the last decade. One of the most important aspects of developing CBR systems is the vocabulary framework. Ontologies have played an important role in providing this vocabulary framework for several CBR applications. For example, (Qin et al. 2018) implemented and ontology supported case-based reasoning approach for computer-aided tolerance specification. In (Amaief and Lu 2013) an ontology was integrated with a CBR system for emergency response services. (Recio-Garía and Díaz-Agudo 2007) presented a system that brought together the CBR *Java* framework *jCOLIBRI* and *Description Logics (DL)* to calculate a concept-based similarity values between terms according to their position in the classification structure within the ontology. The same framework was used in (Kowalski et al. 2013) for the domain of logistics adding some variables that were defined in a specific domain ontology. In (Yin et al. 2010), CBR and ontologies are used to test a criminal

investigation system. An ontology is used as a casebase and structure-based similarity values are calculated for case retrieval.

Instantiated ontologies can be used as case bases for CBR implementations as the case structure can be easily represented in the ontology. Ontologies may also help to compute semantic similarity based on ontological similarity, which can be classified into structure-based similarity and feature-based similarity. Structure-based similarity considers the ontology as a graph where concepts are linked by relations (taxonomic or others). Some methods may be proposed to measure the path distance between a pair of concepts to define their similarity value. For example, in (Avdeenko and Makarova 2018) a hierarchical ontology is used to assign similarity values between some pairs of concepts within a specific domain. Feature-based similarity is focused on comparing sets of features of two different terms, established through their property assertions. Feature-based similarity could provide a deeper and more flexible understanding of the domain vocabulary and thus better results in the semantic similarity computation. That is why in this work a feature-based similarity method is used, see also (Sánchez et al. 2012) and (Bai et al. 2008).

The actual integration of ontologies and CBR does not always receive enough attention in application articles. This paper attempts to cover such gap by providing insight on the different possibilities to integrate ontologies and CBR for the development of knowledge-intensive smart systems.

USE CASE: PREDICTIVE MAINTENANCE SYSTEMS DESIGN

Predictive Maintenance (PdM) is a strategy that aims at triggering maintenance actions based on accurate diagnostics or prognostics before an undesired failure occurs. More specifically, predictive maintenance includes monitoring and modelling the system health, estimating the remaining useful life, and detecting and identifying the actual faults. To perform these tasks there exist several types of models that can be used for diagnostics and prognostics purposes (Montero Jiménez et al. 2020). These models analyze the physical variables measured during the operation of the system of interest. For example, in the case of a turbo-machine, some relevant operation variables to consider for predictive maintenance purposes could be the measurements of combustion temperature, pressure and axis vibrations.

A predictive maintenance approach is normally focused on the health state of the system to be maintained or on the incipient failures that may appear during its functioning, sometimes modelling the evolution of these features over time. For example, a predictive maintenance strategy conceived for health assessment may have as main objective to measure the degradation of the system compared to the optimal operation con-

dition that the system had at the beginning. Another possible task in predictive maintenance is to detect or identify automatically failures that are affecting the system and that might perturb its functioning. In addition, the power of predictive maintenance applications increases when considering the capabilities to forecast the evolution of the system health and even predict an estimation of the remaining useful life.

In order to perform all these tasks, three families of models may be considered: data-driven models, knowledge-based models and physics-based models (Montero Jiménez et al. 2020). Data-driven models have an increased popularity during the last years thanks to the current advances in computational power. Statistical models, stochastic models and machine learning are the main model types within data-driven models. In knowledge-based models, previous experiences are used to infer solutions to current problems, in form of for example rules or cases. Physics-based models are very specific for each application case, as they use a physical simulation of the system to perform predictive maintenance functions with the available data. When developing predictive maintenance systems, one of the challenges lies in the selection of the appropriate model for the knowledge representation (a large amount of options) and to position the retained model(s) for the knowledge representation within the systems architecture (Montero Jiménez and Vingerhoeds 2019).

This work is focused on providing a knowledge-intensive system to find the most adequate predictive maintenance solutions for the particular situations. For this purpose, Case-Based Reasoning appears to a suitable technique, as it allows to deal with symbolic information gathered previous predictive maintenance realisations.

INTEGRATION OF ONTOLOGIES AND CBR FOR THE DEVELOPMENT OF A DECISION SUPPORT SYSTEM

In the current study, a domain ontology (OMSSA) (Montero Jimenez et al. 2021), developed with the ontology editor *Protégé* (Stanford University 2016-2020), was integrated with the *myCBR* engine. An instantiated version of OMSSA serves as case base and provides the knowledge for feature-based similarity measures for the *myCBR* engine. The methodology to estimate feature-based similarity was adopted from (Sánchez et al. 2012). The *OWL API* (Horridge and Bechhofer 2011) is used to develop a direct link between the *myCBR* data files *.prj* and the *.owl* ontologies. The *HermiT* reasoner was used to verify ontology database consistency, infer relations and execute “Description Logic” queries for information extracting, necessary for the feature-based similarity functions. Ontology instantiation is carried out by queries that can retrieve data from different sources, such as for example *.csv* files.

Variables definition for the application case

The application case is a Predictive Maintenance Decision-Support System. In this section, some of the implementation details will be presented, so to show how CBR and ontologies can work together. As mentioned before, a first step to implement a CBR application concerns the definition of the variables describing the cases. In the current study, such cases describe previous solutions of Predictive Maintenance (PdM) that have been successfully implemented on different systems of interest. Based on (Montero Jiménez et al. 2020), the main characteristics of each case were described in a systematic set of data fields. Some of the data fields are used as case retrieval variables. Following the logic of *myCBR*, those variables are attributes that must be included in the concept definition:

- *Task (Symbol)*: what is the specific function of the predictive maintenance system (e.g. health modelling).
- *Case study type (Symbol)*: what is the type of maintainable system (e.g. a rotary machine).
- *Case study (String)*: specific system to be maintained (e.g. jet engines).
- *Input type (Symbol)*: list of measurable physical variables (e.g. temperature, pressure and power).
- *Online/Off-line (Symbol)*: predictive maintenance analysis to be done online or offline?
- *Input for the model (Symbol)*: data format type provided to the predictive maintenance module (e.g. signal data).
- *Publication year (Integer)*: the year in which the study was published.

The attributes of type *Symbol* take textual values among a list of allowed values that must be specified for each of them. In addition to these data fields that are important for the case retrieval, other variables are considered as solution attributes of the case and may be exploited once the relevant cases are retrieved. Some of the solution attributes suggested by the decision support system include: *study title*, *publication identifier*, *predictive maintenance model used*, *performance indicators*, etc.

Instancing cases in the ontology

Originally, *myCBR* stores the case base in *.csv* tabular format. This *.csv* tabular format was replaced by an instantiated version of OMSSA. A *Java* code was developed with specific methods to read all data and to make it fit in an ontology structure: classes, instances and *Object Properties/Data Properties* assertions. An *OWL API* implementation in *Java* was needed for such a purpose. The *CCO* ontologies are taken as a base, especially the *Information Entity Ontology* and the *Artifact Ontology* (Rudnicki 2019). Most of the classes

in the ontology are defined as subclasses of the already existing classes in the previously mentioned ontologies. The relations used to form the case structure are those compiled in the Table 1. Properties written in **blue** are included in *CCO*, those in **green** are included in *OBO Relations Ontology* from *BFO* (International Organization for Standardization 2020) and those in black have been specifically created in *OMSSA*. Parentheses contain the parent ontology, considering that a property can be a sub-property of a parent.

Table 1: *Object Properties* in OMSSA.

Property	Domain	Range
<i>designates</i>	<i>Designative Information Content Entity</i>	No specific range
<i>describes</i>	<i>Descriptive Information Content Entity</i>	No specific range
<i>is carrier of</i>	<i>Independent continuant</i>	<i>Generically dependent continuant</i>
<i>has part</i>	No specific Domain	No specific range
<i>has title</i> <i>(is carrier of)</i>	<i>Predictive maintenance article</i>	<i>Article title</i>
<i>has identifier</i> <i>(is carrier of)</i>	<i>Predictive maintenance article</i>	<i>Article identifier</i>
<i>has publication year</i> <i>(is carrier of)</i>	<i>Predictive maintenance article</i>	<i>Publication year</i>
<i>has synchronization</i> <i>(has quality)</i>	<i>Predictive maintenance system module</i>	<i>Module synchronization</i>
<i>has predictive maintenance function</i> <i>(has function)</i>	<i>Predictive maintenance system module</i>	<i>Predictive maintenance module function</i>

The very basic structure of a case starts with an instance of a *Predictive maintenance case* (subclass of *Designative Information Content Entity*), designating an instance of a *Predictive maintenance system module* (subclass of *Information Processing Artifact*). The items to be maintained are instances of the different subclasses in *Maintainable item* (equivalent to *Artifact*). Then, the models are instances of the type subclasses in the *Predictive maintenance model*. The models operate over a set of variables that are recovered from the operation of the item. These variables are instances of *Data variable* (subclass of *Descriptive Information Content Entity*). As the instances of variables are physical concepts (for example, the temperature), they can be re-used in more than one case of the database. For each one of the different measurable magnitudes that are mentioned in the database, there exists one single instance of *Data variable*. The functioning of the module and the models used for a certain case are explained in the corresponding *Predictive maintenance article*. The title and the identifier of those articles are assigned to instances *Article title* and *Article identifier* respectively (both are subclasses of *Designative Information Content Entity*). In summary, a maintenance module uses one or more models to perform a predictive maintenance diagnosis or prognostic task on a certain system (*Maintainable*

item) based on the data of measured magnitudes recovered during the system operation. This is identified as a predictive maintenance case which is described in a specific research article (see also Figure 1, please note that some elements were omitted for clarity reasons).

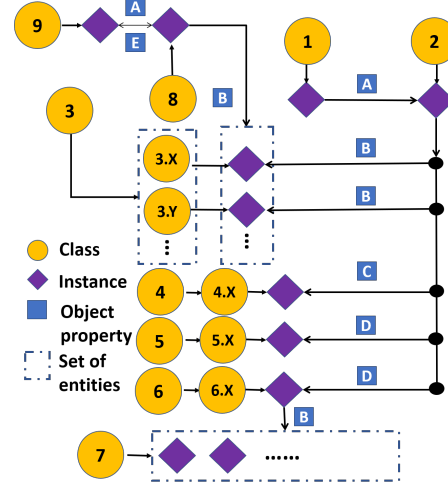


Figure 1: Partial schema of the entities participating in the case representation in the ontology.

- Yellow nodes in Figure 1 represent OMSSA classes:
 - Predictive maintenance case*
 - Predictive maintenance system module*
 - Predictive maintenance model*
 - Predictive maintenance function*
 - Maintainable item*
 - Maintainable item record*
 - Data variable*
 - Predictive maintenance article*
 - Article title*
- The labelled edges in Figure 1 represent OMSSA object properties:
 - designates*
 - is carrier of*
 - has predictive maintenance function*
 - has part*
 - has title*
- The classes with a tag *N.X* are subclasses of the parent class *N*.

In order to assign the textual values to the article title, the article identifier and the reference tag of the case, which will be materialized as *Designative Information Content Entities* in the ontology, the property *information has text value* is used.

Similarity functions

When defining a similarity function, the type of variable is a major point to consider. The similarity functions of a *Symbol* variable are specified in *myCBB* with a set of pairs of textual values with a similarity value assigned between 0 and 1. These values are estimated according to experimental and physical similarity criteria for the case variables *Case study type*, *Input for the model* and *Online/offline*. One of the objectives of this study is to assess feature-based semantic methods to obtain similarity values, so the procedure proposed (Sánchez et al. 2012) was adapted to the application case and the relations existing in the ontology. The basis of this method is to compare two sets of elements A and B. The following formula is used to obtain a normalized similarity of the sets:

$$\text{Similarity} = 1 - \log_2 \left(1 + \frac{A/B + B/A}{A/B + B/A + A \cap B} \right) \quad (1)$$

In the equation above, A/B means the difference between sets A and B, so the elements from A that do not belong to B. $A \cap B$ is the intersection of A and B.

The case feature *Input type* is a list of variables that are monitored during the operation of the system to be maintained. These variables are represented in the ontology with instances of the class *Data variable*. The above-mentioned method is used to compare the set of values in the query case with all cases in the case base. The attribute *Input type* in *myCBB* will store the list of variables as a textual value separated by commas. When executing a query, a list of variables names is proposed, so the algorithm first has to add this list to the allowed values of the attribute. Then, the similarity values are assigned to all the pairs formed by the query string and all the textual values existing in the database. The variable names are separated for each of them and the formula in Equation (1) is applied. Also, a *Levenshtein* method function was implemented to manage slight misspelling in the query string.

Whilst the mathematical method to calculate the ontological similarities for the attribute *Task* is the same as before, the sets of elements to be compared are obtained in a different way. The relation *function uses model* is defined as an inference resulting of the property chain *is predictive maintenance function of* and *is a carrier of* to link instances of functions with instances of models (ontology class *Predictive maintenance model*). This means that the database will be queried automatically for information on the types of models that were used for each type of task concerning predictive maintenance. That allows estimating a semantic similarity between the different subclasses of functions.

The types considered in the ontology are: *Fault detection*, *Fault feature extraction*, *Fault identification*, *Future state forecast*, *Health assessment*, *Health*

modelling, and *Remaining useful life estimation*. The algorithm will query for each of the mentioned subclasses all the instances of models that are linked to their individuals through the inferred relation *function uses the model*. Each of the instances of models belongs to one of the several subclasses of the *Predictive maintenance model* that were defined in the ontology. The purpose is to list all the model types that have instances linked to any instances of each of the function types.

This is how sets of model types related to functions are compared using the method of Equation (1) to get similarity values. An implementation of the *HermiT* reasoner in *Java* has been used to check the ontology consistency, infer the relations and extract the required information through *Description Logics* queries.

From a similarity point of view, using the variable *Publication year*, referring to the year when a case study was published, the more recent an article is, the more accurate and relevant is supposed to be. A simple *Integer* similarity function has been tested with a similarity of 1 for cases published 5 years ago or less and a constant slope descending to a similarity of 0 at 40 years.

Finally, the default *myCBB* string comparison function with *Levenshtein* method has been used for the *String* attribute *Case study*. To obtain the global similarity value of a case from the similarities of the variables, *myCBB* allows using a simple weighted sum or a euclidean average. The weights may be adjusted, but for this study, they have been set to 1 for all the variables.

Procedure to link ontologies and myCBB

The default method for importing databases into *myCBB* is via *.csv* files. As one of the main goals of this project was to create a direct link between ontologies and *myCBB* and use ontologies as database holders, *SPARQL* queries were used to extract the information as organized tabular data and load it automatically into a *myCBB* project file. Using the *Jena API* tool, single string queries with multiple statements were formulated to obtain a query results table with entities names or *Data Properties* content in which the rows correspond to cases and the columns to the variables defining those cases. This table is then stored inside a *myCBB* project file. The procedure may require dividing the query pieces to be executed sequentially when the volume of data is too large, so in that case, the partial results would be simply merged in one data table. The developed application allows loading a case base ready to be used by *myCBB* directly from an ontology *.owl* file following data translation rules specified in *SPARQL* language.

RESULTS AND DISCUSSION

Keeping in mind the objective to propose an integration procedure between *myCBB* and ontologies,

different kinds of queries were tested in *Java* for *OWL* ontologies. *SPARQL* queries seem to be best placed for such integration, having as major advantages to having a fast execution without the using of a reasoner and that the outputs obtained are immediately organized in tabular data. This makes it easy to introduce the data in the *myCBR* project files.

For the information extraction for semantic similarity values definition, *SQWRL* queries were tested, but the obtained performance was low in terms of calculation speed and memory consumption. For the reasoner implementation tested for *SQWRL* queries, it was found that again a slow process resulted, potentially due to the complexity of the case base. In the end, in this study, *Description Logics* queries were used, only requiring to run the reasoner once for the execution of series of queries.

A feature-based similarity method has been tested in two of the variables describing the study case: *Input type* and *Task*. For the case retrieval variable *Input type*, the feature-based method seemed to be suitable criteria to compare sets of elements, suggested in (Qin et al. 2018). The in this way obtained results have been shown to be relevant and useful, demonstrating that the integration procedure is practical for defining accurate similarity values automatically. As to the similarity values between pairs of predictive maintenance tasks or functions, the corresponding similarity values matrix is shown in Table 2, of which the letters should be read considering:

- A) *Fault detection*
- B) *Fault feature extraction*
- C) *Fault identification*
- D) *One step future state forecast*
- E) *Multiple steps future state forecast*
- F) *Health assessment*
- G) *Health modelling*
- H) *Remaining useful life estimation*

Table 2: Similarity values matrix for predictive maintenance functions.

	A	B	C	D	E	F	G	H
A	1	0.033	0.225	0.02	0.03	0.093	0.079	0.063
B	0.033	1	0.018	0	0	0.02	0.033	0.023
C	0.225	0.018	1	0	0	0.082	0.07	0.073
D	0.02	0	0	1	0.061	0.025	0	0.034
E	0.03	0	0	0.061	1	0.037	0.015	0.03
F	0.093	0.02	0.082	0.025	0.037	1	0.13	0.076
G	0.079	0.033	0.07	0	0.015	0.13	1	0.078
H	0.063	0.023	0.073	0.034	0.03	0.076	0.078	1

As can be seen, the similarity values for non-identical concepts are overall very low meaning that the instances in OMSSA seldom show a model that has been used

for two different tasks. Note the existence of one pair (*Fault detection-Fault identification*) that is much higher (over 10% similarity) in comparison to the rest; this is expected as classification models can be used for fault detection and also for fault identification purposes. Another interesting result concerns the pair *One step future state forecast-Multiple steps future state forecast*, basically the latter being an extension of the former, where the similarity could be expected to be close to 1. Both concepts are subclasses of the same predictive maintenance function in the ontology *Future state forecast*. Hence, from a global point of view, the analysis of the results suggest that there is possibly some aspect about the data or the similarity definition that makes it difficult to obtain really meaningful values. A first explanation hypothesis concerns the high degree of specialization of predictive maintenance models in the ontology. The class *Predictive maintenance models* have been automatically filled up with as many subclasses as specific types of models were reported in the literature and therewith declared in the database. Grouping all those types of models into less specific types could have changed the outcome. Another option would be to use a bigger case base, which in itself goes against the generally assumed guidelines for Case-Based Reasoning to work with relatively small case bases. This needs to be investigated in the next steps of the study.

So, the results suggest that some modifications could be needed, specifically in what concerns the similarity between the types of predictive maintenance models considered. However, the value of what has been obtained is not limited to the results in themselves, but also the fact that the integration procedure for ontologies and CBR has been validated so as to be exploited and improved to achieve a superior level of performance for the decision support system.

CONCLUSION AND FUTURE WORK

This paper presents an approach to integrate ontologies and Case-Based Reasoning for a practical application case base for the development of predictive maintenance systems. The automatic transmission of the data in tabular format to ontologies has shown satisfactory results. The implementation of an ontology reasoner and some semantic feature-based similarity methods using *Description Logics* was possible thanks to the *OWL API*. It helped to improve the performances and accuracy of the system for this application and others to be tested in the future. The integration of *myCBR* and *.owl* ontologies through *SPARQL* queries using the *Jena API* is functional and reusable for other applications

This study has especially focused on the retrieval of cases, but the use of ontological knowledge could also be powerful to improve the rest of CBR functions as well. There may be good potential for future work to consider the complete CBR cycle and to automatize all CBR tasks (as suggested in (Mántaras et al. 2005)).

In addition, the study to improve ontological similarity is interesting; more complex and accurate semantic similarity functions could be implemented. In particular, exploiting the ontological reasoner inference could help to make deeper use of the knowledge in the ontology. Another potential improvement may come from taxonomic similarity functions that take better advantage of the hierarchical structure of an ontology. Additional validation work should aim for testing the validity of the recommendations given by the system within the application domain at hand (the design of predictive maintenance system) and to extend with developments on other application domains.

REFERENCES

- Aamodt A. and Plaza E., 1994. *Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches*. AI Communications IOS Press, Vol. 7: 1, pp. 39–59.
- Amailef K. and Lu J., 2013. *Ontology-supported case-based reasoning approach for intelligent m-Government emergency response services*. *Decision Support Systems*, 55, 79–97. ISSN 01679236. doi:10.1016/j.dss.2012.12.034.
- Avdeenko T.V. and Makarova E.S., 2018. *Knowledge Representation Model Based on Case-Based Reasoning and the Domain Ontology: Application to the IT Consultation*. *IFAC-PapersOnLine*, 51, no. 11, 1218–1223. ISSN 2405-8963. doi:https://doi.org/10.1016/j.ifacol.2018.08.424. URL <https://www.sciencedirect.com/science/article/pii/S2405896318315519>. 16th IFAC Symposium on Information Control Problems in Manufacturing INCOM 2018.
- Bach K.; Sauer C.; Althoff K.D.; and Roth-Berghofer T., 2014. *Knowledge Modeling with the Open Source Tool my-CBR*. vol. 1289.
- Bai Y.; Yang J.; and Qiu Y., 2008. *OntoCBR: Ontology-based CBR in context-aware applications*. ISBN 978-0-7695-3134-2, 164–169. doi:10.1109/MUE.2008.56.
- Glimm B.; Horrocks I.; Motik B.; Stoilos G.; and Wang Z., 2014. *Hermit: An Owl 2 Reasoner*. *Journal of Automated Reasoning*, 53. doi:10.1007/s10817-014-9305-1.
- Gruber T., 2009. *Ontology*. Springer US, Boston, MA. ISBN 978-0-387-39940-9, 1963–1965. doi:10.1007/978-0-387-39940-9.1318.
- Gruber T.R., 1993. *A translation approach to portable ontology specifications*. *Knowledge Acquisition*, 5, no. 2, 199–220. ISSN 10428143. doi:10.1006/knac.1993.1008.
- Horridge M. and Bechhofer S., 2011. *The owl api: A java api for owl ontologies*. *Semantic Web*, 2, 11–21. doi:10.3233/SW-2011-0025.
- International Organization for Standardization, 2020. *ISO/IEC FDIS 21838-2.2 Information technology — Top-level ontologies (TLO) — Part 2: Basic Formal Ontology (BFO)*. Available online. URL: <https://www.iso.org/standard/74572.html>.
- Kolodner J., 1993. *Case based reasoning*. Morgan Kaufmann. ISBN 978-1558602373.
- Kowalski M.; Klüpfel H.; Zelewski S.; and Bergenrodt D., 2013. *Integration of Case-Based and Ontology-Based Reasoning for the Intelligent Reuse of Project-Related Knowledge*. ISBN 978-3642328374, 289–299. doi:10.1007/978-3-642-32838-1.31.
- Montero Jimenez J.J.; Vingerhoeds R.; and Grabot B., 2021. *Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning*. IEEE ISSE 2021.
- Montero Jiménez J.J.; Sebastian S.; Vingerhoeds R.; Grabot B.; and Salas M., 2020. *Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics*. *Journal of Manufacturing Systems*, 56, 539–557. doi:10.1016/j.jmsy.2020.07.008.
- Montero Jiménez J.J. and Vingerhoeds R., 2019. *A System Engineering Approach to Predictive Maintenance Systems: from needs and desires to logical architecture*. 1–8. doi:10.1109/ISSE46696.2019.8984559.
- Mántaras R.; Mcsherry D.; Bridge D.; Leake D.; Smyth B.; Craw S.; Faltings B.; Maher M.; Cox M.; Keane M.; Aamodt A.; and Watson I., 2005. *Retrieval, reuse, revision and retention in case-based reasoning*. *Knowledge Eng Review*, 20, 215–240. doi:10.1017/S0269888906000646.
- Noy N.F. and McGuinness D.L., 2000. *Ontology Development 101: A Guide to Creating Your First Ontology*. Stanford University. Available online. URL: https://protege.stanford.edu/publications/ontology_development/ontology101-noy-mcguinness.html. Accessed June 2021.
- OWL Working Group, 2012. *OWL 2 Web Ontology Language*. World Wide Web Consortium. Available online. URL: <http://www.w3.org/TR/2012/REC-owl2-overview-20121211/>. Accessed March 2021.
- Qin Y.; Lu W.; Qi Q.; Liu X.; Huang M.; Scott P.; and Jiang X., 2018. *Towards an ontology-supported case-based reasoning approach for computer-aided tolerance specification*. *Knowledge-Based Systems*, 141, 129–147. doi:10.1016/j.knsys.2017.11.013.
- Recio-Garíá J. and Díaz-Agudo B., 2007. *Ontology based CBR with jCOLIBRI*. ISBN 978-1-84628-665-0, 149–162. doi:10.1007/978-1-84628-666-7.12.
- Rudnicki R., 2019. *An Overview of the Common Core Ontologies*. CUBRC Inc, Buffalo, NY. Available online. URL: https://www.nist.gov/system/files/documents/2019/05/30/nist-ai-rfi-cubrc_inc_004.pdf.
- Stanford University, 2016-2020. *Protégé official website*. Available online. URL: <https://protege.stanford.edu/>. Last visited June 2021.
- Sánchez D.; Batet M.; Isern D.; and Valls A., 2012. *Ontology-based semantic similarity: A new feature-based approach*. *Expert Systems with Applications*, 39, no. 9, 7718–7728. ISSN 0957-4174. doi:https://doi.org/10.1016/j.eswa.2012.01.082.
- Vingerhoeds R.A.; Janssens P.; Netten B.D.; and Aznar Fernández-Montesinos M., 1995. *Enhancing off-line and on-line condition monitoring and fault diagnosis*. *Control Engineering Practice*, 3, no. 11, 1515–1528. ISSN 09670661. doi:10.1016/0967-0661(95)00162-N.
- Yin Z.; Gao Y.; and Chen B., 2010. *On development of supplementary Criminal analysis system based on CBR and Ontology*. In *2010 International Conference on Computer Application and System Modeling (ICCSM 2010)*. vol. 14, V14–653–V14–655. doi:10.1109/ICCSM.2010.5622227.

DSS code Guide

The content in this appendix corresponds to a user guide developed in ISAE-SUPAERO for the ontology-enabled CBR engine for predictive maintenance component selection. Hugo Munoz Hernandez, Juan José Montero Jimenez, Rob Vingerhoeds. “Decision Support System Code Guide: Application to Predictive Maintenance Systems Design”.

Intentionally left blank

Decision Support System Code Guide

**Application to Predictive Maintenance
Systems Design**



Hugo MUNOZ HERNANDEZ
Juan Jose MONTERO JIMENEZ
Rob VINGERHOEDS

July, 2021

Contents

1 Installation and configuration	3
2 Basic usage	4
2.1 <i>SPARQL</i> queries	4
2.2 <i>SQWRL</i> queries	4
2.3 Load data in a table format (<i>.csv</i>) to an ontology file (<i>.owl</i>) using <i>CSVtoOntologyExec</i>	5
2.4 Load data from an ontology file (<i>.owl</i>) to a table format <i>.csv</i> file using <i>OntologytoCSVExec</i>	6
2.5 Preparing project case base and similarity values for <i>myCBR</i> with <i>myCBRSetting</i>	11
2.6 Query and retrieval using the <i>GUI</i> 's	12
2.6.1 <i>GUI2</i>	13
2.6.2 <i>GUI3</i>	14
2.7 The <i>javadoc</i>	15
2.8 Management of the <i>data</i> folder	16
3 Dependencies	17

1 Installation and configuration

For the installation procedure described here it is assumed that the user has installed a version of *Eclipse* that supports at least *Java* 8. Under the previous condition, the following steps may be followed to ensure the correct installation:

1. Download the *.zip* compressed folder of the project and save it in a selected local address in your computer.
2. Make right click over the compressed folder and use *Extract Here*, a decompressed folder with the same denomination has been created. Inside this folder, another folder named *InternshipProject* may be found.
3. Open *Eclipse* and browse to select *InternshipProject* as the working folder.
4. Once *Eclipse* has been initialized, go to *File* → *Import* → *General* → *Existing Projects into Workspace* → *Browse* and select the folder *InternshipProject*. The project will be built in the current workspace of *Eclipse*. The recognition of the folder as an *Eclipse* project is possible because of the file *.project* in the same folder.
5. In the case that some referenced libraries (*.jar*) seem to be missed, right click on *InternshipProject* at the workspace menu on the left → *Build Path* → *Configure Build Path* → *Libraries* → click on *Classpath* in the list below and now the option buttons on the right are available. Delete the paths that are indicated as erroneous and click *Add JARs* on the left to reintroduce the libraries that are missed. Browse to the *external-libs* folder and select the *.jar* files that have to be restored, then just click OK and *Apply and Close*. However, this problem is not likely to occur as the *.classpath* file store the path to all the dependencies of the project. Another possible solution to the problem if this existed would be to open the *.classpath* file with a plain text editor and change the paths that are referencing a local folder in another user computer to a general path starting at the folder *external-libs* of the project.

Now that the code is installed, some configurations are needed to start execution. There is a class named *AppConfiguration* in the *User* package which only contains public static attributes and a couple of methods to build variables. These attributes are read and used by other classes of the project, so they are stored together in the accessory class *AppConfiguration* to be accessed (not modified) by other pieces of code in the application. The very first change that the user must make in the mentioned class before executing the code is setting the *data'path* attribute to the actual local path of the *data* folder of the project. From that point, the attributes on *AppConfiguration* should be updated to the file names that are wanted to be used inside the *data* folder (*.csv*, *.owl*, *.prj*, etc). See the *javadoc* of the project and the comments in the source code of *AppConfiguration* to get known about the meaning of the attributes.

2 Basic usage

The usage of this application will be performed simply by the execution of java classes from Eclipse (or any other IDE). It may also require the manipulation of files (ontology files, *.csv*, *myCBR* project files, etc) in the designated file folders. Unless a modification of the source code is needed for some reason, there are 5 executable classes in the project that concern to the user at this moment: *CSVtoOntologyExec*, *myCBRSetting*, *SPARQL*, *GUI2*, *GUI3*, *OntologytoCSVExec* and *SWRLAPIexec*.

2.1 *SPARQL* queries

A simple executable class has been added to the project to execute *SPARQL* queries on the working ontology in the same way that they are available in *Protégé*. The *Jena* tool (see Section 3 for more information) is used for that purpose. The *SPARQL* queries allows the user to obtain an accurate required information from the ontology. A general reference for the *SPARQL* language can be found at [11]. The user just needs to write the query in a correct syntactical form and execute the class file to get the results of the query on the console. These queries are very quick in their execution as they do not need to run a reasoner and check the ontology consistency. Even if the current implementation in the project is accessory, the *SPARQL* queries could be potentially used to extract information from an ontology for the definition of ontological similarity values.

2.2 *SQWRL* queries

The implementation of the *SWRL API* together with the reasoning engine *drools* allows to add *SWRL* rules to the working ontology execute *SQWRL* queries. See Section 3 for more information. The *SQWRL* language, which is based on the *Semantic Web Rule Language* (*SWRL*), opens the possibility to accurate queries with a relatively simple syntax. There is a paper [21] where the creator of this language give an introduction to the main rules and basic syntax of the queries. An inconvenient has been found for the use the *SQWRL* queries for the application case in the current research project: the reasoner must be run once for each query, so, when working with ontologies having a big number of individuals declared, this process can require an important amount of *RAM* memory in the computer and it can last long if many queries are wanted to be executed. That is why, at this moment, the direct querying through the *OWL API* is used for extracting information from an ontology, as it only requires to run the reasoner (*HermiT*) once before executing as many queries as desired. Indeed, the example class in the project *SWRLAPIexec* allowing to execute *SQWRL* queries may not work correctly due to the compatibility problems of *SWRL API* with the latest version of *OWL API*. See Section 3. However, as it is